The In-Place Program Is Its Own Specification

Mutable borrows inside a dependent type theory

Abstract. Verified systems code is usually written three times: an efficient imperative program, a pure model that the proofs are about, and a refinement proof joining them. We present Ochr, a core calculus that puts Rust-style mutable borrows inside a Lean-style dependent type theory, so that the in-place program itself appears in types and proofs. Definitional equality runs programs on symbolic inputs. A call stuck on an unknown value is *closed off* into *sealed programs*, source programs that compute its result and the final contents of the places it borrowed, in the role of Aeneas's backward functions. An observational equality `Id` compares two computations by what they return and what they write, and computes to ordinary equations, so theorems about in-place code are proved by structural recursion. We give the calculus in full, a Lean checker that checks every program in the paper, and two case studies: Aeneas's hash map, verified with no model and no agreement proof in a development 36% smaller in lines than Aeneas's (32% without its dead code), and an in-place quicksort. Ochr covers mutable borrows only, without shared borrows, loops or borrows in data. Soundness is proved on paper for a first-order fragment, conditional on two stated assumptions; consistency of the full calculus is a conjecture, tested by a differential fuzzer.

1 Introduction

Here is an in-place addition on Peano numerals. It walks down the successors of `*x` until it reaches the final `Z` and moves `y` into that spot:

```
AddM(x : &Nat, y : Nat) : Unit by x :=
  match *x { Z => *x := y, S p => AddM(&p, y) }
```

`&Nat` is a mutable borrow in the sense of Rust: a unique, temporary right to read and write a `Nat` owned by someone else. In the successor case the pattern variable `p` names the predecessor field in place, and `&p` reborrows it for the recursive call; `by x` says that the recursion is structural on `x`. Here is a theorem about `AddM`, stated and proved in the same language:

```
AddMZero(x : &Nat) : Id(Unit, AddM(x, 0), ()) by x :=
  match *x { Z => refl, S p => AddMZero(&p) }
```

The statement says that running `AddM(x, 0)` is indistinguishable from doing nothing: both return `()`, and both leave the number behind `x` as it was. The proof is structural recursion and nothing else. There is no pure model of `AddM`, no refinement relation, no loop invariant, no separation-logic assertion and no translation into another language. The checker accepts the recursive call as a proof of the successor case by running both sides of the equation on a symbolic input and comparing what they leave behind.

This paper is about the type theory that makes that possible.

1.1 The two-language problem

Verified software is usually built in one of two ways. In the first, everything is written in a pure proof assistant such as Lean or Coq, and performance comes from the compiler's ability to turn functional updates into in-place ones where it can prove the old value is dead. In the second, the efficient program is written in an imperative language, and a separate pure specification is related to it by a refinement proof: Verus [30] and Creusot [15] check Rust against specifications discharged by SMT, VeriFast checks C against separation-logic contracts, and Aeneas [23] translates safe Rust into a pure Lean program about which the user then proves theorems.

Both routes keep two artifacts. In the second route this is visible: there is a program and there is a specification, and the proof is about their relationship. In the first route it is hidden but present: the program the proof is

about is not the program that runs, and the gap is bridged by an optimisation the programmer cannot state or rely on.

Aeneas comes closest to closing the gap. Its insight is that Rust's ownership discipline makes mutation equivalent to state passing: a function taking `x : &mut T` is a pure function returning, besides its result, a *backward function* that computes the final value of `*x`. Aeneas performs this translation once, eagerly, for a whole program, and the user proves theorems about the translated Lean code, where the backward functions appear as ordinary definitions. The user therefore works in two languages at once, and must understand how one maps onto the other.

We show that this translation can be performed **inside the type checker, lazily, as definitional equality, and in the source language itself**.

Two lines of work already use one language. Low* [45] writes low-level code in F* against an explicit memory model and reasons about it with pre- and postconditions, and Cogent [43] gives in-place code a purely functional semantics through linear types and generates that semantics for proofs. The claim behind Ochr is stronger: in one system where the in-place algorithm itself appears in types and is unfolded by conversion, a development needs no model and no agreement proof, and so, even without automation, is of comparable size to the three pieces of the usual approach, a specification, an implementation and a proof that they agree (Section 8.2). This paper presents the calculus that makes such developments possible, and the evidence that it is sound.

1.2 Three ideas

Observational equality. The type `Id(A, t, u)` compares two computations `t` and `u` of result type `A`, run from the current state. It holds when they return equal values and leave equal contents in every place they may write. `Id` is a derived proposition, with no introduction or elimination rules of its own: at a given environment it computes to a conjunction of ordinary equations between two *observations*, the tuples of result and final contents, and these compute further by the structure of the data, in the manner of observational type theory [5]. The places a computation may write are read off its syntax, so they are an output of the computation like its result. This is the sense in which Ochr has no pure/impure divide: any program may appear in a type, and what it does to its environment is part of what the type talks about.

Closing off. Definitional equality in Ochr is normalisation by a deterministic symbolic machine. On symbolic inputs the machine gets stuck, and a stuck effectful call, unlike Lean's stuck `Nat.add n 0`, cannot name itself, because part of what it produces lives in the places it borrowed. We *close off* such a call: its result and the final content of each borrowed place become *sealed programs*, closed source programs such as `⌈let c = σ; AddM(&c, 0); c⌉`, the pure wrapper a programmer would write anyway, which run again when a case split refines `σ`. They play the role of Aeneas's backward functions for one call (Section 4).

Induction hypotheses at the call site. When the checker types a recursive call such as `AddMZero(&p)`, it evaluates the callee's statement in the caller's environment, where the statement is observed through the caller's data, successor included, so the induction hypothesis arrives in the form of the goal (Section 2). Evaluation thus performs the frame argument that a proof about a translated program makes by hand, and soundness rests on the frame property that ownership provides: a call can affect only what it is passed.

1.3 The central difficulty: two paths must agree

Pédrot and Tabareau's fire triangle [48] shows that a type theory with substitution of arbitrary terms, dependent elimination and observable effects is inconsistent. Their observable effects concern closed terms, and ownership rules them out: a closed Ochr program can mutate only places it created itself, so it behaves like the value it computes. For open terms Ochr keeps dependent elimination and lets only values into types, sealed programs among them, as in their call-by-value analysis (Section 9). The price is that a statement is evaluated along two paths, once at a definition's most general call, where stuck calls close off, and again directly at each instance, and any decision the two paths take differently can prove `False`. The rules that keep them in agreement follow one principle, that no decision which changes a result is read from a normal form (Section 6.3). Finding them has been the main difficulty of the design: each was forced by a counterexample, a closed proof of `False` or an accepted program that goes wrong.

1.4 Contributions

- A core calculus, Ochr, combining mutable borrows with a dependent type theory with inductive definitions and a universe of proof-irrelevant propositions, whose logical connectives are themselves inductive definitions and whose definitional equality unfolds imperative code (Section 3, Section 4).
- *Closing off*, which gives every stuck effectful call a neutral form made of sealed source programs, including calls that return borrows (Section 4).
- An observational equality between computations, a derived proposition that computes to ordinary equations (Section 5).
- The principle that keeps the two evaluation paths of a type theory whose conversion is an effectful evaluator in agreement, and the side conditions it yields, each with the counterexample that forced it (Section 6).
- The properties Ochr is designed to have, with their evidence (Section 7): soundness of a first-order typed fragment, proved on paper conditional on naturality and on invariants of the machine, both stated as assumptions; consistency and the agreement of the two paths are conjectures, with a set-theoretic model sketched and a differential fuzzer; nothing is mechanised, and termination of type checking is open.
- An executable checker in Lean 4 that checks every example in the paper, with a regression test for every false proof found while designing the calculus, most beside an accepted near-miss; and two case studies, a hash map compared with Aeneas's development and an in-place quicksort (Section 8).

1.5 Scope

Ochr is a core calculus, and it covers a small part of Rust (Figure 1). Its borrow checking is also not modular in the way Rust's is. Checking is by running, so whether a caller checks can depend on its callee's body and on which of its inputs are concrete: a callee that gets stuck leaves every borrowed argument borrowed until its returned borrow ends, while one that runs to completion releases them. A call to an opaque definition, or to a function parameter, is stuck at once and behaves according to its signature alone. Conversion likewise unfolds callees, so extracting a helper function can change which equations hold by `refl` (Section 6).

2 Ochr by example

This section introduces Ochr through its running examples. Everything is informal here; Section 3 onwards makes it precise, and Figure 2 collects the notation.

Rust feature	**In Ochr**
`&mut`, reborrowing	Yes, with exclusivity enforced by running the program (Section 4).
Returned borrows	Yes, with one region per call: the hole goes into every borrowed argument, so a call that returns a borrow into its first argument keeps its second borrowed too, where Rust would accept a lifetime on the first alone.
Shared borrows `&T`	No.
Loops	No; structural recursion only.
Borrows in data	No: no `Option<&mut T>`, `iter_mut` or `split_at_mut`, and no pair of borrows as a result.
Two-phase borrows	No: an argument that reads a place an earlier argument borrows needs a temporary.
Generic `&A`	Yes, for `A : Type₀`, function types included, so a generic `swap` checks (Section 3).
`'static` borrows	No: a returned borrow derives from a borrow argument (Section 10).
Moves, `Copy`, `clone`	Yes: runtime reads of non-copy data move, `clone` copies, and closures follow `Fn` (Section 4).
Integers, arrays	Peano numbers only; arrays as a library, borrowed in parts only through a continuation, and growable vectors as a user type with a dependent field (Section 8.3).
`unsafe`, interior mutability	No.

Figure 1: The part of Rust that Ochr covers.

σ, σ'	an *abstract value*: an unknown value of a known type, as a free variable in Lean
⌈t⌉	a *sealed program*: a closed source program whose run is stuck; a neutral value
borrow₀ v, loan₀	a borrow holding v, and the placeholder it left where it was taken
⊥	a place whose content was moved out
⋆	the value of every proof
{ c ↦ v | x ↦ w }	an environment: bindings, with frames separated by |, oldest first
generic call	the most general call of a definition, at which it is checked (Section 6)
⟦t⟧	the *observation* of t: its result and the final contents of the places it may write

Figure 2: Notation used from Section 2 on.

2.1 Imperative code is evaluated inside types

Places are variables x, dereferences *p and fields such as the predecessor field p.0 of a number. A match on a place binds its pattern variable to a *sub-place*, not a copy: in match *x { Z => …, S p => … }, p is (*x).0.

Add wraps AddM behind a pure interface. It owns its first argument, lends it to AddM, and returns it:

```
Add(x : Nat, y : Nat) : Nat := AddM(&x, y); x
```

Because definitional equality is evaluation, Id(Nat, Add(2, 3), 5) holds by refl: the checker runs Add(2, 3), which runs AddM in place on its own x, and compares the result with 5.

The machine that does this is the symbolic semantics of Aeneas's low-level borrow calculus [23]. An environment Ω maps variables to values; borrowing x moves its content v into the borrow, written borrow₀ v, and leaves a *loan* loan₀ behind; ending the borrow moves the content back.

2.2 Symbolic inputs, and what a stuck call leaves behind

To reason about all numbers at once, the checker runs programs on *abstract values* σ, which play the role of Lean's free variables. Running Add(σ, 0) borrows x ↦ σ and calls AddM(borrow₀ σ, 0), whose body immediately gets stuck: it must match on σ.

In a pure type theory a stuck term is its own normal form. Here the call has also produced something that lives in the environment, namely the final content of the place it borrowed, and that content needs a name. We *close off* the call: the call returns, and the loan it held is filled with a *sealed program*, a closed source program that owns its state and computes that content (the call's result, of type Unit, is ()):

```
{ x ↦ loan₀ } ⊢ AddM(borrow₀ σ, 0)
   →  { x ↦ ⌈let c = σ; AddM(&c, 0); c⌉ } ⊢ ()
```

Write N(σ) for ⌈let c = σ; AddM(&c, 0); c⌉. It is a neutral value, like a stuck Nat.rec in Lean, and it is literally the body of Add(σ, 0): the sealed program *is* the pure wrapper, generated on demand. It computes what Aeneas's backward function for AddM computes [23], without leaving the source language (Section 4 says for which fragment). When a later case split refines σ to S(σ'), N(S(σ')) runs again, now makes progress through one successor, closes off the inner recursive call, and normalises to S(N(σ')).

2.3 Equality between computations

The type Id(Unit, AddM(x, 0), ()) compares two computations. It is evaluated at the environment where it is formed. Each side runs on its own copy of that environment; the result and the final contents of the places the side may write form its *observation*; and Id computes to ordinary equality between the two observations. A definition taking x : &Nat is checked at its *generic call*, its most general call AddMZero(&c) from { c ↦ σ }, where the fresh owned place c stands for whatever a caller lends. So the place that may be written is c, the owner of x's loan:

```
⟦AddM(x, 0)⟧  =  ((), N(σ))          // the call is stuck, so c receives the sealed program
⟦()⟧          =  ((), σ)             // nothing happens; ending x's borrow returns σ to c

Id(Unit, AddM(x, 0), ())  ≡  Eq(Unit, (), ()) ∧ Eq(Nat, N(σ), σ)  ≡  Eq(Nat, N(σ), σ)
```

`Id` is a conjunction of equations, one between the results and one for each place written; a reflexive equation is `⊤`, and `⊤` is a unit for `∧`. Here `⊤` and `∧` are not primitives: they are `True` and `And`, ordinary inductive declarations in `Prop`, and `refl` is `True`'s constructor. The statement "running `AddM(x, 0)` has no effect" has become the ordinary proposition that `N(σ)` equals `σ`.

2.4 Induction hypotheses at the call site

Now the proof:

```
AddMZero(x : &Nat) : Id(Unit, AddM(x, 0), ()) by x :=
  match *x { Z => refl, S p => AddMZero(&p) }
```

The match on `*x` splits on `σ`. In the `Z` branch the goal is `Eq(Nat, N(Z), Z)`; `N(Z)` runs to `Z`, the equation is reflexive and so `⊤`, and `refl` proves it. In the `S` branch the goal is `Eq(Nat, N(S(σ')), S(σ'))`, which normalises to `Eq(Nat, S(N(σ')), S(σ'))`, and then, since `Eq` takes apart two successors, to `Eq(Nat, N(σ'), σ')`.

The recursive call is checked by the same rule that produced the goal: the callee's statement is evaluated at the call site, with its parameter bound to the argument. At the call site the environment is

```
{ c ↦ loan₀ | x ↦ borrow₀ (S(loan₁)) }   and the argument is   borrow₁ σ'
```

because matching `S(p)` through the borrow and taking `&p` leaves the successor in place, around a loan for the predecessor. The owner of the argument's loan is found by following it outwards: `loan₁` sits inside `x`'s borrow, whose loan sits in `c`. So the recursive call's statement observes `c`, and running `AddM` on the argument fills `loan₁` with `N(σ')`, after which `c` holds `S(N(σ'))`:

```
⟦AddM(x, 0)⟧, x := borrow₁ σ'  =  ((), S(N(σ')))
⟦()⟧,          x := borrow₁ σ'  =  ((), S(σ'))
AddMZero(&p) : Eq(Nat, S(N(σ')), S(σ'))  ≡  Eq(Nat, N(σ'), σ')
```

This is the goal: evaluated at the call site, the callee's statement about the place it borrowed is about the caller's number, successor included, which is the frame argument that a proof about a translated program makes by hand. The recursive call is a proof, and proofs are erased at runtime, so the checker does not run it: its borrow argument is returned unchanged.

The same theorem about the pure wrapper, `AddZero(x : Nat) : Id(Nat, Add(x, 0), x)`, has the same goal, since `Add(x, 0)` writes nothing outside itself. It is proved by lending the owned `x` to the in-place lemma: `AddZero(x) := AddMZero(&x)` type-checks, because both statements normalise to the same proposition. Neither proof mentions a model of `AddM`: the statement is about the program itself.

2.5 Returning a borrow

A more idiomatic in-place addition first finds the final node and then writes through it:

```
TailM(x : &Nat) : &Nat by x := match *x { Z => x, S p => TailM(&p) }
AddM'(x : &Nat, y : Nat) : Unit := let t = TailM(x); *t := y
```

`TailM` returns a borrow into its argument. When it is stuck on `σ`, the place it borrowed cannot be given its final content yet, because that depends on what the caller will later write through the returned borrow. Closing off such a call returns a borrow of a fresh `k` and fills the argument's loan with a sealed program that has a *hole* `loan_k` for the returned borrow's final content:

```
{ c ↦ loan₀ } ⊢ TailM(borrow₀ σ)
   →  { c ↦ ⌈let c = σ; let r = TailM(&c); *r := loan_k; c⌉ }
      ⊢ borrow_k ⌈let c = σ; let r = TailM(&c); *r⌉
```

In `AddM'`, the caller writes `y` through `t` and drops it; ending `borrow_k` substitutes the final content `y` for the hole. What remains in `c` is `⌈let c = σ; let r = TailM(&c); *r := y; c⌉`: the effect of `AddM'`, as a program. The

hole plays the role of Aeneas's region abstraction for this one call: it records only the *final* value written through the returned borrow.

The equivalence of the two additions is proved by the same bare recursion:

```
AddMEq(x : &Nat, y : Nat) : Id(Unit, AddM(x, y), AddM'(x, y)) by x :=
  match *x { Z => refl, S p => AddMEq(&p, y) }
```

In the successor branch, re-running `AddM'`'s sealed program on `S(σ')` unfolds `TailM` once, closes off its inner call with a fresh hole, and the pending write `*r := y` fills that hole; the recursive call's statement, evaluated at the call site, performs the same steps through the caller's borrow. The theorem for an owned number follows by lending it, as for `AddZero`.

2.6 Proofs about the current state

Dependent types are not only for stating theorems after the fact. A function can demand, as an argument, a proof about the *current* contents of a borrow. Here is in-place subtraction, which is total only when the subtrahend is no larger than the number:

```
Le(a : Nat, b : Nat) : Prop by a :=
  match a { Z => True, S a' => match b { Z => False, S b' => Le(a', b') } }

SubM(x : &Nat, y : Nat, h : Le(y, *x)) : Unit by y :=
  match y { Z => (), S q => match *x { Z => match h {}, S p => *x := p; SubM(x, q, h) } }
```

The type of `h` mentions `*x`, the content of the borrow at the moment of the call. Where `*x` is `Z` but `y` is not, `h` has type `Le(S(q), Z)`, which computes to `False`: the case cannot arise, and the match with no arms, `match h {}`, says so. Inside `SubM`, after `*x := p` has overwritten that content, the recursive call needs a proof of `Le(q, *x)` about the *new* content; the old hypothesis `h`, of type `Le(S(q), S(p))`, provides it, because that type was formed when `h` was bound and normalises to `Le(q, p)`.

Now a caller that first adds and then subtracts:

```
LeAdd(n : Nat, m : Nat) : Le(n, Add(n, m)) by n := match n { Z => refl, S n' => LeAdd(n',
m) }

AddSub(x : &Nat, y : Nat) : Unit :=
  let old = clone(*x); AddM(&*x, clone(y)); SubM(x, clone(old), LeAdd(old, y))
```

At the call to `SubM`, `*x` has just been mutated in place by `AddM`, so on symbolic input it holds the sealed program `N(σ, y) = ⌈let c = σ; AddM(&c, y); c⌉`, and `SubM` demands a proof of `Le(σ, N(σ, y))`. `clone` copies `*x`, since reading a number through a borrow would move it out. A proof uses a variable as code does, so `old` and `y`, each needed again by the proof `LeAdd(old, y)`, are passed to the code as copies. The lemma `LeAdd` is about the *pure* `Add` of the snapshot `old`, and its type is `Le(σ, Add(σ, y))`. The two meet because `Add(σ, y)` normalises to the very same sealed program: the in-place computation and the pure one are the same program to the type checker. A proof about the pure function is accepted where a proof about the mutated state is required, with no bridging lemma. `LeAdd` is also the idiom for a postcondition: a property of what an in-place function leaves behind is stated by running it on a copy inside the statement, here through its wrapper `Add`, since a type formed before the body runs cannot see the body's writes (Section 6). Using the snapshot after further mutation, `…; AddM(&*x, clone(y)); *x := Z; SubM(x, clone(old), LeAdd(old, y))`, is rejected, since the requirement then mentions `Z`.

2.7 Trees

Nothing above is specific to numbers. With several constructors and several fields, a pattern variable names a field place, and the environment keeps every field that a recursive call does not touch. In-place insertion into a binary search tree recurses into the left or the right subtree according to a comparison, so which place is mutated

depends on a value. Its pure version is not written separately; like `Add`, it runs the in-place one on the tree it is given. Keys are `Word`s, a library type of numbers that code only compares, which is unary in the logic, a machine word in the cost model, and copied when read (Section 4):

```
InsertM(t : &Tree, k : Word) : Unit by t :=
  match *t { Leaf          => *t := Node(Leaf, k, Leaf),
             Node(l, v, r) => let b = Lt(k, v);
                              match b { true => InsertM(&l, k), false => InsertM(&r, k) } }
Insert(t : Tree, k : Word) : Tree := InsertM(&t, k); t
```

A property of the in-place code is then stated and proved directly: insertion adds one node.

```
SizeInsert(t : Tree, k : Word) : Id(Nat, S(Size(t)), Size(Insert(t, k))) by t :=
  match t { Leaf => refl,
            Node(l, v, r) => let b = Lt(k, v); match b {
              true  => rewrite SizeInsert(l, k) in refl,
              false => rewrite SizeInsert(r, k) in rewrite AddS(Size(l), Size(r)) in
refl } }
```

The comparison `Lt(σ_k, σ_v)` is stuck, so the split on `b` generalises the sealed program to a fresh abstract value, everywhere it occurs in that arm and wherever it is derived again there. In each arm, `Insert`'s sealed program runs one step and leaves the other subtree and the key in place. The proof is not bare recursion. Each arm rewrites the goal with the induction hypothesis: `rewrite h in t`, for `h : Eq(A, a, b)`, replaces every occurrence of `b` in the goal by `a` and checks `t` against the result. It is `J` with its motive read off the goal, a proof that never runs, so it adds nothing to the model. The `false` arm also needs an arithmetic lemma, `AddS : x + S(y) = S(x + y)`, proved in place by bare recursion and transferred to `Add` by lending, as `AddZero` was. This is the cost of a property that is not an equation between two recursions of the same shape.

2.8 When the two paths could disagree

A `match` on an abstract value that is not the whole body of a call is closed off like a call, as a *stuck block*: an anonymous function of the places its arms use, whose sealed programs run again when a later split decides the match. This gives the checker two ways to evaluate a statement: at the generic call, where the block closes off, and directly at each instance, where the match runs. Take

```
Clear(x : &Nat) : Id(Unit, (match *x { Z => (), S p => p := Z }), ())
```

At the generic call `*x` is `σ`, the match is stuck, and the block must decide how to take `*x`. Its arm writes `p`, not `*x`; if the block took `*x` by copy, the write would land on the copy, `c` would hold `σ` on both sides, and `refl` would prove the statement. At an instance where `*x` is `2` the match runs, `p := Z` sets the predecessor field of the number to `Z`, and the statement is `Eq(Nat, 1, 2)`, which is `False`. So a write through a pattern variable must count as a write to the scrutinee, and the block takes `*x` by borrow: the capture is decided from the syntax, with `p` resolved to the place `(*x).0`. Every such decision is read from syntax and declared types, never from a normal form (Section 6.3), and `Clear` is rejected.

3 The calculus

Ochr is a dependent type theory in the style of Lean's kernel [11], extended with places, mutable borrows and sequencing. This section gives its syntax and the runtime structures of its evaluator. Data and the logical connectives are introduced alike, by inductive declarations; the rules in the body are illustrated on natural numbers, and Section A gives the general definition.

$$
\begin{array}{rcll}
t, u, A, B & ::= & x \mid \mathsf{Prop} \mid \mathsf{Type}_i & \text{variables, universes} \\
& \mid & \Pi(x_1 : A_1 ... x_n : A_n).B \mid \mathbf{fix}\ f(x_1 : A_1 ... x_n : A_n) : B\ \ \mathbf{by}\ x_j := t \mid t(u_1, ..., u_n) & \text{functions, calls} \\
& \mid & \mathsf{D}(a_1, ..., a_m) \mid \mathsf{C}(t_1, ..., t_k) & \text{inductive types, constructors} \\
& \mid & \mathsf{Eq}\ A\ t\ u \mid \mathsf{J}(A, a, b, P, h, t) \mid \mathbf{rewrite}\ h\ \ \mathbf{in}\ t & \text{equality} \\
& \mid & \&A \mid p \mid \&p \mid p := t \mid \mathbf{let}\ x = t; u \mid \mathbf{let}\ x : A = t; u \mid t; u & \text{places and borrows} \\
& \mid & \mathbf{match}\ p\ \{\mathsf{C}_1(\overline{y}_1) \Rightarrow t_1, ..., \mathsf{C}_n(\overline{y}_n) \Rightarrow t_n\} \quad (n \geq 0) \mid \mathbf{split}\ f\ \ \mathbf{in}\ t & \text{case analysis} \\
& \mid & \mathsf{Id}\ A\ t\ u & \text{computation equality} \\
p & ::= & x \mid *p \mid p.g & \text{places } (g \text{ a field}) \\
d & ::= & \mathbf{inductive}\ \mathsf{D}\ (x_1 : A_1 ... x_m : A_m) : s := \mathsf{C}_1\big(\overline{g_1 : B_1}\big) \mid ... \mid \mathsf{C}_n\big(\overline{g_n : B_n}\big) & \text{declarations}
\end{array}
$$

$$
\begin{array}{ll}
\mathbf{inductive}\ \mathsf{Nat} : \mathsf{Type}_0 := \mathsf{Z} \mid \mathsf{S}(\mathrm{pred} : \mathsf{Nat}) & \mathbf{inductive}\ \mathsf{Unit} : \mathsf{Type}_0 := \mathsf{Tt} \\
\mathbf{inductive}\ \mathsf{False} : \mathsf{Prop} & \mathbf{inductive}\ \mathsf{True} : \mathsf{Prop} := \mathsf{I}
\end{array}
$$

$$
\mathbf{inductive}\ \mathsf{Pair}\ (A : \mathsf{Type}_0)(B : \mathsf{Type}_0) : \mathsf{Type}_0 := \mathsf{Mk}(\mathrm{fst} : A, \mathrm{snd} : B)
$$

$$
\mathbf{inductive}\ \mathsf{And}\ (P : \mathsf{Prop})(Q : \mathsf{Prop}) : \mathsf{Prop} := \mathsf{Intro}(l : P, r : Q)
$$

Figure 3: Syntax of Ochr, and the library declarations. A declaration has sort $s \in \{\mathsf{Type}_0, \mathsf{Prop}\}$ and $n \geq 0$ constructors. (), $A \times B$, (a, b), $\top$, **refl**, $P \wedge Q$ and $\langle h, k \rangle$ are notation for `Tt`, `Pair`(A, B), `Mk`(a, b), `True`, `I`, `And`(P, Q) and `Intro`(h, k), and $p.0$, $p.1$ name a place's first and second fields. A **fix** without **by** is an ordinary λ.

3.1 Syntax

Figure 3 gives the syntax. Types and terms share one grammar, as in any pure type system. Functions are n-ary and calls are saturated: a partial application would be a closure capturing its arguments, and a closure capturing a borrow is outside the core (Section 4.3). A recursive function names the parameter it recurses on.

Inductive definitions. Every type of data and every logical connective is an inductive declaration: a name, parameters, a sort (`Type₀` for data, `Prop` for propositions) and any number of constructors with named fields. Natural numbers, the unit type and pairs are declarations (the checker builds `Nat` and `Unit` in, with the same behaviour), and so are `False`, the proposition with no constructors, `True`, with one constructor and no fields, and `And(P, Q)`, with one constructor whose two fields are proofs of `P` and of `Q`. `⊤`, `P ∧ Q`, `⟨h, k⟩` and `refl` are notation for them. Their eliminations are matches: ex falso is the match with no arms, `match h {}`, and a proof of `P ∧ Q` is taken apart by `match h { Intro(l, r) => … }`. Constructors also take their declaration's parameters as leading arguments, `Intro(P, Q; h, k)`, which we omit when they can be inferred. Fields are first-order: they may mention the type being declared, other declared types and the parameters, and may be equations between first-order data, but not `Π` or `&`, which is strict positivity in its simplest form.

`Eq` is the one primitive proposition. It is not an inductive family, for two reasons: it computes by the structure of the values it compares, as in observational type theory (Section 5), and the core has no indices. A family indexed by values is *Forded* instead: the index is a parameter, and each constructor stores an equation fixing it, as in `Vec(A, n) := VNil(h : Eq(Nat, n, Z)) | VCons(m : Nat, x : A, xs : Vec(A, m), h : Eq(Nat, n, S(m)))`, so a case that cannot arise receives an equation that computes to `False`.

The imperative fragment is small. A *place* `p` is a variable, a dereference `*p`, or a field `p.g` of a constructor value, such as the predecessor field `p.0` of a number. A place used as a term reads it; `&p` borrows it; `p := t` assigns it; `let x = t; u` introduces a new place `x`. The pattern variables of a `match` are *sub-places*: in `match p { S y => u }`, `y` stands for `p.0`, and nothing is copied, so a read of `y` after `p` is reassigned sees the new content, where a Rust binding would have taken the old value or rejected the assignment.

`&A` is the type of a mutable borrow of an `A`, and `A` must be in `Type₀`: data, a function type into data or functions, or a type variable declared in `Type₀`. Propositions and universes are not, since `Prop : Type₁` and `Type₀ : Type₁` (universes are not cumulative). `&` occurs only at the top of a type as written, as the type of a variable, parameter or result; never inside another type, and never as the result of computing one. So there are no borrows stored in data structures and no borrows of borrows. Shared borrows, loops and `'static` borrows are absent (Figure 1), and recursion is structural.

The propositional fragment is Lean's: `Prop` is an impredicative universe with definitional proof irrelevance, erased at runtime, and `Eq` is its equality. `Id(A, t, u)` compares two *computations* `t` and `u` of type `A`; it is a derived proposition whose computation rule is observation (Section 5).

3.2 Values and environments

v, w	$::=$	$\mathsf{C}(v_1, ..., v_k) \mid \star \mid f \mid \langle \overline{\kappa} \vdash \mathbf{fix}\ f\big(\overline{x} : \overline{A}\big) : B... := t\rangle \mid \text{types}$	data, proofs, functions, closures, types
	$\mid$	$\text{borrow}_\ell v \mid \text{loan}_\ell \mid \bot$	borrows, loans, moved-out
	$\mid$	n	neutrals
n	$::=$	$\sigma \mid \lceil t \rceil$	abstract values, sealed programs
Ω	$::=$	$\cdot \mid \Omega, x : A \mapsto v \mid \Omega \mid \Omega'$	bindings, grouped in frames

Figure 4: Runtime structures. A closure is code with its captured values $\overline{\kappa}$, and a Π-type is one too, $\langle \overline{\kappa} \vdash \Pi\big(\overline{x} : \overline{A}\big).B\rangle$ (Section 4.3).

The evaluator manipulates the values of Figure 4. Following the low-level borrow calculus of Aeneas [23], a borrow carries the borrowed content with it: `borrow_ℓ v` is a borrow, identified by `ℓ`, whose current content is `v`, and `loan_ℓ` is the placeholder left at the place it was taken from. When the borrow ends, its content is put back where the loan is. We depart from Aeneas in one respect: **a loan is a variable bound by its borrow**. It may occur anywhere inside a value, several times inside sealed programs, and ending the borrow substitutes its content for every occurrence. `⊥` marks a place whose content has been moved out, and `⋆` is the value of a proof.

Neutral values are stuck computations. An *abstract value* `σ` is an unknown value of a known type, playing the role of a free variable in Lean's kernel: the checker introduces one for each parameter of a definition it checks, and for each field that a case split or a generalisation exposes; concrete evaluation never introduces one. A *sealed program* `⌈t⌉` is a closed program whose run is stuck on an abstract value. A value is *borrow-free* if it contains no borrow and no live loan.

An environment `Ω` is a stack of frames, one per active call, each binding variables to values. `content(Ω, p)` follows a place through variables, borrows (`*p` looks inside `borrow_ℓ v` at `v`) and fields (`p.0` looks inside `S(v)` at `v`). We write `Ω[p ↦ v]` for the environment with the content of `p` replaced.

4 Evaluation and closing off

Definitional equality in Ochr is normalisation by one deterministic machine. With effects, rewriting-anywhere is not confluent: evaluating `(λy. (x := 2; y))(x := 1)` argument-first and substituting-first leave `x` holding different numbers. A fixed call-by-value order restores uniqueness, and the machine then plays the role that normalisation by evaluation plays for pure type theories [2]: two terms are definitionally equal when the machine sends them to the same normal form. On concrete inputs it is an ordinary interpreter. On abstract inputs it runs until it has to inspect an abstract value, and the question the rest of this section answers is what it returns then.

The judgement is big-step: `⟨Ω, t⟩ ⇓ ⟨Ω', v⟩` runs `t` from environment `Ω` to value `v` and environment `Ω'`, and a run may instead be *stuck* (it needs to inspect a neutral) or fail with a borrow error.

4.1 Borrows

$$\frac{\text{borrow}_\ell v \text{ in } \Omega}{\Omega \rightsquigarrow \Omega[\text{borrow}_\ell v := \bot][v/\text{loan}_\ell]}\ \textsc{End} \qquad \frac{v = \text{content}(\Omega, p)}{\langle \Omega, \&p\rangle \Downarrow \langle \Omega[p \mapsto \text{loan}_\ell], \text{borrow}_\ell v\rangle}\ \textsc{Borrow}$$

$$\frac{v = \text{content}(\Omega, p)\ \text{borrow-free}, p \text{ of a copy type}}{\langle \Omega, p\rangle \Downarrow \langle \Omega, v\rangle}\ \textsc{Copy} \qquad \frac{v = \text{content}(\Omega, p)\ \text{borrow-free, not a copy}}{\langle \Omega, p\rangle \Downarrow \langle \Omega[p \mapsto \bot], v\rangle}\ \textsc{Read}$$

$$\frac{\text{content}(\Omega, p) = \text{borrow}_\ell v}{\langle \Omega, p\rangle \Downarrow \langle \Omega[p \mapsto \bot], \text{borrow}_\ell v\rangle}\ \textsc{Move} \qquad \frac{\langle \Omega, t\rangle \Downarrow \langle \Omega_1, v\rangle \qquad \Omega_2 = \text{drop}(\Omega_1, \text{content}(\Omega_1, p))}{\langle \Omega, p := t\rangle \Downarrow \langle \Omega_2[p \mapsto v], ()\rangle}\ \textsc{Assign}$$

Figure 5: Borrows. Every rule that accesses a place `p` is preceded by [Access]: end every borrow whose loan lies on the path to `p` or inside its content (for a match, which inspects only the head: on the path, at the head, or anywhere inside a neutral head). `drop` ends the borrows inside a dropped value and fails on a live loan in a dropped owned value. ⊥ may not be read, borrowed, or left inside a borrow's content when the borrow ends.

Figure 5 gives the rules for borrows. They are Aeneas's, simplified by treating loans as variables.

Ending a borrow [End] replaces it by ⊥ and substitutes its content for its loan. There is no side condition: if the content itself contains loans (because something has reborrowed part of it), they travel with it, and the reborrows remain valid. Ending is allowed at any time; the machine does it lazily, when a place is accessed. [Access] makes every access exclusive: before a place is read, borrowed, assigned or matched on, every borrow whose loan lies on the path to it or inside its content is ended. A borrower that has been ended holds ⊥, and any later use of it is an error. This is the whole of the borrow checker: exclusivity of mutable borrows is not a separate analysis but a consequence of running the program.

Reading a place moves its content out, as in Rust, and leaves ⊥ behind: a borrow [Move], or data [Read]. Types and proofs read as code does. Data of a *copy type* is copied instead [Copy]: `Unit`, sorts, propositions, inductive types declared `copy`, and non-recursive inductive types whose fields are copy types. The library declares `Word`, a copy type of numbers that code only computes with, as Rust's `usize` is `Copy`; `Nat` is not one, since in-place code walks and moves it. `clone(p)` copies explicitly. A borrow must be whole again, with no ⊥ in its content, when it ends, is returned, or is passed to a call that closes off. Reborrowing is explicit (`&*x`). A statement, and each argument of a statement, runs on its own copy of the state, so `Id(Nat, Add(x, Z), x)` is well formed, each side using `x` once, while `Add(x, x)` must be written `Add(clone(x), x)`, in a type as in code.

4.2 Calls and matches

A call evaluates its arguments left to right, each into a temporary of the current frame, so that [Access] can see and end them: `f(&x, &x)` ends the first borrow when the second is taken, and the call then fails reading ⊥. It then pushes a frame binding the parameters, runs the body, and pops the frame, dropping its bindings. Popping a frame that still lends out one of its owned places is an error: something would outlive the place it borrows.

Types and proofs are erased at runtime, and the machine treats them accordingly: an erased term is evaluated on a *private copy* of the environment, argument evaluation included, and the copy is then discarded, so the machine may skip a proof altogether, returning ⋆. Erasure is decided by syntax and declared types, never by a normal form (Section 6.3). A call whose head is a neutral (an abstract function or a sealed program) is stuck at once.

`match p { Z => t, S y => u }` inspects the head of `p`'s content. `Z` selects the first arm, and `S(v)` selects the second with `y` standing for the sub-place `p.0`. Through a borrow, `match *x` leaves the successor in the borrowed content, so `&y` then reborrows the predecessor field in place: $\texttt{x} \mapsto \texttt{borrow}_0\ \texttt{(S(loan}_1\texttt{))}$ and the new borrow is $\texttt{borrow}_1\ \texttt{v}$. If the head is a neutral, the run is stuck. A match on a proof never inspects it, since every proof is ⋆: whether a match is on a proof is read from the declarations of the constructors its arms name, and such a match is decided by the proof's type (Section 6.6).

4.3 Functions and closures

A `fix` evaluates to a *closure* $\langle \overline{\kappa} \vdash \textbf{fix}\ f\left(\overline{x} : \overline{A}\right) : B... := t\rangle$: its code with the current values $\overline{\kappa}$ of its free variables, in binding order, each with its binding's declared type and proof flag. A Π-type is formed the same way, and a top-level function is just its name. Capturing is reading: a borrow that lends a captured variable out is ended

first, and at runtime the closure takes the variable's value, moving it unless its type is a copy type. In a type it is copied. Anything but a borrow or a moved-out place can be captured: data, proofs, types, closures, abstract values and sealed programs. A closure may run more than once, so, as with Rust's `Fn`, its body may not move what it captured, and a call does not consume the closure it calls. A `λ` inside a function that takes `x : &Nat` therefore clones the number first, and clones it again where the body uses it, as in `let n = clone(*x); let f = (λ(y : Nat) : Nat => clone(n))`, and may still take borrows as parameters, as `λ(z : &Nat) : Unit => AddM(z, 0)` does.

Captures are snapshots, by the mechanism that forms a type once (Section 6): there is no capture by reference, and a closure that assigns a captured variable assigns its own copy, afresh at each call, so calling twice a closure that increments its capture gives the same result twice, where Rust's `FnMut` would accumulate and `Fn` would reject the assignment. A closure holds no borrow; it is a copy when everything it captured is, and otherwise reading it moves it. It can be stored in data only through a type parameter, as in `Box(Π(n : Nat). Nat)` (`Box(Prop)` is a universe error, since `Prop : Type₁`); a borrow inside a closure would be a borrow stored inside a value (Section 10). Calling a closure pushes one frame holding its captured values, itself if it is recursive, and the parameters. Two function values are convertible when their Π-types are, their captured values are pairwise, and their generic calls have the same observation (Section A.3). The only capture of places, as in Rust's closures, is by a closed-off stuck match (Section 4.5), and it makes them the arguments of a call.

4.4 Closing off

When the body of a call is stuck, the call itself is stuck. A pure type theory would return it as a neutral term, but the caller also needs the final contents of the places the call borrowed, so we *close off* the call instead.

$$\text{the body of } f(w_1, ..., w_n) \text{ is stuck}, \quad w_i = \text{borrow}_{\ell_i} u_i \ \text{ for } \ i \in I, \quad u_i \text{ loan-free}$$

$$L \coloneqq \text{let } c_i = u_i \quad (i \in I) \qquad C \coloneqq f(a_1, ..., a_n), \quad a_i = \begin{cases} \&c_i \text{ if } \ i \in I \\ w_i \quad \text{otherwise} \end{cases}$$

result type	result	each loan_{ℓ_i} becomes
borrow-free D	$\lceil$L; C$\rceil$	$\lceil$L; C; $\ c_i\rceil$
$\&T$	$\text{borrow}_k\lceil$L; let r = C; *r$\rceil$	$\lceil$L; let r = C; *r := $\ \text{loan}_k$; $\ c_i\rceil$

Figure 6: [Close]. The partial run of the body is discarded; the borrows `borrow_ℓᵢ` are consumed; `k` is fresh.

Figure 6 gives the rule. Take the call at the point where its arguments have been evaluated. For each borrow argument `borrow_ℓᵢ uᵢ`, the canonical program `L; C` recreates the situation the call was in, but with the borrowed contents owned by fresh local places `cᵢ`: it moves each `uᵢ` into `cᵢ` and calls `f` on `&cᵢ`. This program is closed. Running it and reading `cᵢ` afterwards computes exactly what the call would have left behind the borrow, so each loan is filled with the sealed program ⌈`L; C; cᵢ`⌉. The call returns the sealed program ⌈`L; C`⌉, which at result type `Unit` is η-normal, `()` (Section 5).

Sealed programs compute what Aeneas's backward functions compute, in the source language. For `AddM(borrow₀ σ, y)`, the loan receives ⌈`let c = σ; AddM(&c, y); c`⌉, which is the body of `Add(σ, y)`, where Aeneas's translation would emit `add_m_back σ y`. This correspondence is a conjecture (property 3 of Figure 9), and it is narrower than Aeneas. A sealed program belongs to one call, not to a region of the callee's signature, so a borrow-returning call's hole goes into every borrowed argument (Section 1.5). It is computed by unfolding the callee, where Aeneas reads the signature. And it has no counterpart for loops or for borrows stored in data. Nothing about the call is lost: when `σ` is later refined, the sealed program runs again.

Returning a borrow. If `f` returns a borrow, the contents of the places it borrowed depend on what the caller will write through the returned borrow before it ends. The rule returns a borrow of a fresh `k`, and fills each `loan_ℓᵢ` with a sealed program that has a *hole*: it writes `loan_k` through the returned borrow before reading `cᵢ`. Because loans are variables, the ordinary [End] rule fills the hole with the returned borrow's final content, which is all that Aeneas's backward function takes as input. If `f` has several borrow arguments, the hole occurs in each of their sealed programs, and one [End] fills them all.

The precondition that each `uᵢ` is loan-free is guaranteed by [Access], which ended those loans when the argument was read or borrowed. Without it, a live loan copied into a sealed program would survive its borrow.

4.5 Sealed programs and refinement

Values are kept in normal form. The normal form of a sealed program `⌈L; C; K⌉` is obtained by running it from the empty environment, with one restriction: its *head call* `C` may not itself be closed off (calls made inside `C`'s body may). This is the analogue of the guard on fixpoint unfolding in the calculus of inductive constructions. Without it, a sealed program would re-close into itself forever; with it, a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run (a hole whose returned borrow is still live) is *inert*: the run treats it like an abstract value. A borrow error during normalisation is a type error at the point that triggered it. When a case split has generalised a sealed program to an abstract value (Section 6), every later derivation of the same closed program in that arm normalises to that value.

A *refinement* substitutes a constructor pattern for an abstract value, `σ := Z` or `σ := S(σ')`, when a case split learns it (Section 6). Substitution into a sealed program is followed by normalisation, which runs the sealed program again. For `N(σ) = ⌈let c = σ; AddM(&c, 0); c⌉` and `σ := S(σ')`, the head call `AddM` now unfolds, matches the successor, reborrows the predecessor, and makes the inner recursive call, which is stuck on `σ'` and is closed off; so `N(S(σ')) = S(N(σ'))`. Normal forms of this shape are what makes the induction of Section 2 go through. The key property, a conjecture of Section 7, is that refinement commutes with closing off: re-normalising a closed-off call after refining its arguments gives the same normal form as closing off the refined call.

Stuck blocks. A stuck `match` that is not the body of a call is closed off in the same way, as the body of a call to an anonymous function of its free places. Pattern variables are first resolved to the sub-places they denote, and the function takes the resulting maximal places the way Rust infers closure captures: a place the match moves out of, in any arm, is moved in; a place it writes or borrows is passed by borrow; a place it only reads is copied. The function's result type is the match's type, which must agree across the arms or be annotated (`let x : A = match …`). Only a match whose arms have been type-checked is closed off, including a match inside a type, and a stuck block is erased exactly when each of its arms is a declared proof. This covers stuck matches inside types and, during checking, a stuck match that is followed by more code (Section 6).

5 Observational equality

`Id(A, t, u)` states that the computations `t` and `u`, both of result type `A`, are indistinguishable from where the statement is made: run from the current environment, they return equal values and leave equal contents in every place either of them may write. This section defines that notion precisely and shows that it reduces to ordinary propositional equality.

5.1 Footprints and observations

A computation affects the world only through the places it names. The places a term may **write** are read off its syntax: they are the free places that appear under `&_` or on the left of `:=`, and the borrow variables it mentions, since a borrow grants write access to someone else's place. Each of these is traced back to the owned place that will receive the write. For a place rooted at an owned variable `x`, that is `x`. For a borrow variable holding `borrow_ℓ v`, it is the set of *owners* of `ℓ`:

$$\mathrm{owners}(\ell) = \bigcup_{\text{occurrences of loan}_\ell} \begin{cases} \mathrm{owners}(m) & \text{if the occurrence lies inside the content of borrow}_m \\ \{x\} & \text{if it lies inside the owned binding } x \end{cases}$$

Following loans outward through enclosing borrows is what connects a reborrow with the place it ultimately writes. In the successor case of `AddMZero`, the argument `borrow₁ σ'` has its loan inside the content of `x`'s `borrow₀`, whose loan is in the generic caller's `c`; so the owner of the recursive call's argument is `c`, the same place the goal observes. The owners form a set because a hole left by a returned borrow can occur in several sealed programs at once, and all of them must be observed (Figure 15).

The *footprint* `W(t, u)` is the union of the owners of the places `t` or `u` may write. The places are read off the syntax, so the footprint does not depend on unrelated places in the environment; their owners are found by following loans, so a later refinement can shrink an owner set, removing a component both sides agree on.

The *observation* of `t` at `Ω` on footprint `W` runs `t` on a private copy of `Ω`, ends every borrow that remains, and reads off the result and the contents of the footprint:

$$[\![t]\!]^W_\Omega = (v, \Omega'(k)_{k\in W}) \quad \text{where } \langle \Omega, t\rangle \Downarrow \langle \Omega'', v\rangle \text{ and } \Omega'' \overset{*}{\rightsquigarrow} \Omega' \text{ with no borrows left}$$

If `t` is itself stuck outside any call, it is first closed off as a stuck block, so an observation always exists. Ending every remaining borrow makes the observation independent of which borrows happened to be live when `t` finished; that the order in which they are ended does not matter is property 4 of Figure 9, assumed rather than proved.

5.2 `Id` is equality of observations

$$\mathrm{Id}\ A\ t\ u \quad\equiv\quad \mathrm{Eq}\ A\ r\ r' \wedge \mathrm{Eq}\ T_1\ w_1\ w_1' \wedge \ldots \wedge \mathrm{Eq}\ T_k\ w_k\ w_k' \qquad \text{where } [\![t]\!]^W_\Omega = (r, \overline{w}),\ [\![u]\!]^W_\Omega = \left(r', \overline{w'}\right),\ W = W(t,u)$$

$$\mathrm{Eq}\ \mathsf{D}(\overline{a})\ \ \mathsf{C}(\overline{v})\ \ \mathsf{C}(\overline{w}) \equiv \mathrm{Eq}\ B_1\ v_1\ w_1 \wedge \ldots \wedge \mathrm{Eq}\ B_k\ v_k\ w_k \qquad \mathrm{Eq}\ \mathsf{D}(\overline{a})\ \ \mathsf{C}(\overline{v})\ \ \mathsf{C}'(\overline{w}) \equiv \mathsf{False} \quad \text{if } \ \mathsf{C} \neq \mathsf{C}'$$

$$\mathrm{Eq}\ A\ a\ b \equiv \top \quad \text{if } \ a \equiv b \qquad \top \wedge P \equiv P \equiv P \wedge \top$$

Figure 7: `Id` computes to a conjunction of equations between the two observations, both run from Ω on independent copies (T_i is the type of the i-th place of W). `Eq` computes on constructor values by structure (B_j are the fields' types; no fields gives `⊤`) and on reflexive instances. `⊤`, `∧` and `False` are the library inductives of Figure 3, and `refl : ⊤`.

Figure 7 is the whole of `Id`. Both sides run from the same environment, each on its own copy; sharing one copy and running the sides in sequence would let the first side's effects change what the second observes. The result type `A` must be borrow-free: a borrow has no meaning once the computation that produced it has finished. An observation is a tuple of the machine, not a pair value, so `A` may be any borrow-free type, a proposition or a universe included.

The equality `Eq` is Lean's, living in the proof-irrelevant universe `Prop`, and it computes on constructor values by structure, in the style of observational type theory [5, 47]. Two values built by the same constructor are equal when their fields are (injectivity), two built by different constructors are not (disjointness), a reflexive equation is `⊤`, and `⊤` is a unit for `∧`. Values at `Unit` are η-normal: a value whose type is known to be `Unit` is `()`, so `Eq` decides on values alone. The rules produce the library inductives `And`, `True` and `False`, so these three are known to the conversion checker, the one point where the library declarations are special. The unit laws are conversion rules, not a rewriting of stored types: a hypothesis of type `⊤ ∧ P` keeps its `And` and can still be taken apart by a match. Together the rules strip away what no side changed, as in the `AddM` example of Section 2. Disjointness lets an observation that differs in a constructor reach `False` directly: at a definition taking `x : &Nat`, the statement `Id(Unit, *x := 0, *x := 1)` computes to `False`, so a hypothesis of that type is eliminated by `match e {}`. An `Id` over several places computes to a conjunction, which `match h { Intro(l, r) => … }` takes apart. `refl` proves `⊤`, and so, by conversion, every reflexive equation. Transport `J(A, a, b, P, h, t)` takes its endpoints explicitly, since a reflexive `Eq(A, a, a)` computes to `⊤` and no longer records them, and it computes to `t` only when `a ≡ b`, as Lean's `Eq.rec` does. Under a hypothesis that equates unequal values a cast stays stuck, so there is no equality reflection and no value of the wrong type.

Two consequences are worth drawing out.

Owned locals are observed. The footprint includes places the current function owns, not only the places behind its borrow parameters. This is forced. A context such as `(□; x)`, which reads `x` afterwards, distinguishes `x := 6` from `()`; if `Id` ignored owned locals it would equate them, and transport along that equation would prove `Eq(Nat, 6, 5)`.

Unobserved moves are harmless. The two sides of an `Id` may consume different borrow variables, but borrow variables are never in the footprint, their owners are, and ending the remaining borrows puts every borrowed content back in its owner. They may also consume different owned data, `Id(Nat, let z = a; Z, Z)` holds: a

place only read is not observed, and code that reads it after the side that moved it is rejected, so no accepted context tells the sides apart by it.

6 Typing

Typing is the machine run on abstract inputs, with case splitting where the checked program itself inspects an abstract value. The judgement `Ω ⊢ t ⇓ v : A ⊣ Ω'` says that from `Ω` the term `t` runs to the value `v` of type `A`, leaving `Ω'`. It returns the value, not just the type, because later types may depend on it: after `let z = (let y = &x; *y := 2; x)`, the type `Id(Nat, z, 2)` holds by `refl` only because the checker knows that `z` is `2`. The rules for the basic forms are the evident ones, each the corresponding machine rule annotated with types; Figure 8 gives the rules particular to Ochr. The side conditions follow from one principle (Section 6.3).

Ochr is a type theory in a narrower sense than Lean or Rocq. There is no separate declarative system of which the checker is an algorithm: this judgement, the machine run with types, is the definition, and a type is a value formed at a state. The properties a kernel usually rests on are therefore about the machine: in place of a substitution lemma, the stability of the checker's decisions under refinement, and in place of subject reduction, naturality, a symbolic run's agreement with every concrete one (Section 7.2). Both, like consistency, are conjectures.

$$\frac{\Omega;\,(x_1 : T_1 \mapsto w_1, \ldots, x_{i-1} : T_{i-1} \mapsto w_{i-1}) \vdash A_i \Downarrow T_i \text{ type}, \quad A_i' \equiv T_i \quad (1 \le i \le n) \qquad \Omega;\,\big(\overline{x} : \overline{T} \mapsto \overline{w}\big) \vdash B \Downarrow T_B \text{ type}}{\text{callty}_\Omega\big(\Pi\big(\overline{x} : \overline{A}\big).B,\ \overline{w} : \overline{A'}\big) = T_B}\ \textsc{Call-type}$$

$$\frac{\Omega \vdash \overline{u} \Downarrow \overline{w} : \overline{A'} \dashv \Omega_1 \qquad \text{callty}_{\Omega_1}\big(\Pi, \overline{w} : \overline{A'}\big) = B' \qquad \text{rec}_{\Omega_1}(F, \overline{w}) \qquad \langle \Omega_1, F(\overline{w})\rangle \Downarrow \langle \Omega', v\rangle}{\Omega \vdash F(\overline{u}) \Downarrow v : B' \dashv \Omega'}\ \textsc{T-Call}$$

$$\frac{\text{content}_\Omega(p) = \sigma \quad \Omega[\sigma := \mathsf{C}_i(\overline{\sigma}_i)] \vdash t_i \Downarrow v_i : A_i \dashv \Omega_i, \quad A_i \equiv B[\mathsf{C}_i(\overline{\sigma}_i)/\sigma] \quad (1 \le i \le m) \qquad \langle \Omega, \text{block}(\textbf{match}\ p\{\ldots\})\rangle \Downarrow \langle \Omega', v\rangle}{\Omega \vdash \textbf{match}\ p\ \{\mathsf{C}_1(\overline{y}_1) \Rightarrow t_1, \ldots, \mathsf{C}_m(\overline{y}_m) \Rightarrow t_m\} \Downarrow v : B \dashv \Omega'}\ \textsc{Split}$$

$$\frac{\Gamma;\,\varphi \vdash B \Downarrow G \text{ type} \quad \Gamma;\,\varphi \vdash b \Downarrow v : A \dashv \Omega' \text{ on each path of its case splits} \qquad \text{pop}(\Omega', v) \text{ succeeds} \quad A \equiv G \text{ as refined on that path}}{\vdash \textbf{fix}\ f\big(\overline{x} : \overline{A}\big) : B \ \ \textbf{by}\ x_j := b \ \text{ ok}}\ \textsc{Def}$$

Figure 8: The typing rules particular to Ochr. [Call-type] evaluates the codomain with the parameters bound to the arguments' values, on a private copy: the type of every call, recursive calls included. [T-Call] then runs the callee in the machine, where a stuck body closes off; `rec` is [Rec]: in the body of `fix f … by xⱼ`, `f` occurs only as the head of a call, and each recursive call passes in position `j` a strict subterm of `xⱼ`'s value on entry. [Split] checks each arm under a refinement of the abstract scrutinee `σ` (with fresh $\overline{\sigma}_i$; a sealed scrutinee is first generalised to a fresh `σ`), and runs what follows from the state in which the match is closed off as a stuck block. [Def] checks a definition at its generic call: Γ binds a fresh owned place `cᵢ ↦ σᵢ` for each borrow parameter, and φ binds each parameter to `&cᵢ`, to `⋆` for a proof, or to a fresh `σᵢ`. The full rules are in Section A.4.

6.1 One rule for goals and induction hypotheses

A definition is checked by running it at its most general call [Def]. For `AddMZero(x : &Nat)` that is `AddMZero(&c)` from `{ c ↦ σ }`: the parameter becomes a borrow of a fresh owned place, which stands for whatever the real caller lends. The goal is computed by [Call-type], the rule that types every call. The same rule types the recursive call in the successor case, at a different environment, where `&p` borrows the predecessor field of `c`'s content; the callee's statement, evaluated there, observes `c` through the successor around the field, so the induction hypothesis arrives already wrapped in `S`. That a statement proved at the generic call remains true at every call site rests on the frame property (Section 7): a call affects only what it is passed, so the caller's observation is the callee's, placed in a context.

6.2 Types are formed once, and matches split

A type is evaluated once, when it is formed, on a private copy of the environment, and becomes a closed statement about values that later mutation cannot change; a Π-type is a closure over values, whose parameters alone are bound later. So a body that mutates its argument and then proves something about the new contents does not prove the goal: `λ(x : &Nat). (*x := 5; refl)` does not have type `Π(x : &Nat). Id(Nat, *x, 5)`.

A `match` on an abstract value cannot pick an arm, so each arm is checked with the value refined [Split], and what follows the match is checked once, from a state in which the match has been closed off as a stuck block. Its sealed programs reduce as soon as a later split decides the match, so there is no join and no loss of information. A match and a call to a function containing it run alike, but they are not convertible: conversion has no δ-rule for stuck calls, so `F(x)` is not convertible with `F`'s body inlined, nor are two functions with the same body and different names, and extracting a helper can change which equations hold by `refl`.

6.3 The two paths must agree

Every statement is evaluated along two paths: at a definition's generic call, where calls and matches on abstract values close off, and directly at each instance, where they run. [Call-type] and [Split] identify the results, so if the paths take a decision differently, and the decision changes a result, a statement proved on one path is used on the other, where it may be false. Where the generic call sees abstract values and sealed programs, an instance sees constructors, so a decision read from a normal form can differ between the paths, and one read from syntax and declared types cannot. Most of Ochr's side conditions are this one principle, *no decision that changes a result is read from a normal form*, applied to each decision the machine makes:

- *What is erased.* A term is erased exactly when it is a proof, that is, its declared type is a proposition, or when it forms a type: a type former, a call to a function whose declared codomain is a sort, or anything in a type position. A `let`, sequence or match that computes a type is not erased as a whole; its parts decide. A *declared type* is read from the declared types of the term's heads, without normalising (Section A.4.1), and the checker decides erasure exactly so, in a pass before it runs a term.
- *What a stuck block captures, and how.* Pattern variables are resolved to places first; the block takes the maximal places its arms mention, moved if an arm moves them, borrowed if an arm writes or borrows them, and copied otherwise.
- *Which places an observation reads.* The footprint is read off the syntax of the two sides, and every owner of each place is observed.
- *Which calls recurse, and on what.* A recursive function occurs only as the head of a call, in nested functions too, and passes a strict subterm of its parameter's value on entry.
- *How a call is closed off.* Whether [Close] returns a borrow is read from the callee's declared codomain, and `&` occurs only at the top of a declared type.
- *Whether a match is on a proof.* It is read from the declarations of the constructors the arms name, and the arm from the head of the scrutinee's type (Section 6.6).

Each of these was once read from a normal form, and each time the result was a closed proof of `False` or an accepted program that goes wrong (Figure 15).

Erased terms leave no trace. Every erased term, nested ones included, runs on its own private copy of the environment, which is then discarded, so running a proof and skipping it are indistinguishable, and the machine skips proofs, as a compiled program would. A write inside a statement lands on the statement's copy and is lost with it.

6.4 Types carry what the machine needs

Decisions read from declared types agree on the two paths only if conversion preserves declared types, that is, never identifies two things that the machine treats differently. Two independent reviews of an earlier version of Ochr found closed proofs of `False` exactly where this failed (Figure 15): conversion let a function whose codomain computes to `Prop`, and whose calls therefore run, stand where a function into `Prop`, whose calls are erased, was expected; and a type such as `T(Z)`, with `T(n) : P(n)` and `P(n)` computing to `Prop`, was a proposition by computation but not by declaration. Two rules close the gap:

- *A Π-type records its class* (returns types, returns proofs, or other) *and whether it returns a borrow*, as its codomain term determines them when it is formed, and Π-types that differ in either are not convertible. A function value is then used only at a type of its own class, so the class that the machine reads from the value at a call is the one that the static type promised.
- *Sorts are syntactic.* Whatever is written where a type is expected must have a declared type that is syntactically a sort. Evaluation preserves declared types, and universes are not cumulative, so a type's declared sort is its computed sort: there is one notion of proposition, and every value of a proposition is `⋆`.

Both rules only remove identifications, so the set model validates every conversion that remains. The cost is that a codomain must be written in its class, `Prop` rather than a constant that evaluates to `Prop`, and that a family whose sort is computed, such as `T`, cannot be used as a type. By the same principle, two function values are convertible only if their generic calls leave the same observation, results and writes alike (Section A.3).

A later review found another closed proof of `False`, the fourth that a cold reader has found, through a decision that our list had missed: the type at which an observed place is compared. In one arm of a split on `n`, a stuck call `g(())`, with `g : Π(u : Unit). T(n)` and `T(Z)` computing to `Box(Unit)`, was generalised to a fresh value of type `Box(Unit)`. In the other arm the call has type `Box(Bool)` but derives the same sealed text, which mentions `g` and not `n`, so it was replaced by that value with its stale type, and comparing `MkBox(true)` with `MkBox(false)` at `Box(Unit)` gave `⊤`, since `Eq` then treated any two values at `Unit` as equal. Two rules that on their own only admit more good programs, generalisation records shared across arms and that η rule, together admitted a false one. It is fixed: a generalisation record belongs to the arm that made it, and η for `Unit` is a property of values whose type is known, not of `Eq`.

6.5 Stuck, never ill-typed

Where the rules cannot decide, the machine stays stuck rather than produce a value of the wrong type. `J` computes only when its endpoints are convertible, as Lean's `Eq.rec` does: under an absurd hypothesis a cast stays stuck, where reducing it would put `5` at type `Bool`, or embed the untyped λ-calculus in conversion. A match with no arms is erased in a proof position, and anywhere else it is stuck when reached, which happens only when a refinement has made a hypothesis false. [Access] ends every borrow whose loan might lie in the accessed place, including anywhere inside a neutral, whose layout is unknown. Closed runs never meet these situations; open ones do, when a sealed program is re-run under a refinement, and staying stuck there is what keeps the invariants of Section A.5, values at their types among them, in open terms.

6.6 Proofs are matched by their type

A proof cannot be inspected, since its value is `⋆`, so a match on a place whose type is an inductive declared in `Prop` is decided by that type, never by the content. `match h {}` on `h : False` has no arms and is well typed at any result type: this is ex falso. A match on `True` or `And` has one arm, whose fields hold `⋆`; a declaration with several constructors, such as a user's `Or`, gets one arm per constructor, every arm must be a proof, and the match is erased. As in Lean [11], a match on a proof may produce data or a type only if the inductive has no constructors, or one whose fields are all proofs (subsingleton elimination): otherwise `IsL(h : Or(True, True)) : Bool`, returning `true` on `Inl` and `false` on `Inr`, would tell apart two proofs that proof irrelevance identifies, and would have no set-theoretic meaning (note 10 of Section A.7).

6.7 The conditions, by principle

Figure 15, in the appendix beside the notes that work each case, lists the side conditions by principle, each with the counterexample that forced it, found while designing the calculus or by a review; only subsingleton elimination is Lean's. Each counterexample is a regression test, most of them beside an accepted near-miss (Section 8), so each condition is *necessary* for some verdict. The tests do not show that together they *suffice*: that is the stability conjecture (Conjecture 2), and the reviews' counterexamples were cases that no earlier test anticipated. Nor does removing rules one at a time find every rule that matters: a closed `False` can need two rules removed together, neither of which admits one alone.

Property	Status	Evidence
The machine		
1. Machine steps preserve well-formedness (Section A.5).	assumed for F	Assumption 11 (T1)
2. Frame: a call affects only what it is passed, and its effect is its isolated run, plugged back into the caller.	assumed for F	Assumption 11 (T2); the fuzzer's frame-property oracle
3. The sealed programs of [Close] compute the call's result, the final contents of its borrowed places and its backward function.	conjecture	the checker's traces (Section 8); the fuzzer, which compares re-run sealed programs with direct runs
4. The order in which borrows end does not change the resolved state.	assumed for F	Assumption 11 (T3)
5. The machine is deterministic.	by construction	the checker's machine is a function
6. Substituting a value for a returned borrow's hole is injective, and so is the caller's context around it when every owner of the hole is observed; with fewer owners it can be constant.	assumed for F	Assumption 11 (T4); with fewer owners, a counterexample of Figure 15
7. Concrete runs of programs accepted by [Rec] terminate.	conjecture	for F, part of Theorem 4, under its assumptions
Typing		
8. Stability: the two evaluation paths take the same decisions (Conjecture 2), a substitution lemma for typing decisions.	conjecture	the counterexamples of Figure 15 and their near-misses (Section 8), and the differential fuzzer; a probe corrected item (6), footprints (`FootprintProbe`)
9. Naturality and adequacy (Conjecture 3): refining and running commute once every borrow is ended, so an `Id` computed on abstract inputs holds on every concrete input.	conjecture	the fuzzer; the form without resolution is false (the `Pick` example below)
10. Consistency: no closed term has type `False` (Conjecture 1).	conjecture	proved for a fragment, conditional on naturality and the machine's invariants (Section A.8); the model sketch below; the regression suite

Figure 9: What Ochr is meant to satisfy, and the evidence. Nothing here is mechanised. "Assumed for F" marks the invariants that the paper proof of Section A.8 takes as an assumption.

7 Metatheory

This section states the properties Ochr is designed to have and the evidence for each (Figure 9). None of them is mechanised. Soundness is proved on paper for a first-order fragment, conditional on two stated assumptions (Section A.8). Consistency, the agreement of the checker's two evaluation paths, and adequacy are conjectures; for them we sketch the model and the argument we expect. The rest of the evidence is the checker's regression suite (Section 8).

7.1 The model, as a sketch

Following Aeneas [23], a computation is read as a pure function from the current contents of its borrows to its result and their final contents; types get a set-theoretic interpretation after Carneiro [11].

What consistency would be relative to. Computations translate into CIC_L, the intensional type theory of Lean 4 [39], with an impredicative, definitionally proof-irrelevant `Prop` and `propext`. Types need more: Ochr's conversion identifies propositions that CIC only proves equivalent, through the `Eq` rules (Section 5) and through [Call-type], so no translation into intensional CIC preserves conversion. We therefore interpret types directly in sets, following Carneiro's construction, which extends Werner's set model of the calculus of inductive constructions [56], with one inaccessible cardinal per universe level. Carneiro needs unique typing to decide whether a `λ` or `Π` denotes a proof or a set-theoretic function; in Ochr that decision is syntactic (Section 6), so the interpretation can be defined by recursion on derivations. Borrows are only of `Type₀` (Section 3): borrowing a universe would let `Π(x : &Type₀)(a : *x). *x` live in `Type₀`, which would be impredicative, with no set model.

The interpretation (Section A.6). An inductive type in `Type₀` denotes its least set, one in `Prop` a subsingleton, and `Eq` a truth value. A function type with borrow parameters denotes functions that return, besides their result, the final contents of the borrowed places, and, for a returned borrow, a backward function from its final value to those contents, which must be injective: [Call-type] forces this, and it is false of a Rust function that returns a leaked `'static` borrow (Section 10). Terms translate to state-passing functions on *views*, in which a loan is a *hole* for its borrow's final content, and a symbolic and a concrete run are related through their *resolutions*, which end every borrow. Four invariants of the machine make this well defined (Section A.5), and that machine steps preserve them is property 1.

Soundness, as a conjecture. For a program accepted by [Def] and [Rec] and every valuation of its abstract values, we conjecture that types of sort s denote elements of s's universe, that convertible types denote the same set, that every term denotes a function from views before its run to its type's denotation and views after it, and that every definition lies in R. The proof we expect is by induction on derivations, with most cases reading the translation backwards. The `Eq` rules hold because propositions with the same truth value are equal; injectivity and disjointness are instances, since constructors are injective and have disjoint images (in CIC_L, `propext` applied to the no-confusion property of constructors). A match on a proof is interpreted through the proof's type, which subsingleton elimination makes well defined. The hardest case is [Call-type], which needs the frame property (property 2) extended to types, the injectivity of contexts (property 6) and naturality (property 9).

Conjecture 1 (consistency). Relative to ZFC with one inaccessible per universe level, no closed term has type `False` (nor, therefore, `Eq(Nat, Z, S(Z))`, which computes to it).

Since `J` computes only on convertible endpoints, conversion has no equality reflection. Whether normalisation, and so type checking, terminates is open: types may combine large elimination, `J` with type-valued motives and an impredicative, proof-irrelevant `Prop`, which Abel and Coquand show can defeat normalisation [1]. Lean is in the same position, since Abel and Coquand's counterexample applies to it too. The model interprets derivations, so consistency does not depend on termination.

7.2 The two paths agree

A statement is evaluated along two paths, at a definition's generic call and directly at each instance, and [Call-type] and [Split] identify the results (Section 6.3). A *refinement* α substitutes values for abstract values, definable closures for abstract functions, and values for holes, each of a type convertible with the one it replaces; it covers case splits and call-site instantiation, where an argument's type need only convert to the parameter's.

Conjecture 2 (stability). Let t be checked at Ω and let α be a refinement. Compare the run of t at Ω, with α applied afterwards, with the run of $t\alpha$ at $\Omega\alpha$. At every subterm both runs reach: (1) the same subterms are erased, and the same matches are decided by the type of a proof; (2) each closed-off stuck block captures the same places, in the same modes; (3) each match's arms are well typed on both runs or on neither; (4) each call has the same class and [Close] row; (5) each observed place is compared at the same type; (6) the footprint of each observation made at Ω contains the one made at $\Omega\alpha$, and α turns the equation for each extra place into $\top$, so the two `Id` types are convertible by `And`'s unit laws.

The stability conjecture is a substitution lemma: typing decisions commute with instantiation, for a type theory whose conversion is an evaluator. We expect a proof by induction on the machine's rules, in which the decisions are the rules' side conditions that read something other than a normal form: a declared type, the syntax of a term, or a place. Item (6) is containment, not equality. A returned borrow's hole sits in the fill of every place it may point into, so at an abstract `n` the owners of `Pick(n, &a, &b)` are `a` and `b`, and an `Id` about writes through it has an equation for each. At `n = 0` the only owner is `a`. Refining `n := 0` after the type is formed leaves `b`'s fill without the hole, so both sides leave `b` with the same value and its equation computes to $\top$; the checker then shows the refined type as `False ∧ ⊤` where the direct path gives `False`. The type formed before the refinement is the stronger one, so the difference costs nothing.

Each decision is a function of syntax and declared types (Section 6.3), a refinement changes only normal forms, and conversion preserves declared types (Section 6.4). Earlier statements took refinement to substitute values of

the same type, not a convertible one, and the reviews' closed proofs of `False` lay in that gap; each earlier list of decisions also missed one, most recently item (5).

Testing it. A differential fuzzer generates random typed programs and statements, with borrows, returned borrows, stuck matches, closures, `Id`, proofs, and the shapes of both reviews' attacks. It checks each statement at its generic call and compares every refinement with a direct run of the refined instance, with oracles for value agreement, truth at ground instances, proof irrelevance, the frame property and adequacy, and it runs each accepted data function at ground inputs and compares the result with the function's erased run. In 10^6 generated cases no refinement disagrees with the direct path on a value, no execution of a generated program disagrees, and the checker rejects no true statement the fuzzer poses; 65 findings of one class, values local to one match arm that appear in a stuck block's inferred type, were re-triaged with a targeted generator against the closed proof of Section 6.4, and are fail-safe in the shapes it generates (5,000 cases, no false acceptance). While the checker could still switch rules off, the fuzzer found the failure for 17 of the then 20 rules whose removal admitted a closed proof of `False`, both reviews' attacks among them; the misses were three rules about declarations, which it does not generate. It has also found a closed proof of `False` in the checker's implementation, and it decided when the move semantics could be turned on: accepted generated programs that failed at runtime, where stuck blocks hid moves, went from 15,567 in 10^6 to 950, 10 and 0 as the gaps it exposed were closed. Its record is not clean either. A refinement that leaks from one match arm into another gave a closed proof of `False` twice, first through a closure's capture, found while building the array library, and then through generalisation records reused across arms, found by a reviewer (Section 6.4); both times the fuzzer's own analysis had called the family fail-safe. An argument about a class of findings is not a proof. Two later findings are fixed: an accepted function could fail when a borrowed local was dropped (now the [Drop] rule), and a match on a value of stuck type took its constructors from its patterns.

Naturality holds only up to resolution, which costs completeness. With `Pick(n, x, y) := match n { Z => x, S _ => y }` and `n` abstract,

```
let r = Pick(clone(n), &a, &b); let z = b; match n { Z => *r := 5, S _ => () }
```

runs for every concrete `n` but is rejected: `Pick`'s hole sits in the fills of both `a` and `b`, so reading `b` ends `r` symbolically, and `r` is dead in the `Z` arm. The refined symbolic state and the concrete one agree only after every borrow is ended (so the stronger form, equality before resolution, is false), which is why naturality is stated up to resolution; Rust likewise treats `r` as borrowing both places while it is live.

Conjecture 3 (naturality). Let every definition of the program be accepted, let $\Omega \vdash t \Downarrow \langle \Omega', v \rangle$ on the symbolic path, and let α be a refinement. If the run of $t\alpha$ at $\Omega\alpha$ ends, it ends without error, as $\langle \Omega'', v'' \rangle$, and after every borrow is ended in both, the final states and results are equal up to a renaming of loans: $\rho(\Omega'\alpha) = \rho(\Omega'')$ and $\rho(v\alpha) = \rho(v'')$, where α re-normalises sealed programs and ρ ends every borrow. In particular (adequacy), an `Id` that computes to $\top$ at Ω computes to $\top$ at $\Omega\alpha$.

Whether the run of $t\alpha$ ends is property 7; the requirement that every definition be accepted is needed because a call closed off at Ω runs at $\Omega\alpha$, and only its own [Def] check says it runs without error there.

A theorem for a fragment. For a first-order fragment F (natural numbers, `Unit`, `True`, `False`, `And`, `Eq`, `Id`, top-level functions with borrow parameters and returned borrows, [Close] and [Split], but no closures, stuck blocks, user inductives or type families), Section A.8 proves on paper:

Theorem 4 (soundness of F, conditional on naturality and the machine's invariants). If naturality holds step by step (Assumption 10) and the machine preserves well-formedness, has the frame property, ends borrows independently of order and has injective contexts (Assumption 11), then for every accepted program of F and every concrete input, each data function runs without error and a borrow it returns is live, and each lemma whose hypotheses hold has a true conclusion. Consistency of F and adequacy of `Id` follow.

8 Implementation and evaluation

8.1 The checker

We implemented the calculus as an executable reference checker in Lean 4, about 7,000 lines with no dependencies, plus about 6,500 lines of examples and tests and 5,100 lines of case studies. Its purpose is fidelity, not performance: a hand derivation can be charitable where the rules are silent, and a program cannot. Programs are written in a concrete syntax close to the one used here, in named blocks of definitions. The checker runs inside Lean's editor, checking each definition as you type (Section A.9). A `def` must be accepted and a `reject def` rejected. Each verdict is an assertion checked during the build, and each test file also asserts how many verdicts it contains, so a file that silently checks nothing fails the build. The checker's trace of `AddMZero` reproduces the derivation of Section 2 symbol for symbol. The suite contains every program this paper prints (the artifact's notes map each to its test), every program that must be rejected, among them every counterexample of Figure 15, and every accepted program that went wrong while we designed the calculus. All 1,558 verdicts are as expected: 1,140 in the examples and tests, and 418 in the case studies.

Why each rule is there. The checker has no switches for turning its rules off. Which rules it applies is its own business: a test says only whether a program is accepted, rejected, or runs to a given result. Each side condition is justified instead by the counterexample that forced it (Figure 15). That counterexample is in the suite as a `reject def`, most of them beside an accepted near-miss that differs only in the offending part, so the rejection is seen to come from that part. A few near-misses are weak, restating a reflexive equation or an existing test, and two counterexamples have none: one pins a known incompleteness, and the other's near-miss would need an inhabitant of an empty type. Whether a rule is still needed is checked when the question arises, by removing it on a throwaway branch and running the suite. That shows a rule is needed for some verdict, not that the rules suffice, and not that a rule is sound. The checker decides erasure as Section 6.3 states it, in a pass over declared types before it runs a term; its earlier classification after the fact is kept as an internal assertion, which agrees on the whole suite.

8.2 A verified hash map, against Aeneas

We wrote the resizing hash map that Aeneas verifies [23] as one Ochr program, and proved Aeneas's theorems about it. The baseline is Aeneas's 2022 development with its F* backend, where Z3 discharges the proof obligations; Aeneas has since gained a Lean backend, which we have not compared against. The program is Aeneas's, recursive as theirs is, and generic in the value type `V`: buckets are association lists; the table is an array of buckets (Section 8.3), and an operation reaches the key's bucket through the array's element borrow, as Rust's `&mut slots[i]` does; `Insert` doubles the table once the entries reach the number of buckets, by inserting every entry again; and there are `Get`, `MapGetMut`, `Remove`, `ContainsKey`, `New` and `Clear`. The theorems are stated about these operations directly. Aeneas states them about a model: it translates the Rust into pure functions, the user writes a second, simpler pure model of the map (`find_s`, the invariant), proves that the translated functions agree with that model, and proves the properties of the model.

What is written. Ochr's development is 36% smaller than Aeneas's in lines (Figure 10). Most of the difference is that Ochr writes no model and no agreement proof, and trusts no translation: in Aeneas those layers are 29% of the hand-written lines, and dropping them, net of Ochr's own statement vocabulary, saves 596 lines. Ochr's property proofs take 1,350 lines against Aeneas's 1,680, statements included, but we claim no advantage in the proofs, which spell out every case analysis that Aeneas leaves to Z3; the saving is the model and agreement layers that Ochr does not write. The margin depends on counting: Aeneas's file contains 183 lines of dead code, and without them Ochr's development is 32% smaller. The case study also shaped the language. The first complete version was 21% smaller, its proofs 1.05 times Aeneas's, and the proofs then shrank by almost a quarter, through three forms added during the study, `rewrite` (Section 2), a destructuring `let` and `split f in t` (described below), and through one change to the proofs: `get_mut`'s theorems now follow from `insert`'s, because writing `w` through the borrow `MapGetMut` returns is proved equal to `InsertNoResize` of `k` and `w`, a theorem between two in-place operations of the same program. Aeneas reports four person-days for its proofs; we have no comparable measure of effort.

	Spec, model	Implementation	Agreement	Property proofs	Total
Hash map					
Aeneas	321	210	459	1,680	2,670
Ochr, first version	144	205	0	1,765	2,114
Ochr, final	184	162	0	1,350	1,696
Quicksort					
Verus	6	36	0	66	108
Ochr, first version	30	52	0	826	908
Ochr, final	30	53	0	784	867

Figure 10: The case studies in lines, without blanks and comments. Line counts measure the size of what is written, not the effort of writing it. *Spec, model*: Aeneas's trusted model of the map (82 lines) and the pure models its proofs go through; the vocabulary of Ochr's statements and the definitions its proofs use; Verus's two spec functions. *Implementation*: for Aeneas, the Rust and 9 termination measures. *Agreement*: proofs that the translated functions equal the model; not counted are the 550 lines of F* that Aeneas generates and trusts. *Property proofs* include the theorem statements. *First version*: before `rewrite`, destructuring `let` and `split`, and for the hash map before `get_mut` was specified by `insert`. The final rows are checked with the current rules, and the final hash map is generic in its values; the first versions predate moves. Ochr's rows count everything written for the case study and none of its general library (827 lines: booleans, words, lists, options, logic and arrays), as Aeneas's primitives and Verus's `vstd` are not counted; the library holds the counting lemmas that Ochr's permutation proof uses (126 lines), where Verus's `perm` and swap lemma are counted. The split of Verus's file into columns is ours.

Coverage. Every theorem of Aeneas's interface has a counterpart that the checker accepts: lookups after `insert` (with resizing), `get_mut` and `remove`, at the key and at any other key; the length, which grows by one exactly when the key was absent; the invariant (the length counts the entries, the keys of a bucket are distinct, every key lies only in its own bucket), kept by every operation; the load factor; and `new` and `clear`. Aeneas's statement about `remove` (`hash_map_remove_back_lem` in `Hashmap.Properties.fsti`) asserts the invariant of the map it was given rather than of the map it returns, which its proof establishes; Ochr's is about the result.

The program is its own specification. Where Aeneas says that inserting `key` leaves every other binding unchanged, `forall k'. k' <> key ==> find_s hm' k' == find_s self k'`, about the model `find_s`, Ochr's statement uses the map's own lookup. `Find(V, m, k)` runs the in-place bucket lookup on the map it is given, as `Add` runs `AddM` on its argument in Section 2, and copies the value it finds; `Get`, which returns a borrow of the value instead, is proved to read the same. In a statement, `m` is a copy of `*hm`:

```
InsertFindOther(V : Type, hm : &HashMap(V), k : Word, v : V, k2 : Word,
                h : Eq(Bool, EqB(k, k2), false)) :
    Id(Opt(V), (InsertNoResize(V, &*hm, k, v); Find(V, clone(*hm), k2)),
      (let r = Find(V, clone(*hm), k2); InsertNoResize(V, &*hm, k, v); r)) :=
  let HM(n, _, MkArray(s)) = *hm; split BInsert in
    SlotInsertFindOther(V, Succ(n), &s, Idx(k, n), IdxLt(k, n), Idx(k2, n), IdxLt(k2, n),
      k, v, k2, h)
```

The destructuring `let` takes `*hm` apart, and its array, as a one-arm `match` would. `InsertNoResize` records the new length according to what the bucket insert returned, a result that is stuck on the unknown bucket. `split f in t` finds, in the goal, the first sealed program whose run is stuck on the result of a call of `f`, generalises that result as [Split] generalises a sealed scrutinee, and checks `t` in every arm; like `rewrite`, it is a typing form that never runs. Here `split BInsert` splits on the insert's result, and in both arms the lemma for the slots applies. That lemma is the bucket lemma lifted through the element borrow by a recursion on the index that follows the borrow's own, so the untouched buckets stay in place without a congruence step; the proof may take the abstract array apart, which runtime code may not. The invariant is stated the same way: the proposition that every key lies only in the bucket the index returns for it is `Π(k : Word). OnlyIn(V, Succ(n), s, Idx(k, n), k)`, whose definition uses the bucket lookup.

Where the programs differ. Ochr's numbers are unbounded, so there are no overflow obligations and no failing operations; in Aeneas's proofs, overflow bounds take about 3% of the lines, and the cases where an operation fails about 6% more. The map stores its size beside the array, whose type depends on it, and a resize replaces both (Section A.4.8); the index's bound is a lemma, not a runtime check. `Get` and `MapGetMut` return a borrow of a present key's value and take a proof that the key is present, since Rust's `Option<&mut V>` stores a borrow inside data; `Insert` and `Remove` return the value they move out. The load factor is fixed at one. As in Rust, reading a value moves it, and reading a number that is only computed with copies it: keys, sizes and lengths are `Word`s. The code copies no value; the statements' lookup `Find` copies the value it finds, which costs nothing at run time.

What is still costly. Each lemma is stated three times, for a bucket, for the array's view and for the map; a single lemma about lifting through the index would need quantification over programs. And there is no automation: every case analysis that Aeneas leaves to Z3 is written out, as a match or a `split`. The thesis holds on these numbers only as far as they go: one development instead of three layers, a third smaller in lines, which measure the size of what is written and not the effort of writing it.

8.3 Arrays and quicksort

Arrays are a library. Ochr has no built-in array. `Cells(E, n)` is a type computed by recursion on `n`, with exactly `n` elements; a view `Slice(E, n)` and an owned `Array(E, n)` wrap it, so the length lives only in the type. An array never changes its length, so a growable vector is user code over one, `Vec(E) := MkVec(len : Word, cap : Word, buf : Array(Opt(E), cap), hl : Le(len, cap))`, whose buffer's type depends on its capacity; `Push` moves the elements into a larger array when it is full, and the dependency may be broken while the vector is mutated, and is re-checked wherever the value must be whole again (Section A.4.8); a proof field such as `hl`, assigned while part of the vector is lent, must hold whatever the borrow leaves there. The model functions (reading, writing, taking, dropping and joining elements) are ordinary Ochr definitions, and proofs reason about them directly. Five functions are primitive at runtime: taking the view of an array, `GetMut` (a borrow of one element, through which elements are read and written), `WithSplit`, and making an array from a function of the index or from a list (the model of an array literal, which a compiler lowers directly, building no list). The checker runs each one's Ochr body as its model; compiled code would call native code, which must be trusted to implement the model and to be the only code that touches the representation. The checker enforces the second: the model types are declared `abstract`, so their constructors are used and matched at runtime only in the natives' `implemented by` bodies and in model functions, which take or return an unsized value that runtime code cannot produce, and a slice is `unsized`, so it is only ever borrowed, never read, moved, assigned, cloned or matched. A part of an array is borrowed only while a continuation runs, since a pair of borrows cannot be returned (Figure 1): `WithSplit` passes borrows `l : &Slice(E, k)` and `r : &Slice(E, Sub(n, k))` of the two pieces to a function, and joins what they hold when it returns. So calling any `g` on the first `k` elements leaves the view equal to `JoinS(E, n, k, (let c = TakeS(E, n, k, *s, h); g(&c); c), DropS(E, n, k, *s))`, the old rest joined to `g`'s result, by `refl`.

Quicksort is a capability result: a sort of this cons-cell model of arrays, through an interface designed to be implemented by flat arrays, verified in full, with sub-range borrows, in a dependent type theory where the program itself appears in the proofs. Lomuto's quicksort partitions the view in place by swapping, then borrows the two sides of the pivot with `WithSplit` and sorts each. Until Ochr has recursion on a measure, it recurses on fuel. Reads move, as in Rust: indices, lengths and the elements sorted are `Word`s, which are copied, and six `clone`s copy a generic element, a whole view or the recursion where one is used twice (five in the library, one in quicksort). With fuel equal to the length, the checker accepts its correctness, stated about the in-place program: the result is sorted, and no value's count changes.

```
QSCorrect(n : Word, s : &Slice(Word, n), q : Word) :
    (let c = *s; QS(n, n, &c); Sorted(n, c)) ∧
    (let old = clone(*s); Eq(Word, Count(q, n, (QS(n, n, &*s); *s)), Count(q, n, old))) :=
  ⟨QSSortedFull(n, n, s, LeRefl(n)), QSPerm(n, n, s, q)⟩
```

The proofs are long (Figure 10). Verus's quicksort [30], whose partition is a loop, proves the same two properties in an eighth of the lines, with Z3 discharging the obligations and `vstd` supplying multiset lemmas. Half of Ochr's proofs (392 lines) establish the partition's positional invariant, which Verus states as two quantified loop invariants and leaves to the solver, while in Ochr every case split and every step of index arithmetic is written out. This comparison is not a test of the thesis of Section 1.1, about developments that write the program twice (Section 8.2); it measures what SMT automation buys, and automation is what Ochr lacks. Ochr needs no solver or translation. `rewrite` made the proofs 7% shorter in lines; reading and swapping through element borrows, not natives, made them 3% longer.

9 Related work

Functional translation of borrows. Aeneas [23] translates safe Rust into a pure functional program in Lean, F* or Coq. Its symbolic semantics of the low-level borrow calculus is the direct ancestor of our machine: we take its values with loans and borrows, its lazy ending of borrows, and its insight that a mutable borrow parameter is a forward computation plus a backward function. Later work gives that symbolic semantics joins and loops and proves it sound as a borrow checker [24]. Aeneas translates eagerly, whole-program, once, and the user reasons about the translated artifact and a hand-written model of it (Section 8.2). Ochr never produces a translated program: the same state-passing content appears lazily inside conversion, only as far as an equation needs, in source syntax, but per call rather than per region, and without loops or nested borrows (Section 4). Aeneas is modular in that it never unfolds a callee; Ochr must unfold, because unfolding is what definitional equality is, and closing off is what keeps the unfolding finite. Electrolysis [52] translated safe Rust into Lean before Aeneas, by the same functional reading of mutation, and a backward function is a lens's put [18], specialised to one call. Rupicola [44] goes the other way, compiling a functional model into low-level code with a proof that they agree, so the model is the program the user reasons about.

Prophecies. RustHorn [38], RustHornBelt [37] and Creusot [15] model a mutable borrow as its current value paired with a *prophecy* `^x` of its final value. A prophecy is available before it is resolved and specifications may mention it, which makes those specifications modular and lets them cover borrows stored in data and loops. Prusti's pledges [6] specify what holds when a returned borrow expires, the specification-level analogue of Ochr's holes. Ochr's hole resembles an unresolved prophecy only superficially: it is inert, specifications cannot name it, and it exists only for borrows returned by stuck calls. Borrow parameters are modelled as Aeneas does, by an owned place whose content the call's end returns, and unlike prophecy-based models this needs the backward functions of borrow-returning functions to be injective (Section 7.1).

Foundations of Rust's borrows. RustBelt [28] proves Rust's type system sound in a semantic model built in Iris, together with several standard libraries implemented with unsafe code, and Stacked Borrows [27] and Tree Borrows [54] are operational aliasing models that say which accesses invalidate which pointers when unsafe code is present. Ochr's [Access] is a much coarser operational rule for safe code only, under which using a place ends every borrow derived from it; we have not related it to either model.

Specification-based verification of imperative code. Verus [30], Creusot, Prusti [7], Flux [31] and VeriFast [25] verify imperative code against specifications written separately, in pre- and postconditions, refinement types or separation logic, and discharge the obligations with automated provers. Dafny [32] and Why3 [16] let specifications call pure functions and ghost code, Why3′s regions [17] control aliasing statically so that proofs need no memory model, and Linear Dafny [34] verifies in-place code with linear types in one language, discharging the rest by SMT. RefinedRust [22] and Gillian-Rust [9] extend such verification to unsafe code, RefinedRust with a foundational soundness proof, and SPARK [26] adds Rust-like ownership and borrowing to Ada so that its deductive verifier can reason about pointers. Low* [45] and Cogent [43] (Section 1.1) still keep the proof apart from the program: a Low* proof uses a function's pre- and postconditions over a memory model, and a Cogent proof is about the functional semantics that the compiler generates in Isabelle/HOL, whereas Ochr's conversion unfolds the in-place code itself. Characteristic formulae [13] reason about imperative code in Coq without a separate model, and symbolic execution with separation logic [10], on which VeriFast builds, is the technique

closest to Ochr's typing. Hoare type theory [41] and Ynot [42] internalise the triples in a dependent type theory, as do F*'s separation-logic DSLs Steel and Pulse [19]; F*'s Dijkstra monads [3] internalise weakest preconditions, and `reify` turns a stateful computation into a state-passing function, so two programs can be compared through their reified pure functions. Relational logics such as ReLoC [20] prove two programs equivalent inside a program logic. Relational Hoare type theory [40] states relations between two runs of effectful programs inside a dependent type theory, as `Id` does, through a monad and its assertions. In Ochr the specification of an in-place function is another program, or a theorem about the function itself, equality of computations is a type, and it is decided on the effectful code by conversion where possible.

Effects in dependent type theory. The fire triangle [48] shows that substitution, dependent elimination and observable effects cannot coexist consistently, and ∂CBPV organises the possible compromises. Closed Ochr programs are not observably effectful in their sense (Section 1.3). For open terms Ochr takes the call-by-value way out, keeping dependent elimination and restricting substitution to values; Lepigre's PML [33] relaxes the value restriction to observational equivalence with a value, where Ochr's typing runs the argument and substitutes its value, and unlike the call-by-value theory Pédrot and Tabareau obtain through ∂CBPV, Ochr has a dependent `let` and large elimination. Zombie [12] proves equations between programs of a call-by-value fragment by evaluating both sides to a common term, as Ochr does for programs that are total but mutate their environment. Krishnaswami, Pradic and Benton [29] integrate linear and dependent types with linear state, but their types depend only on non-linear terms, so a type cannot mention the current contents of a linear resource, as Ochr's types do. Vákár's dependent call-by-push-value [55] and Ahman's eMLTT [4] give effects and dependent types a common framework; eMLTT's definitional equations come from an algebraic theory of global state, while Ochr's state is local and borrow-shaped and its equations come from running the program.

Observational equality. Observational type theory [5, 47] computes the equality type by the structure of the type: equality of pairs is a pair of equalities, equality of functions is pointwise. Ochr's `Eq` computes by the structure of constructor values (injectivity and disjointness), and `Id` applies the idea at the type of observations, which is why `Eq` stays primitive rather than an inductive family. Unlike OTT, Ochr compares functions by their generic observation rather than pointwise, and its cast `J` computes only on convertible endpoints. Pujet and Tabareau's impredicative OTT [46] studies the same combination of an impredicative, definitionally proof-irrelevant `Prop` with a computing equality, and Gilbert et al. [21] show why an eliminator of a definitionally irrelevant equality must not reduce on open terms, which is the reason for Ochr's rule for `J`.

Inductive propositions. Ochr's connectives are inductive declarations in `Prop`, as in Lean, Coq and Agda, rather than impredicative encodings such as `False := Π(P : Prop). P` from the calculus of constructions [14], and matching on a proof follows Lean's subsingleton elimination [11].

Pure languages with in-place updates. Lean 4 performs destructive updates on values it can prove unshared at runtime [50], and Koka's fully in-place fragment [35] guarantees statically that a function allocates nothing. Lean's `do` notation [51] writes local mutation that is elaborated into pure code, and Liquid Haskell's refinement reflection [53] proves properties of a program by unfolding it, as Ochr's conversion does, for pure code. In all of these the programmer writes pure code and cannot express a borrow returned to the caller. Mutable value semantics [49] reads an `inout` parameter as a function returning its new value, the reading that Aeneas and Ochr's sealed programs share, but as a programming discipline rather than for verification. Linear and quantitative type theories [8], Granule's fractional uniqueness [36] and ATS [57] track resources in types, and Verus's ghost and tracked modes [30] separate specification from runtime code as Ochr's erasure does; as in quantitative type theory, a variable used in a type is not consumed.

10 Discussion

10.1 Limitations and directions

Opaque definitions. A call through an abstract function value is stuck at once and closed off, so it behaves according to its signature alone, as every callee does in Aeneas [23]; but there is not yet a way to check a body once and then hide it from conversion, which is what modular verification needs.

No `'static` borrows. Safe Rust lets a `fn(&mut u64) -> &mut u64` return `Box::leak(Box::new(0))`, whose backward function is constant, while Ochr proves that every `Π(x : &Nat). &Nat` has an injective one (Section 7.1); an opaque or foreign definition of a borrow-returning type is therefore an unchecked injectivity assumption.

Borrows stored in data. Borrow types never occur inside other types, ruling out an iterator `IterM : &(List Nat) → List (&Nat)`: the number of borrows it returns is unknown on symbolic input, so no fixed set of holes can stand for their final values. The natural generalisation is a hole for a whole structure of final values, the neutral form of an Aeneas region abstraction with a borrow projector. A fixed number of returned borrows is easier and gives `split_at_mut`.

Other directions. A shared borrow needs no sealed program of its own, since its final value is its initial one, but its interaction with mutable borrows through reborrowing and two-phase borrows is not designed. A loop would be a tail-recursive local function whose type is its invariant. Global state, interior mutability and unsafe aliasing would each break the frame property, and would have to be confined behind interfaces. Indexed families are Forded, which relies on the primitive `Eq`; native indices would let `Eq` be declared, but it would no longer compute by structure.

10.2 The checked machine and the compiled program

The machine that the checker runs, and that the proofs are about, moves data as Rust does, so `AddM` allocates nothing, and among the programs the fuzzer generates, its execution oracle finds none that is accepted and goes wrong when run (Section 7.2). What remains is the relation to compiled code: the machine runs types and proofs on discarded private copies, where a compiled program omits them. That the compiled program behaves as the checked one is, for now, an assumption. The gap is narrower than the one Section 1.1 criticises in the pure route, since where the program mutates in place is written and checked. Erased reads do not count as uses, which is what lets `x + x = 2 · x` be stated without `clone`, as in quantitative type theory [8].

11 Conclusion

The usual way to verify an efficient imperative program is to write it twice and prove that the two agree. For programs whose mutation is disciplined by Rust-style ownership, the pure reading of the in-place program can instead be generated on demand, inside the type checker, as source code: a call that definitional equality cannot run is closed off into sealed programs, which a later case split runs again, and an observational equality lets theorems about in-place code be stated about that code and proved by structural recursion. The price is a type theory whose conversion runs effectful code on two paths that must agree; we have given the principle that keeps them in agreement, and the evidence for it, but its sufficiency and the consistency of the calculus remain conjectures.

Bibliography

[1] Andreas Abel and Thierry Coquand. 2020. Failure of Normalization in Impredicative Type Theory with Proof-Irrelevant Propositional Equality. *Logical Methods in Computer Science* 16, 2 (2020).

[2] Andreas Abel. 2013. Normalization by Evaluation: Dependent Types and Impredicativity. In *Habilitation thesis, LMU Munich*, 2013.

[3] Danel Ahman, Catalin Hritcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *POPL*, 2017.

[4] Danel Ahman. 2018. Handling Fibred Algebraic Effects. In *Proc. ACM Program. Lang. (POPL)*, 2018.

[5] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational Equality, Now!. In *PLPV*, 2007.

[6] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 147:1–147:30.

[7] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification. In *Proc. ACM Program. Lang. (OOPSLA)*, 2019.

[8] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *LICS*, 2018.

[9] Sacha-Élie Ayoun, Xavier Denis, Petar Maksimović, and Philippa Gardner. 2025. A Hybrid Approach to Semi-automated Rust Verification. *Proc. ACM Program. Lang.* 9, PLDI (2025), 186:1–186:23.

[10] Josh Berdine, Cristiano Calcagno, and Peter W. O'Hearn. 2005. Symbolic Execution with Separation Logic. In *APLAS (LNCS)*, 2005. 52–68.

[11] Mario Carneiro. 2019. The Type Theory of Lean.

[12] Chris Casinghino, Vilhelm Sjöberg, and Stephanie Weirich. 2014. Combining Proofs and Programs in a Dependently Typed Language. In *POPL*, 2014. 33–46.

[13] Arthur Charguéraud. 2011. Characteristic Formulae for the Verification of Imperative Programs. In *ICFP*, 2011. 418–430.

[14] Thierry Coquand and Gérard Huet. 1988. The Calculus of Constructions. *Information and Computation* 76, 2–3 (1988), 95–120.

[15] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*, 2022.

[16] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3 – Where Programs Meet Provers. In *ESOP (LNCS)*, 2013. 125–128.

[17] Jean-Christophe Filliâtre, Léon Gondelman, and Andrei Paskevich. 2016. *A Pragmatic Type System for Deductive Verification.*

[18] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem. *ACM Trans. Program. Lang. Syst.* 29, 3 (2007), 17.

[19] Aymeric Fromherz, Aseem Rastogi, Nikhil Swamy, Sydney Gibson, Guido Martínez, Denis Merigoux, and Tahina Ramananandro. 2021. Steel: Proof-Oriented Programming in a Dependently Typed Concurrent Separation Logic. In *Proc. ACM Program. Lang. (ICFP)*, 2021.

[20] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *LICS*, 2018.

[21] Gaëtan Gilbert, Jesper Cockx, Matthieu Sozeau, and Nicolas Tabareau. 2019. Definitional Proof-Irrelevance without K. *Proc. ACM Program. Lang.* 3, POPL (2019), 3:1–3:28.

[22] Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. 2024. RefinedRust: A Type System for High-Assurance Verification of Rust Programs. *Proc. ACM Program. Lang.* 8, PLDI (2024), 192:1–192:25.

[23] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proc. ACM Program. Lang. (ICFP)*, 2022. 116:1–116:31.

[24] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. 2024. Sound Borrow-Checking for Rust via Symbolic Semantics. In *Proc. ACM Program. Lang. (ICFP)*, 2024.

[25] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. 2011. VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In *NASA Formal Methods (NFM)*, 2011.

[26] Georges-Axel Jaloyan, Claire Dross, Maroua Maalej, Yannick Moy, and Andrei Paskevich. 2020. Verification of Programs with Pointers in SPARK. In *ICFEM (LNCS)*, 2020. 55–72.

[27] Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. 2020. Stacked Borrows: An Aliasing Model for Rust. *Proc. ACM Program. Lang.* 4, POPL (2020), 41:1–41:32.

[28] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. RustBelt: Securing the Foundations of the Rust Programming Language. *Proc. ACM Program. Lang.* 2, POPL (2018), 66:1–66:34.

[29] Neelakantan R. Krishnaswami, Cécilia Pradic, and Nick Benton. 2015. Integrating Linear and Dependent Types. In *POPL*, 2015. 17–30.

[30] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023.

[31] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. In *PLDI*, 2023.

[32] K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *LPAR-16 (LNCS)*, 2010. 348–370.

[33] Rodolphe Lepigre. 2016. A Classical Realizability Model for a Semantical Value Restriction. In *ESOP*, 2016. 476–502.

[34] Jialin Li, Andrea Lattuada, Yi Zhou, Jonathan Cameron, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2022. Linear Types for Large-Scale Systems Verification. *Proc. ACM Program. Lang.* 6, OOPSLA1 (2022), 69:1–69:28.

[35] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. 2023. FP^{2}: Fully in-Place Functional Programming. In *Proc. ACM Program. Lang. (ICFP)*, 2023.

[36] Daniel Marshall and Dominic Orchard. 2024. Functional Ownership through Fractional Uniqueness. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 1040–1070.

[37] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI*, 2022.

[38] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *ESOP*, 2020.

[39] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.

[40] Aleksandar Nanevski, Anindya Banerjee, and Deepak Garg. 2013. Dependent Type Theory for Verification of Information Flow and Access Control Policies. *ACM Trans. Program. Lang. Syst.* 35, 2 (2013), 6:1–6:41.

[41] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and Separation in Hoare Type Theory. In *ICFP*, 2006.

[42] Aleksandar Nanevski, Greg Morrisett, Avraham Shinnar, Paul Govereau, and Lars Birkedal. 2008. Ynot: Dependent Types for Imperative Programs. In *ICFP*, 2008.

[43] Liam O'Connor, Zilin Chen, Christine Rizkallah, Sidney Amani, Japheth Lim, Toby Murray, Yutaka Nagashima, Thomas Sewell, and Gerwin Klein. 2016. Refinement through Restraint: Bringing Down the Cost of Verification. In *ICFP*, 2016. 89–102.

[44] Clément Pit-Claudel, Jade Philipoom, Dustin Jamner, Andres Erbsen, and Adam Chlipala. 2022. Relational Compilation for Performance-Critical Applications: Extensible Proof-Producing Translation of Functional Models into Low-Level Code. In *PLDI*, 2022. 918–933.

[45] Jonathan Protzenko, Jean-Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella-Béguelin, Antoine Delignat-Lavaud, Cătălin Hriţcu, Karthikeyan Bhargavan, Cédric Fournet, and Nikhil Swamy. 2017. Verified Low-Level Programming Embedded in F*. *Proc. ACM Program. Lang.* 1, ICFP (2017), 17:1–17:29.

[46] Lo\̈ic Pujet and Nicolas Tabareau. 2023. Impredicative Observational Equality. *Proc. ACM Program. Lang.* 7, POPL (2023), 74:1–74:26.

[47] Loïc Pujet and Nicolas Tabareau. 2022. Observational Equality: Now for Good. *Proc. ACM Program. Lang. (POPL)* 6, (2022).

[48] Pierre-Marie Pédrot and Nicolas Tabareau. 2020. The Fire Triangle: How to Mix Substitution, Dependent Elimination, and Effects. *Proc. ACM Program. Lang. (POPL)* 4, (2020), 58:1–58:28.

[49] Dimitri Racordon, Denys Shabalin, Daniel Zheng, Dave Abrahams, and Brennan Saeta. 2022. Implementation Strategies for Mutable Value Semantics. *Journal of Object Technology* 21, 2 (2022), 2:1–11.

[50] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *IFL*, 2019.

[51] Sebastian Ullrich and Leonardo de Moura. 2022. `do' Unchained: Embracing Local Imperativity in a Purely Functional Language (Functional Pearl). *Proc. ACM Program. Lang.* 6, ICFP (2022), 512–539.

[52] Sebastian Ullrich. 2016. Simple Verification of Rust Programs via Functional Purification. Master's thesis.

[53] Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2018. Refinement Reflection: Complete Verification with SMT. *Proc. ACM Program. Lang.* 2, POPL (2018), 53:1–53:31.

[54] Neven Villani, Johannes Hostert, Derek Dreyer, and Ralf Jung. 2025. Tree Borrows. *Proc. ACM Program. Lang.* 9, PLDI (2025), 188:1–188:24.

[55] Matthijs Vákár. 2015. A Framework for Dependent Types and Effects.

[56] Benjamin Werner. 1997. Sets in Types, Types in Sets. In *TACS (LNCS)*, 1997. 530–546.

[57] Hongwei Xi. 2003. Applied Type System (extended abstract). In *TYPES*, 2003.

A Formal definition

This appendix defines Ochr completely: its syntax and runtime structures (Section A.1), the machine (Section A.2), observation, `Id` and conversion (Section A.3), typing (Section A.4) and well-formed environments (Section A.5). Every derivation in the paper can be checked against it. Each group of rules names, in small type, the functions of the accompanying Lean checker (Section 8) that implement it (in `Machine.lean` unless noted). Section A.7 records the points where a choice had to be made, and, for each side condition of Figure 15, a counterexample showing why the weaker reading fails.

A.1 Syntax and runtime structures

A.1.1 Terms

t, u, A, B	$::=$	$x \mid \mathsf{Prop} \mid \mathsf{Type}_i$	variable, sorts ($i \geq 0$)
	$\mid$	$\Pi(x_1 : A_1 ... x_n : A_n).B$	dependent function type ($n \geq 1$)
	$\mid$	$\mathbf{fix}\ f(x_1 : A_1 ... x_n : A_n) : B\ \ \mathbf{by}\ x_j \coloneqq t$	function, recursive on x_j
	$\mid$	$\mathbf{fix}\ f(x_1 : A_1 ... x_n : A_n) : B \coloneqq t$	function, not recursive (λ)
	$\mid$	$t(u_1, ..., u_n)$	saturated call
	$\mid$	$\mathsf{D}(a_1, ..., a_l) \mid \mathsf{C}(a_1, ..., a_l; t_1, ..., t_k)$	inductive type, constructor
	$\mid$	$\mathsf{Eq}\ A\ t\ u \mid \mathsf{J}(A, a, b, P, h, t)$	equality (primitive), transport
	$\mid$	$\mathbf{rewrite}\ h\ \ \mathbf{in}\ t \mid \mathbf{rewrite} \leftarrow h\ \ \mathbf{in}\ t$	rewriting, a proof ([T-Rewrite])
	$\mid$	$\&A \mid p \mid \&p \mid p \coloneqq t \mid \mathbf{clone}(p)$	borrow type; read, borrow, assign, copy
	$\mid$	$\mathbf{let}\ x = t; u \mid \mathbf{let}\ x : A = t; u \mid t; u$	sequencing
	$\mid$	$\mathbf{match}\ p\ \{\mathsf{C}_1(\overline{y}_1) \Rightarrow t_1, ..., \mathsf{C}_m(\overline{y}_m) \Rightarrow t_m\}$	case analysis, one arm per constructor ($m \geq 0$)
	$\mid$	$\mathbf{split}\ f\ \ \mathbf{in}\ t$	case analysis on a stuck call found in the goal ([T-Split-goal])
	$\mid$	$\mathsf{Id}\ A\ t\ u$	equality of computations
	$\mid$	v	embedded value
p, q	$::=$	$x \mid *p \mid p.g$	places (g a field name)
$\mathcal{P}$	$::=$	$d \mid (f \coloneqq \mathbf{fix}\ ...) \mid \mathcal{P}\ \mathcal{P}$	programs
d	$::=$	$\mathbf{inductive}\ \mathsf{D}\ (x_1 : X_1 ... x_l : X_l) : s \coloneqq \mathsf{C}_1\big(\overline{g_1 : A_1}\big) \mid ... \mid \mathsf{C}_m\big(\overline{g_m : A_m}\big)$	declaration

Figure 11: Terms and programs. Embedded values occur only in sealed programs and in the code of closures, never in source programs.

Inductive types. A declaration introduces a type former D with parameters $x_1 : X_1, ..., x_l : X_l$, a sort $s \in \{\mathsf{Type}_0, \mathsf{Prop}\}$, and $m \geq 0$ constructors C_i, each with named fields $g_{i1} : A_{i1}, ..., g_{ik_i} : A_{ik_i}$, whose types may mention the parameters. Every constructor builds a value of $\mathsf{D}(\overline{x})$; there are no indices. A parameter whose declared type is syntactically a sort is a *type parameter*, any other a *value parameter* (`n : Nat`). A recursive occurrence of `D` in a field may take other arguments than the parameters, at a value parameter an earlier field (`Vec(A, m)` inside `Vec(A, n)`), so a family indexed by values is *Forded*: the index is a value parameter and each constructor stores an equation fixing it ([Ind-decl]). At parameters $\overline{a}$ the type is $\mathsf{D}(\overline{a})$, written D when $l = 0$, and the field types are $A_{ij}[\overline{a}/\overline{x}]$. A constructor takes the parameters as leading arguments, $\mathsf{C}(\overline{a}; \overline{t})$, and a constructor value records them; we omit them in examples when they can be inferred from the fields or an annotation (`Intro(h, k)` for `Intro(P, Q; h, k)`), and when there are none (`S(t)`). The library declares

inductive Nat : $\mathsf{Type}_0 \coloneqq \mathsf{Z} \mid \mathsf{S}(\mathrm{pred} : \mathsf{Nat})$ **inductive** Unit : $\mathsf{Type}_0 \coloneqq \mathsf{Tt}$ **inductive** False : Prop
inductive Pair $(A : \mathsf{Type}_0)(B : \mathsf{Type}_0) : \mathsf{Type}_0 \coloneqq \mathsf{Mk}(\mathrm{fst} : A, \mathrm{snd} : B)$
inductive True : Prop $\coloneqq \mathsf{I}$ **inductive** And $(P : \mathsf{Prop})(Q : \mathsf{Prop}) : \mathsf{Prop} \coloneqq \mathsf{Intro}(l : P, r : Q)$

`()`, `A × B`, `(a, b)`, `⊤`, `refl`, `P ∧ Q` and `⟨h, k⟩` are notation for `Tt`, `Pair(A, B)`, `Mk(a, b)`, `True`, `I`, `And(P, Q)` and `Intro(h, k)`; and `t.0`, `t.1` are the first and second fields, so `p.0` is the predecessor place of a number. A pair is taken apart like any value, by a match or through its field places; there is no η rule. `False`, `True` and `And` are known to conversion, because the computation rules of `Eq` produce them (Section A.3); nothing else distinguishes them from other declarations.

Binding. In `Π(x̄:Ā). B` and `fix f (x̄:Ā) : B … := t`, each $\mathtt{x}_i$ is bound in $\mathtt{A}_{i+1}$ … $\mathtt{A}_n$, in `B` and in `t`. With `by` $\mathtt{x}_j$, `f` is bound in `t` and nowhere else; without `by`, `f` is not bound at all, and in no case is `f` bound in `Ā` or `B`. `let x = t; u` binds `x` in `u`. In a declaration, each parameter x_i is bound in $X_{i+1}, ..., X_l$ and in the field types. Terms are identified up to renaming of bound variables.

Pattern variables are places. The variables $\overline{y}_i = y_1, ..., y_{k_i}$ of the arm for C_i are not binders: we identify the arm t_i with $t_i\left[p.g_{i1}/y_1, ..., p.g_{ik_i}/y_{k_i}\right]$, which replaces the root y_j of every place in t_i by the field place $p.g_{ij}$ (for `Nat`, `y` becomes `p.0` and `y.0` becomes `p.0.0`), and write the resolved match $\mathbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1, ..., \mathsf{C}_m \Rightarrow t_m\}$, abbreviated m below. Every definition below (free places, footprints, captures) is on such resolved terms.

Borrow types. `&A` is well formed exactly when `A : Type₀` ([T-Ref]): data, a function type into data or into functions, or a type variable declared in `Type₀`. Propositions, `Prop` and the universes are excluded by the universe check alone, since `Prop : Type₁`; a Π over a proposition, such as `Π(P : Prop). Nat`, is in `Type₁`, as in Coq. The borrow term `&p` applies the same test to the place's type, and a place whose type is stuck is judged by its head's declared universe (note 11). And `&` occurs only syntactically at the top of a declared type, as the whole declared type of a variable, a parameter or a function's result: never inside another type, and never as the value of a type term that is not written `&A` (a codomain that computes to `&Nat` is an error), so that [Close]'s row, read from the declared codomain, always sees a borrow result. There are no borrows inside data and no borrows of borrows. The machine also checks this where values are built ([Ctor], [Borrow], closure capture).

checker: `isDataType`, `Term.refsOk (Basic.lean)`, `Term.refTopOk (Basic.lean)`

Type positions. The following positions of a term are *type positions*: the annotation `A` of `let x : A = t; u`; the parameter types and the codomain of `Π` and of `fix`; the argument of `&`, the arguments of an inductive type $\mathsf{D}(\overline{a})$ or of a constructor at its type parameters, and the first argument of `Eq`, of `Id` and of `J`. What stands in a type position is a type ([Type-pos]) and is erased (Section A.4.1). The two sides of `Eq`, the arguments `a`, `b` and `P` of `J`, and the arguments of an inductive type or constructor at its value parameters are not type positions, but they are *erased positions*: each runs on its own private copy, as an erased term does (Section A.4.1).

A.1.2 Values, neutrals and types

v, w, κ	::=	$\mathsf{C}(\overline{v}; v_1, ..., v_k)$	data (`Z`, `S(v)`, `()`, `(v, w)`, …)
	|	$\star$	the value of every proof
	|	$\mathrm{borrow}_\ell v \mid \mathrm{loan}_\ell \mid \bot$	borrow, loan, moved out
	|	$n \mid F \mid T$	neutrals, function values, types
n	::=	$\sigma \mid \lceil t \rceil$	abstract value, sealed program
F	::=	$f \mid \langle \overline{\kappa} \vdash \mathbf{fix}\ f\left(\overline{x} : \overline{A}\right) : B... := t \rangle$	top-level function, closure
T	::=	$s \mid \mathsf{D}(v_1, ..., v_l) \mid \&T \mid \mathsf{Eq}\ T\ v\ w$	types (`Nat`, `False`, `And(T, T')`, …)
	|	$\langle \overline{\kappa} \vdash \Pi\left(\overline{x} : \overline{A}\right).B \rangle \mid n$	Π-closure, neutral type
s	::=	$\mathsf{Prop} \mid \mathsf{Type}_i$	sorts
Ω	::=	$\varphi_1; ...; \varphi_k$	frames, oldest first
φ	::=	$(x : A \mapsto v)^*\ v^*$	bindings, then temporaries

Figure 12: Values and environments. ℓ ranges over loan labels and σ over abstract values.

A *closure* $\langle \overline{\kappa} \vdash t \rangle$ is code `t` whose free variables are bound, in order, to the captured values $\overline{\kappa}$; a *Π-closure* is the same for a Π-type. They are the paper's "closures" and "Π-types are closures over values". A top-level function `f` is a value by itself; its code is in the signature Σ. A neutral is a type exactly when its type is a sort; for instance the sealed program $\lceil$`Le(σ, σ')`$\rceil$ is a proposition.

A *sealed program* $\lceil$`t`$\rceil$ contains a closed term `t`: it has no free variables, though it may embed values, including abstract values and loans. [Close] produces sealed programs of the form $\lceil$`L; C; K`$\rceil$ with a distinguished *head call* `C`, which we mark $f(\overline{a})^h$.

Loans. loans(v) is the set of labels ℓ such that loan_ℓ occurs in `v`, looking inside sealed programs, closures and types. In an environment Ω, ℓ is *live* if borrow_ℓ occurs in Ω, and *inert* otherwise; inert loans occur only inside the runs of [Seal]. A value is *loan-free* if it contains no live loan, and *borrow-free* if moreover it contains no borrow_ℓ. It is *whole* if it contains no $\bot$. A type is a *copy type* if it is `Unit`, a sort, a proposition, an inductive type declared `copy` (the library's `Word`), or a non-recursive inductive type in $\texttt{Type}_0$ whose field types are copy types and which has no dependent constructor (Section A.4.8) unless it is declared `copy`; a Π-type is not one, so a closure is never copied.

A.1.3 Environments and states

An environment $\Omega = \varphi_1; \ldots; \varphi_k$ is a stack of frames, oldest first. A frame holds a sequence of *bindings* `x : A ↦ v`, where the stored type `A` is omitted for the parameters bound by the untyped machine, followed by a sequence of *temporaries* `v`, the values in flight during evaluation (evaluated arguments, the right-hand side of an assignment, a result while its frame is popped). The *positions* π of Ω are its bindings and temporaries, ordered by frame, then bindings before temporaries, then by index; "in the order of Ω" refers to this order. Variables are resolved in the top frame. We write $\Omega(\pi)$ and $\Omega(x)$ for contents, $\Omega + (x : A \mapsto v)$ for a new binding in the top frame, $\Omega \cdot v$ for a new temporary, and $\Omega; \varphi$ for a pushed frame.

A *state* is an environment together with: Δ, the type of every abstract value; ρ, the refinements made so far, of abstract values and of generalised sealed programs; the goal G while a definition is being checked; and the [Rec] stack. We write Ω for the whole state and mention the other components only where a rule changes them. The types in Δ of the abstract values a private copy introduces, and the generalisations it records, survive the copy, which discards its changes to the environment, the goal and the refinements of abstract values; a generalisation record lives until the [Split] arm that made it ends. Fresh names (abstract values and loan labels) are never reused. The global signature Σ records every inductive declaration, and for each top-level definition its Π-type and its code. A program is checked item by item ([Ind-decl], [Def], [Const]) against the signature of the items before it.

checker: `Env.lean`, `Check.lean`

A.1.4 Auxiliary definitions

1. **Content and update.** $\text{content}_\Omega(x) = \Omega(x)$; $\text{content}_\Omega(*p) = w$ if $\text{content}_\Omega(p) = \text{borrow}_\ell w$; $\text{content}_\Omega(p.g) = v_j$ if $\text{content}_\Omega(p) = \mathsf{C}(\overline{a}; v_1, \ldots, v_k)$ and g is the j-th field of `C`, and $\text{content}_\Omega(p.g) = \star$ if $\text{content}_\Omega(p) = \star$ (a field of a proof is a proof, Section A.4.6); otherwise $\text{content}_\Omega(p)$ is undefined (so reading `p.g` needs a known constructor with field g, as reading `p.0` needs a known `S`, and `*p` needs a borrow). $\Omega[p \mapsto v]$ replaces that sub-value. The *prefixes* of a place are $\text{pre}(x) = \{x\}$, $\text{pre}(*p) = \text{pre}(p) \cup \{*p\}$, $\text{pre}(p.g) = \text{pre}(p) \cup \{p.g\}$; $q \sqsubseteq p$ means $q \in \text{pre}(p)$.

   checker: `content`, `setPlace`

2. **Loans an access must end** ([Access]). For reading and borrowing,

$$L^R_\Omega(p) = \{\ell \mid \text{content}_\Omega(q) = \text{loan}_\ell, q \in \text{pre}(p)\} \cup \text{loans}(\text{content}_\Omega(p)),$$

   for assigning, $L^A_\Omega(p) = \{\ell \mid \text{content}_\Omega(q) = \text{loan}_\ell, q \in \text{pre}(p)\} \cup \text{ol}(\text{content}_\Omega(p))$, where ol is loans except that it does not look inside a borrow, $\text{ol}(\text{borrow}_m w) = \emptyset$; and for matching, $L^M_\Omega(p) = \{\ell \mid \text{content}_\Omega(q) = \text{loan}_\ell, q \in \text{pre}(p)\} \cup \text{hl}(\text{content}_\Omega(p))$, where $\text{hl}(\lceil t \rceil) = \text{loans}(t)$ (a loan's position inside a neutral is unknown, so it is treated as the head) and $\text{hl}(v) = \emptyset$ for every other `v`. Both keep only live labels, ordered from the root of `p` outward and then left to right inside the content.

   checker: `accessPath`, `accessInside`

3. **Owners.** For a live label ℓ,

$$\mathrm{owners}_\Omega(\ell) = \bigcup_{\pi:\, \ell \in \mathrm{loans}(\Omega(\pi))} \begin{cases} \mathrm{owners}_\Omega(m) & \text{if } \Omega(\pi) = \mathrm{borrow}_m w \\ \{\pi\} & \text{otherwise,} \end{cases}$$

a set of positions of Ω (well-defined by acyclicity, Section A.5). The owners of a variable are $\mathrm{own}_\Omega(x) = \mathrm{owners}_\Omega(\ell)$ if $\Omega(x) = \mathrm{borrow}_\ell w$, $\emptyset$ if $\Omega(x) = \bot$ and x has a stored type $\&T$, and $\{x\}$ otherwise (a moved-out owned place is still observed, and its equation is stuck: Section A.3).

checker: `owners (Obs.lean)`

4. **Place occurrences.** $\mathrm{occ}(t)$ is the set of free place occurrences of a resolved term `t`, each with its kind: read (rd), borrowed (bw), assigned (as) or matched (sc). $\mathrm{occ}(p) = \{(p, \mathrm{rd})\}$, $\mathrm{occ}(\&p) = \{(p, \mathrm{bw})\}$, $\mathrm{occ}(p := t) = \{(p, \mathrm{as})\} \cup \mathrm{occ}(t)$, $\mathrm{occ}(\textbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1, ..., \mathsf{C}_m \Rightarrow t_m\}) = \{(p, \mathrm{sc})\} \cup \bigcup_i \mathrm{occ}(t_i)$; a binder removes the places rooted at the variable it binds; every other form takes the union over its subterms, including the types and the code of `Π` and `fix`. A variable `x` is a *borrow variable* in Ω if its stored type is `&T` or $\Omega(x)$ is a borrow.

checker: `Term.placeOccs (Basic.lean)`, `Term.freeOccs (Basic.lean)`

5. **Footprint**.

$$W_\Omega(t, u) = \bigcup \{\mathrm{own}_\Omega(\mathrm{root}(p)) \mid (p, k) \in \mathrm{occ}(t) \cup \mathrm{occ}(u),\ k \in \{\mathrm{bw}, \mathrm{as}\} \text{ or } (k = \mathrm{rd} \text{ and } p \text{ is a borrow variable})\},$$

ordered canonically, by syntax, without repeats: first, for `t` and then `u`, the places the term writes or borrows, in the order evaluation reaches them (left to right, except that `p := t'` reaches the places of `t'` before `p`), each contributing the owners of its root in the order of Ω; then, in the same order, the borrow variables read whole, which hands the borrow to code that may write through it. The order of Ω would not do: closing off a match binds its captures in its own order, so the same `Id` would list its conjuncts differently on the two paths, and `∧` is not commutative by conversion. A place that is only read contributes nothing, even under a borrow: a side that reads `*x` does not observe `x`'s owner.

checker: `footprint (Obs.lean)`

6. **Substitution and normal form.** For an atom $a \in \{\sigma, \mathrm{loan}_\ell\}$, or a neutral `n` (for generalisation), $v[w/a]$ replaces every occurrence of `a` in `v`, including inside sealed programs, closures and types, and then restores normal form: every sealed program that changed is replaced by its normal form ([Seal]), and every `Eq` type is rebuilt by the smart constructor `eq` of Section A.3. $\Omega[w/a]$ does this to every value of Ω; $\Omega[w/a]^+$ also to the stored types, to Δ and to the goal.

checker: `substV`, `substT`, `substEnv`

7. **Refinement and generalisation** ([Split]). A refinement of an abstract value σ of inductive type $\mathsf{D}(\overline{a})$, with `D` declared in `Type₀`, is $\sigma := \mathsf{C}(\overline{a}; \sigma_1, ..., \sigma_k)$ for a constructor $\mathsf{C}(g_1 : A_1, ..., g_k : A_k)$ of `D`, with σ_j fresh and $\Delta(\sigma_j) = A_j[\overline{a}/\overline{x}]$ (proofs are never refined: their value is $\star$); $\Omega[\sigma := r]$ is $\Omega[r/\sigma]^+$, and ϱ records it. *Generalising* a neutral $n = \lceil t \rceil$ (or an inert loan) at a place of type T takes σ fresh with $\Delta(\sigma) = T$, forms $\Omega[\sigma/n]^+$, and records $n := \sigma$ in ϱ, so that it persists: whenever normalisation derives n again it yields $\rho^*(\sigma)$ ([Seal-stuck]), and whenever ϱ grows every sealed program of the state is re-normalised. The *refined value* $\rho^*(v)$ applies ϱ repeatedly, through constructors; the *strict subterms* are $\mathrm{sub}(\mathsf{C}(v_1, ..., v_k)) = \bigcup_j (\{v_j\} \cup \mathrm{sub}(v_j))$ and $\mathrm{sub}(v) = \emptyset$ otherwise.

checker: `refine`, `expandRefs`, `strictSubterms`, `generalizeNeutral`, `canonNeutral`, `renormAll`

8. **Drop** ([Drop]). $\mathrm{drop}(\Omega, v) = \Omega'$ where $\Omega \cdot v \overset{\ell}{\rightsquigarrow} \Omega' \cdot \bot$ ([End]) if $v = \mathrm{borrow}_\ell w$; $\mathrm{drop}(\Omega, v) = \Omega$ if `v` is loan-free. Otherwise a live loan in `v` ends its borrower ([End]) when the borrower is held in a binding (a variable, or a place inside one), as a write through [Access] would, repeating for the loans the ended borrow's content brings in; when the borrower is a value in flight (a temporary, such as the borrow a block or a function is returning), $\mathrm{drop}(\Omega, v) = \mathsf{err}$. So a place may die while a borrow of it is never used again, but nothing may return a borrow of its own local. Popping a frame drops its bindings, newest first, with the call's result held as a temporary of the caller's frame: $\mathrm{pop}(\Omega;\ \varphi, v)$.

checker: `dropTopBind`, `dropValue`, `popFrame`

9. **Types of places and values.** $\text{type}_\Omega(x)$ is the stored type of x, or $\text{typeof}(\Omega(x))$ if it has none; $\text{type}_\Omega(*p) = T$ if $\text{type}_\Omega(p) = \&T$; $\text{type}_\Omega(p.g) = A_j[\overline{a}/\overline{x}]$ if $\text{type}_\Omega(p) = \mathsf{D}(\overline{a})$ with $g_j : A_j$ the field g of the constructor C of D that the place names (a field place $p.g_{ij}$ records its constructor C_i, Section A.1), and $\text{content}_\Omega(p)$ is $\mathsf{C}(...)$ or $\star$ (so a proof's fields have types in every arm of a match on it). For values: $\mathsf{C}(\overline{a};\overline{v})$ has type $\mathsf{D}(\overline{a})$ for C a constructor of D; σ has $\Delta(\sigma)$; f has its type in Σ; a closure $\langle\overline{\kappa} \vdash \textbf{fix } f\big(\overline{x} : \overline{A}\big) : B ...\rangle$ has the Π-closure $\langle\overline{\kappa} \vdash \Pi\big(\overline{x} : \overline{A}\big).B\rangle$; $\text{borrow}_\ell w$ has $\&\text{typeof}(w)$; a type has its sort (Section A.4).

checker: `placeType`, `valType`

10. **The generic call**. For a Π-closure $\Pi = \langle\overline{\kappa} \vdash \Pi(x_1 : A_1 ... x_n : A_n).B\rangle$, the *generic arguments* $\overline{a}$ are built left to right. In the frame $\overline{\kappa}, x_1 : T_1 \mapsto a_1, ..., x_{i-1} : T_{i-1} \mapsto a_{i-1}$, evaluate A_i in a type position to T_i (Section A.4). If $T_i = \&T$, add an owned place $c_i : T \mapsto \text{loan}_{\ell_i}$ to a bottom frame and let $a_i = \text{borrow}_{\ell_i}\sigma_i$ with $\sigma_i : T$ and ℓ_i fresh (this is the argument $\&c_i$ after [Borrow]); if T_i is a proposition (its declared sort, which is its computed sort by [Type-pos]), let $a_i = \star$; otherwise $a_i = \sigma_i$ with $\sigma_i : T_i$ fresh. The *generic state* is the frame of owned places $\Gamma(\Pi) = \big(c_i : T \mapsto \text{loan}_{\ell_i}\big)_{i:T_i=\&T}$, and the *parameter frame* is $\varphi(\Pi) = \big(\overline{\kappa}, \overline{x} : \overline{T} \mapsto \overline{a}\big)$, each type stored with its binding. The generic call is $F(\overline{a})$ at $\Gamma(\Pi)$.

checker: `checkFix`

A.2 The machine

The machine is the big-step judgement $\langle\Omega, t\rangle \Downarrow r$, where the outcome r is $\langle\Omega', v\rangle$, stuck (the run needs to inspect a neutral) or err (a borrow error). It is deterministic. **Propagation:** when a premise that evaluates a subterm has outcome stuck or err, so does the conclusion; the only rules that inspect a stuck premise are [App-close], [App-head] and [Seal]. We write $\langle\Omega, t\rangle \Downarrow \langle\Omega', v\rangle$ only for successful runs.

A.2.1 Borrows

Ending a borrow replaces it by $\bot$ and substitutes its content for its loan everywhere, in normal form (Section A.1.4, item 6). A borrow occurs only as the whole value of a binding or temporary (Section A.5), so it has a position π.

$$\frac{\Omega(\pi) = \text{borrow}_\ell w}{\Omega \underset{\ell}{\rightsquigarrow} (\Omega[\pi \mapsto \bot])[w/\text{loan}_\ell]} \textsc{End}$$

[**Access**]. $\text{acc}_p^R(\Omega)$, $\text{acc}_p^A(\Omega)$ and $\text{acc}_p^M(\Omega)$ end, one at a time and in the order of item 2 of Section A.1.4, the borrow of the first label of $L_\Omega^R(p)$, $L_\Omega^A(p)$ or $L_\Omega^M(p)$, until that set is empty:

$$\text{acc}_p^X(\Omega) = \begin{cases} \Omega & \text{if } L_\Omega^X(p) = \emptyset \\ \text{acc}_p^X(\Omega') & \text{if } \ell \text{ is the first label of } L_\Omega^X(p) \text{ and } \Omega \underset{\ell}{\rightsquigarrow} \Omega' \end{cases} \quad (X \in \{R, A, M\}).$$

Every rule that reads or borrows a place p first computes acc_p^R, an assignment to p computes acc_p^A, and a match on p computes acc_p^M. Before assigning p, [Access] ends every borrow whose loan lies in the part of `content(p)` that p owns: on the path to p, in its owned content, or inside a neutral there, since a loan's position inside a neutral is unknown. A loan behind a borrow held in `content(p)`, including one inside a neutral behind that borrow, is not ended: the [Drop] of the old content ends that borrow ([End]), which carries the loan back to its owner, where it stays live. This is Rust's rule that a reborrow `&mut (*x).f` lasts as long as the reference x held, not the variable x. Reads, moves and borrows of p still end every loan inside `content(p)`. This is the borrow checker: an ended borrower holds $\bot$, and every later use of it is an error.

checker: `endBorrow`, `accessPath`, `accessInside`, `accessNeutralHead`, `Value.ownedLoans (Basic.lean)`

$$\frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = v \notin \{\bot, \mathrm{borrow}_\ell w\},\ v \text{ whole} \quad \mathrm{type}_{\Omega_1}(p) \text{ a copy type}}{\langle \Omega, p\rangle \Downarrow \langle \Omega_1, v\rangle}\ \textsc{Copy} \qquad \frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = v \notin \{\bot, \mathrm{borrow}_\ell w\},\ v \text{ whole} \quad \mathrm{type}_{\Omega_1}(p) \text{ not a copy type}}{\langle \Omega, p\rangle \Downarrow \langle \Omega_1[p \mapsto \bot], v\rangle}\ \textsc{Read}$$

$$\frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = \mathrm{borrow}_\ell w}{\langle \Omega, p\rangle \Downarrow \langle \Omega_1[p \mapsto \bot], \mathrm{borrow}_\ell w\rangle}\ \textsc{Move}$$

$$\frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) \text{ undefined, not whole, or } \bot}{\langle \Omega, p\rangle \Downarrow \mathsf{err}}\ \textsc{Read-err}$$

$$\frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \ell \text{ fresh} \quad \mathrm{content}_{\Omega_1}(p) = v \notin \{\bot, \mathrm{borrow}_m w\},\ v \text{ whole}}{\langle \Omega, \&p\rangle \Downarrow \langle \Omega_1[p \mapsto \mathrm{loan}_\ell], \mathrm{borrow}_\ell v\rangle}\ \textsc{Borrow}$$

$$\frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) \text{ undefined, } \bot \text{ , a borrow, or not whole}}{\langle \Omega, \&p\rangle \Downarrow \mathsf{err}}\ \textsc{Borrow-err}$$

$$\frac{\mathrm{acc}_p^R(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = v \notin \{\bot, \mathrm{borrow}_m w\}}{\langle \Omega, \mathbf{clone}(p)\rangle \Downarrow \langle \Omega_1, v\rangle}\ \textsc{Clone} \qquad \frac{\langle \Omega, t\rangle \Downarrow \langle \Omega_1, v\rangle \quad \mathrm{acc}_p^A(\Omega_1 \cdot v) = \Omega_2 \cdot v' \quad \mathrm{content}_{\Omega_2}(p) = w \quad \mathrm{drop}(\Omega_2, w) = \Omega_3}{\langle \Omega, p := t\rangle \Downarrow \langle \Omega_3[p \mapsto v'], ()\rangle}\ \textsc{Assign}$$

An erased term reads as code does: only [Copy] and [Clone] copy, and a borrow cannot be cloned. [End] requires the content `w` to be whole, and so do a returned borrow and a borrow argument of a call that closes off: moving data out through a borrow is allowed only if the place is filled again before the borrow ends, as with `mem::replace` in Rust. In [Assign] the new value travels as a temporary while `p` is accessed, so that ending a borrow can substitute into it; $\mathrm{drop}(\Omega_2, w)$ ends the old content if it is a borrow, and fails if it still holds a live loan.

checker: `readPlace`, `borrowPlace`, `assignPlace`, `evalCore (.prim "clone")`

A.2.2 Sequencing and data

$$\frac{\langle \Omega, t\rangle \Downarrow \langle \Omega_1, v\rangle \quad \langle \Omega_1 + (x \mapsto v), u\rangle \Downarrow \langle \Omega_2 + (x \mapsto v'), w\rangle \quad \mathrm{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\langle \Omega, \mathbf{let}\ x = t;\ u\rangle \Downarrow \langle \Omega_3, w'\rangle}\ \textsc{Let}$$

$$\frac{\langle \Omega, t\rangle \Downarrow \langle \Omega_1, v\rangle \quad \mathrm{drop}(\Omega_1, v) = \Omega_2 \quad \langle \Omega_2, u\rangle \Downarrow r}{\langle \Omega, t;\ u\rangle \Downarrow r}\ \textsc{Seq}$$

The annotated `let x : A = t; u` runs as `let x = t; u`: its annotation is a type position. A discarded value (`t; u`) is dropped like a binding: a borrow ends, and a live loan is an error.

checker: `evalCore (.letIn, .seq)`, `dropTopBind`, `dropValue`

$$\frac{\langle \Omega, \overline{t}\rangle \Downarrow^* \langle \Omega', \overline{v}\rangle}{\langle \Omega, \mathsf{C}(\overline{t})\rangle \Downarrow \langle \Omega', \mathsf{C}(\overline{v})\rangle}\ \textsc{Ctor}$$

A constructor evaluates its fields left to right, each into a temporary ([Args], below); its field types are borrow-free, so a constructor value holds no borrow. A pair is the constructor `Mk`.

checker: `evalCore (.ctor, .zero, .succ, .tt, .fst, .snd)`

A.2.3 Functions, types and proofs

Capture ([Capture]). A closure or Π-type captures, when it is formed, the current contents of its free variables $y_1, ..., y_m$ (in the order of Ω), each accessed as a read: $\Omega_0 = \Omega$, $\Omega_i = \mathrm{acc}^R_{y_i}(\Omega_{i-1})$, $\kappa_i = \mathrm{content}_{\Omega_i}(y_i)$, each recorded with the declared type and proof flag of y_i's binding (note 27). A captured value may be neither $\bot$ nor a borrow (closures capture no borrows); otherwise the rule fails. At runtime, capturing a variable whose type is not a copy type moves it, as [Read] does; in an erased term it copies. A closure's body may not move a captured value out, since the closure may run again (Rust's `Fn`): a runtime read of a captured value that is not a copy is an error, and `clone` copies it. A call reads its head in place and does not consume it. The other type formers compute type values; `Eq` uses the smart constructor of Section A.3.

$$\frac{f \in \Sigma}{\langle \Omega, f\rangle \Downarrow \langle \Omega, f\rangle}\ \textsc{Global} \qquad \frac{\overline{\kappa}\ \text{captured}, \Omega_m}{\langle \Omega, \mathbf{fix}\ f\big(\overline{x} : \overline{A}\big) : B... := t\rangle \Downarrow \langle \Omega_m, \langle \overline{\kappa} \vdash \mathbf{fix}\ f\big(\overline{x} : \overline{A}\big) : B... := t\rangle\rangle}\ \textsc{Fix}$$

$$\frac{\overline{\kappa}\ \text{captured}, \Omega_m}{\langle \Omega, \Pi\big(\overline{x} : \overline{A}\big).B\rangle \Downarrow \langle \Omega_m, \langle \overline{\kappa} \vdash \Pi\big(\overline{x} : \overline{A}\big).B\rangle\rangle}\ \textsc{Pi} \qquad \frac{}{\langle \Omega, s\rangle \Downarrow \langle \Omega, s\rangle}\ \textsc{Sort} \qquad \frac{\langle \Omega, \overline{a}\rangle \Downarrow^* \langle \Omega', \overline{v}\rangle}{\langle \Omega, \mathsf{D}(\overline{a})\rangle \Downarrow \langle \Omega', \mathsf{D}(\overline{v})\rangle}\ \textsc{Ind}$$

$$\frac{\langle \Omega, A\rangle \Downarrow \langle \Omega', T\rangle}{\langle \Omega, \&A\rangle \Downarrow \langle \Omega', \&T\rangle}\ \textsc{Ref} \qquad \frac{\langle \Omega, A\rangle \Downarrow \langle \Omega_1, T\rangle \quad \langle \Omega_1, a\rangle \Downarrow \langle \Omega_2, v\rangle \quad \langle \Omega_2, b\rangle \Downarrow \langle \Omega_3, w\rangle}{\langle \Omega, \mathsf{Eq}\ A\, a\, b\rangle \Downarrow \langle \Omega_3, \mathrm{eq}(T, v, w)\rangle}\ \textsc{Eq}$$

$$\frac{a \text{ and } b \text{ evaluate to convertible values} \quad \langle \Omega, t\rangle \Downarrow r}{\langle \Omega, \mathsf{J}(A, a, b, P, h, t)\rangle \Downarrow r}\ \textsc{J}$$

$$\frac{a \text{ and } b \text{ evaluate to values that are not convertible}}{\langle \Omega, \mathsf{J}(A, a, b, P, h, t)\rangle \Downarrow \mathsf{stuck}}\ \textsc{J-stuck}$$

In [Ind] the parameters are evaluated like arguments ([Args], below); a type $\mathsf{And}(T, T')$ is kept as written, and the unit laws apply to it only in conversion ([Conv-unit]). `Id(A, t, u)` is a type former too; its value is given in Section A.3. A constructor of an inductive declared in `Prop` (such as `refl` or `⟨h, k⟩`) is a declared proof, so the machine never evaluates it ([Erase-proof]). `J` returns the value of `t` when its endpoints are convertible, as Lean's `Eq.rec` does, and is stuck otherwise, so a cast under a hypothesis that equates unequal values never produces a value of the wrong type. `a` and `b` are evaluated in erased positions, on private copies; `A`, `P` and the proof `h` are never run by the machine. A top-level constant `c : A := t` evaluates to the value stored in Σ, which is $\star$ when `A` has declared sort `Prop`.

checker: `capture`, `evalCore (.pi, .fix, .sort, .ref, .eq, .tind, .prim "J", .const)`, `evalTInd`, `ctorIsProof`

Erased terms. Whether an occurrence is erased is read from syntax and declarations, never from a normal form or a computed type (Section A.4.1); the machine needs no types for it, only one flag per binding, saying whether the binding was declared a proof, and at a call the class recorded in the callee's declared Π-type, which conversion preserves ([Conv-pi]). An erased term runs on a private copy of the environment: its effects are discarded and only its value is kept, which for a proof is $\star$, so a proof need not be run at all.

$$\frac{t \text{ erased, a declared proof}}{\langle \Omega, t\rangle \Downarrow \langle \Omega, \star\rangle}\ \textsc{Erase-proof} \qquad \frac{t \text{ erased, a type} \quad \langle \Omega, t\rangle \Downarrow_0 \langle \Omega', T\rangle}{\langle \Omega, t\rangle \Downarrow \langle \Omega, T\rangle}\ \textsc{Erase-type}$$

Here $\Downarrow_0$ is the judgement in which the rule for `t`'s own form is applied at the root instead of [Erase-type]. Every erased occurrence, nested ones included, runs on its own copy, so whatever it assigns, borrows or moves changes only that copy.

checker: `eval`, `callFn`, `fnClass`, `paramFlags`, `preFlags`, `declOf`, `stmtCopy`, `onCopy (Env.lean)`

A.2.4 Calls

Arguments are evaluated left to right, each into a temporary of the caller's top frame, so that [Access] can see, and end, the earlier ones:

$$\frac{}{\langle \Omega, \varepsilon \rangle \Downarrow^* \langle \Omega, \varepsilon \rangle}\ \textsc{Args-nil} \qquad \frac{\langle \Omega, u \rangle \Downarrow \langle \Omega_1, w \rangle \qquad \langle \Omega_1 \cdot w, \overline{u} \rangle \Downarrow^* \langle \Omega_2 \cdot w', \overline{w} \rangle}{\langle \Omega, u\,\overline{u} \rangle \Downarrow^* \langle \Omega_2, w'\,\overline{w} \rangle}\ \textsc{Args}$$

$$\frac{\langle \Omega, t_0 \rangle \Downarrow \langle \Omega_0, F \rangle \quad \langle \Omega_0 \cdot F, \overline{u} \rangle \Downarrow^* \langle \Omega_1 \cdot F', \overline{w} \rangle \quad \text{no } w_i \text{ is } \bot \quad \langle \Omega_1, F'(\overline{w}) \rangle \Downarrow_{\text{app}} r}{\langle \Omega, t_0(\overline{u}) \rangle \Downarrow r}\ \textsc{Call} \qquad \frac{\langle \Omega_0 \cdot F, \overline{u} \rangle \Downarrow^* \langle \Omega_1 \cdot F', \overline{w} \rangle \qquad \text{some } w_i = \bot}{\langle \Omega, t_0(\overline{u}) \rangle \Downarrow \mathsf{err}}\ \textsc{Call-err}$$

An argument is $\bot$ when a later argument ended its borrow, as in `f(&x, &x)`: it is not of its parameter's type. The judgement $\langle \Omega, F(\overline{w}) \rangle \Downarrow_{\text{app}} r$ applies a function value at its *call point*, the state after the arguments have been evaluated. Let `F` have code $\mathbf{fix}\ f\big(\overline{x} : \overline{A}\big) : B\ \big[\mathbf{by}\ x_j\big] := b$ and captured values $\overline{\kappa}$ (none for a top-level `f`), and let φ be the frame $\overline{\kappa}, [f \mapsto F], \overline{x} \mapsto \overline{w}$, where `f` is bound only with `by`.

$$\frac{\langle \Omega;\, \varphi, b \rangle \Downarrow \langle \Omega';\, \varphi', v \rangle \qquad \text{pop}(\Omega';\, \varphi', v) = \langle \Omega'', v' \rangle}{\langle \Omega, F(\overline{w}) \rangle \Downarrow_{\text{app}} \langle \Omega'', v' \rangle}\ \textsc{App}$$

$$\frac{\langle \Omega;\, \varphi, b \rangle \Downarrow \mathsf{stuck} \qquad \text{the call is not a head call}}{\langle \Omega, F(\overline{w}) \rangle \Downarrow_{\text{app}} \text{close}(\Omega, F, \overline{w})}\ \textsc{App-close}$$

$$\frac{\langle \Omega;\, \varphi, b \rangle \Downarrow \mathsf{stuck} \qquad \text{the call is a head call } F(\overline{w})^h}{\langle \Omega, F(\overline{w})^h \rangle \Downarrow_{\text{app}} \mathsf{stuck}}\ \textsc{App-head}$$

$$\frac{F = n \text{ a neutral} \qquad \text{the call is not a head call}}{\langle \Omega, n(\overline{w}) \rangle \Downarrow_{\text{app}} \text{close}(\Omega, n, \overline{w})}\ \textsc{App-neutral} \qquad \frac{F = n \text{ a neutral}}{\langle \Omega, n(\overline{w})^h \rangle \Downarrow_{\text{app}} \mathsf{stuck}}\ \textsc{App-neutral-head}$$

[App-close] discards the partial run of the body: close starts again from the call point Ω. The head guard of [Seal] covers neutral-headed calls too: a sealed program whose head call has a neutral head stays as it is.

checker: `evalCall`, `callFn`, `runBody`, `popFrame`

A.2.5 Closing off

close$(\Omega, F, \overline{w})$ closes off the call $F(\overline{w})$ at its call point Ω. Its precondition is that every argument is loan-free or is borrow$_\ell u$ with `u` loan-free; [Access] guarantees it. Let $I = \big\{ i \;\big|\; w_i = \text{borrow}_{\ell_i} u_i \big\}$, let c_i $(i \in I)$ be fixed names, and let

$$L := (\mathbf{let}\ c_i = u_i;)_{i \in I} \qquad C := F(a_1, ..., a_n)^h \quad \text{with } a_i = \begin{cases} \&c_i \text{ if } i \in I \\ w_i \quad \text{otherwise.} \end{cases}$$

The row is chosen by the *declared* codomain `B` of `F`'s type: the codomain term of `f`'s definition, of the closure's code, of $\Delta(\sigma)$ for an abstract function, or the type of a stuck block.

declared codomain `B`	result `r`	each loan$_{\ell_i}$, $i \in I$, becomes f_i
`&T` (`k` fresh)	borrow$_k$⌈L; let r = C; *r⌉	⌈L; let r = C; *r := loan$_k$; c_i⌉
any other	⌈L; C⌉	⌈L; C; c_i⌉

Figure 13: [Close]: close$(\Omega, F, \overline{w}) = \langle \Omega\big[f_i/\text{loan}_{\ell_i}\big]_{i \in I}, r \rangle$. The borrows borrow$_{\ell_i}$ are consumed: at the call point they are no longer in Ω.

checker: `closeCall`, `declKind`, `resultKind`, `kindOf`

A.2.6 Matching

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \qquad \mathrm{content}_{\Omega_1}(p) = \mathsf{C}_i(\overline{v}) \qquad \langle \Omega_1, t_i \rangle \Downarrow r}{\langle \Omega, \mathbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1, ..., \mathsf{C}_m \Rightarrow t_m\} \rangle \Downarrow r}\ \textsc{Match}$$

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \qquad \mathrm{content}_{\Omega_1}(p) \text{ is a neutral or an inert loan}}{\langle \Omega, \mathbf{match}\ p\{...\} \rangle \Downarrow \mathsf{stuck}}\ \textsc{Match-stuck}$$

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \qquad \mathrm{content}_{\Omega_1}(p) \text{ is undefined or } \bot}{\langle \Omega, \mathbf{match}\ p\{...\} \rangle \Downarrow \mathsf{err}}\ \textsc{Match-err}$$

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \qquad \mathsf{C}_1 \text{ is the only constructor of a } \mathsf{D} \text{ declared in } \mathsf{Prop} \qquad \langle \Omega_1, t_1 \rangle \Downarrow r}{\langle \Omega, \mathbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1\} \rangle \Downarrow r}\ \textsc{Match-prop}$$

$$\frac{\mathbf{match}\ p\{\} \text{ not in a proof position}}{\langle \Omega, \mathbf{match}\ p\{\} \rangle \Downarrow \mathsf{stuck}}\ \textsc{Match-none}$$

In arm i the pattern variables are the field places $p.g_{ij}$, so `&y` reborrows a field in place (for `Nat`, the predecessor `p.0`). A match whose arms name the constructors of an inductive declared in `Prop` is a *match on a proof*. The machine recognises it from those names, which is a fact about declarations, and never inspects the content of `p`, which is $\star$: with one constructor it takes the only arm ([Match-prop]), in which the fields of `p` hold $\star$ (Section A.1.4, item 1); with several, the match is a declared proof (Section A.4.1) and is never run. A match with no arms in a proof position is erased like any proof; anywhere else, reaching it is stuck ([Match-none]), since it is reached only under a hypothesis that a refinement has made false. [Match], [Match-stuck] and [Match-err] apply to the other matches. A match on a place whose content was moved out, which holds $\bot$, is an error.

checker: `evalMatch`, `evalMatchInd`, `accessNeutralHead`, `byTypeMatch`, `evalMatchByType`

A.2.7 Sealed programs

Values are kept in normal form. The normal form of a sealed program is computed by running it from the empty environment `ε` (one empty frame), where its head call may not close off ([App-head]):

$$\frac{\langle \varepsilon, t \rangle \Downarrow \langle \Omega', v \rangle}{\mathrm{nf}(\lceil t \rceil) = v}\ \textsc{Seal} \qquad \frac{\langle \varepsilon, t \rangle \Downarrow \mathsf{stuck}}{\mathrm{nf}(\lceil t \rceil) = \begin{cases} \rho^\star(\sigma) \text{ if } \rho \text{ records } \lceil t \rceil := \sigma \\ \lceil t \rceil \quad \text{otherwise} \end{cases}}\ \textsc{Seal-stuck} \qquad \frac{\langle \varepsilon, t \rangle \Downarrow \mathsf{err}}{\mathrm{nf}(\lceil t \rceil) = \mathsf{err}}\ \textsc{Seal-err}$$

The run has runtime semantics, except that the final read `K` of a sealed program $\lceil$`L; C; K`$\rceil$ copies, since it is an observation. Every loan in `t` whose borrow lives outside the run is inert in it: [Access] does not end it (it is not live), [Read] copies it (it is loan-free), drop accepts it, and a match on it is stuck, as on an abstract value. An error is a type error at the point that triggered the normalisation. Normalisation happens whenever a sealed program is created or changed: by [Close], by [End] (substituting a loan), and by refinement or generalisation (Section A.1.4, items 6 and 7); a generalised sealed program stays generalised wherever it is derived again. The embedded values of a stuck sealed program are already normal, because substitution normalises inner sealed programs first.

checker: `nfSealed`, `substV`, `canonNeutral`

A.2.8 Stuck blocks

Block ([Block]). A stuck match that is not the body of a call is closed off as a call of an anonymous function of its free places, captured as Rust captures closure variables. This happens only in the typing judgement ([Split], Section A.4), after the match's arms have been checked, which gives its type `B` and the set `M` of variables that some arm leaves $\bot$ (moves out of).

Let $m = \mathbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1, ..., \mathsf{C}_m \Rightarrow t_m\}$ be stuck in Ω, and give each occurrence $(q, k) \in \mathrm{occ}(m)$ a *mode*: `mv` (moved) if `q` is a whole variable `x` and either `x` is a borrow variable and $k \in \{\mathrm{rd}, \mathrm{as}\}$, or $x \in M$, or a function

or Π-type in an arm mentions `x` and its type is not a copy type (a closure captures by move, on every path); otherwise `ref` (borrowed) if $k \in \{\mathrm{bw}, \mathrm{as}\}$; otherwise `cp` (copied). The *captures* $q_1, ..., q_n$ are the maximal places of occ(m), those with no strict prefix among its places. Each takes the largest mode of the occurrences at or below it (cp < ref < mv), and they are ordered by the position of their root, then by length. Capture q_i becomes a parameter z_i with argument a_i:

$$(z_i : U_i, a_i) = \begin{cases} (z_i : \&\mathrm{type}_\Omega(q_i),\ \&q_i) & \text{if } q_i \text{ is ref,} \\ (z_i : \mathrm{type}_\Omega(q_i),\ q_i) & \text{otherwise (a move if its content is a borrow, a copy if not).} \end{cases}$$

The body m' is `m` with each place $q_i\pi$ renamed to $(*z_i)\pi$ if q_i is `ref`, and to $z_i\pi$ otherwise. Then

$$\mathrm{block}(\Omega, m, B, M) \coloneqq F_m(a_1, ..., a_n) \quad \text{where } F_m = \langle \vdash \mathbf{fix}\ _(z_1 : U_1 ... z_n : U_n) : B \coloneqq m' \rangle,$$

a non-recursive closure with no captured values. Evaluating the block by [Call] gets stuck on the head of `m` inside F_m's body and closes off by [App-close], with [Close]'s row chosen by `B`. A function or Π-type in an arm of a stuck match that mentions a place the block takes by `&` (because an arm writes it) is a type error where the block is formed, since the block's re-run would capture that borrow parameter.

checker: `closeOffMatch`, `splitThenClose`, `Term.fnOccs (Basic.lean)`

A.3 Observation, `Id` and conversion

`Eq` computes. Every `Eq` type is built by the smart constructor eq, which takes apart two values built by the same constructor and conjoins the equations between their fields with and, which drops `True` conjuncts:

$$\mathrm{eq}(\mathsf{D}(\overline{a}), \mathsf{C}(\overline{a}; v_1, ..., v_k), \mathsf{C}(\overline{a}; w_1, ..., w_k)) = \mathrm{and}(\mathrm{eq}(B_1, v_1, w_1), ..., \mathrm{eq}(B_k, v_k, w_k)) \quad \text{fields } B_j \text{ at } \overline{a}; \mathsf{True} \text{ if } k = 0 \quad \text{[Eq-inj]}$$

$$\mathrm{eq}(T, v, w) = \mathsf{True} \quad \text{if } v \equiv w \text{ (and the first clause does not apply)} \quad \text{[Eq-refl]}$$

$$\mathrm{eq}(T, \mathsf{C}(\overline{v}), \mathsf{C}'(\overline{w})) = \mathsf{False} \quad \text{if } \mathsf{C} \neq \mathsf{C}' \text{ are constructors of the same inductive} \quad \text{[Eq-disj]}$$

$$\mathrm{eq}(T, v, w) = \mathsf{Eq}\ T\ v\ w \quad \text{if } \bot \text{ occurs in } v \text{ or } w, \text{ or otherwise} \quad \text{[Eq-stuck]}$$

$$\mathrm{and}(\mathsf{True}, P) = P \qquad \mathrm{and}(P, \mathsf{True}) = P \qquad \mathrm{and}(P, Q) = \mathsf{And}(P, Q) \quad \text{otherwise}$$

The first case of [Eq-stuck] is checked before the others: a stuck block takes a place by move if any arm moves it, so `⊥` at the generic call need not mean moved at an instance. These realise the conversion rules of Figure 7 for `Eq`, and they are why `Id`'s observation computes to the single interesting equation. A type `And(P, Q)` written in a program is not rebuilt by and: it keeps its head, so a proof of `⊤ ∧ P` can be taken apart by a match, and the unit laws reach it only through conversion ([Conv-unit]). The first and third clauses apply only when both sides are constructor values (injectivity and disjointness, note 26 of Section A.7); a neutral is never split. `refl`, the constructor of `True`, has type `True`, so by conversion it proves every reflexive equation; an equation between distinct constructors is `False`, eliminated by `match h {}`.

checker: `mkEqM`, `mkAnd (Basic.lean)`, `distinctCtors (Basic.lean)`

Observation. Let $W = \pi_1, ..., \pi_k$ be positions of Ω, in the order of the footprint. The observation of `t` at Ω on `W` runs `t` on a private copy of Ω, ends every remaining borrow, and reads off the result and the footprint. A borrow-typed result (only [Conv-fun] observes one, since `Id` needs a borrow-free type) is observed as its current content, after a given abstract value σ_w is written through it, so that its owners show where it points:

$$\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad v \text{ is not a borrow} \qquad \Omega_1 \cdot v \overset{*}{\rightsquigarrow} \Omega_2 \cdot v' \text{ with no borrow left in } \Omega_2 \cdot v'}{[\![t]\!]^W_\Omega = \mathrm{tuple}(v', \Omega_2(\pi_1), ..., \Omega_2(\pi_k))} \ \textsc{Obs}$$

$$\frac{\Omega \vdash t \Downarrow \mathrm{borrow}_k u : \&T \dashv \Omega_1 \qquad \Omega_1 \cdot u \cdot \mathrm{borrow}_k \sigma_w \overset{*}{\rightsquigarrow} \Omega_2 \cdot u' \cdot \bot \text{ with no borrow left}}{[\![t]\!]^{W;\sigma_w}_\Omega = \mathrm{tuple}(u', \Omega_2(\pi_1), ..., \Omega_2(\pi_k))} \ \textsc{Obs-borrow}$$

where $\overset{*}{\rightsquigarrow}$ repeatedly ends the first borrow in the order of Ω (so in [Obs-borrow] σ_w reaches the owners of loan_k, as `*r := σ_w` would), $\mathrm{tuple}(v, w_1, ..., w_k)$ is a tuple of the machine, not a pair value. The run of `t` pops every frame

it pushes, so the positions of `W` are still positions of Ω_2. A stuck match in `t` is closed off by [Split] (Section A.4), so an observation exists unless `t` has an error. This holds wherever `Id` is evaluated, the machine included (a callee's body, a sealed program's re-run). There the side's run is the machine's when it is not stuck, which is what typing computes; when it is stuck, the side is observed by the typing judgement.

checker: `observe`, `endAll`

`Id` computes. Both sides run from the same Ω, on independent copies:

$$\frac{\Omega \vdash A \Downarrow T \text{ type}, T \text{ contains no } \&\quad W = W_\Omega(t,u) = \pi_1, ..., \pi_k \quad [\![t]\!]^W_\Omega = \text{tuple}(r, w_1, ..., w_k) \text{ with } t : A_t \equiv T \quad [\![u]\!]^W_\Omega = \text{tuple}(r', w'_1, ..., w'_k) \text{ with } u : A_u \equiv T \quad T_i = \text{type}_\Omega(\pi_i)}{\Omega \vdash \mathsf{Id}\ A\, t\, u \Downarrow \text{and}(\text{eq}(T, r, r'), \text{eq}(T_1, w_1, w'_1), ..., \text{eq}(T_k, w_k, w'_k)) : \mathsf{Prop} \dashv \Omega}\ \textsc{Id}$$

When `W` is empty, `Id` is the one equation between the results. `A` may be any borrow-free type, a proposition or a universe included, since no pair value is built.

checker: `idType`, `footprint (Obs.lean)`

Conversion. Definitional equality is equality of normal forms, and values are always kept in normal form, so conversion $v \equiv w$ is the least relation closed under the following rules. It is the relation of every "$\equiv$" premise in Section A.4 and of the reflexive clause of eq.

- [Conv-refl] $v \equiv v$, where values are identified up to renaming of bound variables (binder names are ignored).
- [Conv-cong] Two values with the same outermost former (a constructor, a borrow, an inductive type, `&`, `Eq`, a sealed program, a closure) are convertible if their immediate components are pairwise convertible: terms (the code of closures, the programs inside sealed programs) component by component, with embedded values compared by $\equiv$. A constructor value is compared by its constructor and fields; the parameters it records are determined by its type and are ignored.
- [Conv-unit] $\mathsf{And}(\mathsf{True}, P) \equiv P \equiv \mathsf{And}(P, \mathsf{True})$: two types are convertible if they are after the unit laws are applied to each, through nested `And`s. This is the only way the unit laws act on a type written in a program; stored types keep their `And`.
- [Conv-pi] Two Π-closures with the same number of parameters are convertible if they have the same class (Section A.4.1) and both or neither return a borrow, and if, at the generic state Γ and generic arguments $\overline{a}$ of the first (Section A.1.4, item 10), their parameter types, each evaluated with the earlier parameters bound to $\overline{a}$, are pairwise convertible, and so are their codomains. So Π-types are compared under binders by instantiating the binders, as function values are ([Conv-fun]), and captured values matter only through the types they help compute: $\langle m \mapsto \sigma \vdash \Pi(n : \mathsf{Nat}).\ \mathsf{Nat}\rangle \equiv \langle \vdash \Pi(n : \mathsf{Nat}).\ \mathsf{Nat}\rangle$. Two Π-closures with convertible captured values and the same code are convertible by [Conv-cong]; the checker tries that first, as a fast path.
- [Conv-fun] Two function values `F`, `G` are convertible if their Π-types are, their captured values are pairwise convertible (none, for a top-level function), and at the generic call of their Π-type (Section A.1.4, item 10), with the same generic state Γ, arguments $\overline{a}$ and owned places $\overline{c}$, their observations agree: $[\![F(\overline{a})]\!]^{\overline{c}}_\Gamma \equiv [\![G(\overline{a})]\!]^{\overline{c}}_\Gamma$. For a codomain `&T` both observations use [Obs-borrow] with one shared fresh $\sigma_w : T$, so `λ(x y : &Nat). x` and `λ(x y : &Nat). y` differ: one leaves (σ_w, σ_y) in the owned places, the other (σ_x, σ_w). So a function value's normal form is its captured values together with the observation of its generic call, and two functions with different effects are never convertible.

There is no η rule. Values at `Unit` are η-normal instead: a value is replaced by `()` wherever its type is known to be `Unit`, that is, a parameter or abstract value whose declared type, or evaluated declared type, is `Unit`, and the result of a call whose declared codomain, or computed result type, is `Unit`. `eq` therefore decides on values alone and never reads a type; a path on which a type is still neutral keeps the value unnormalised and is only more stuck. Proof irrelevance needs no rule: every proof value is $\star$. The relation is *least*: a function value that occurs in its own generic observation, as the head of a sealed program when its body is stuck at its generic call, is compared there by [Conv-refl] and [Conv-cong]. Read coinductively, [Conv-fun] would identify any two closures whose bodies are stuck at the generic call, such as `λ(x:&Nat). match *x { Z => (), S _ => *x := Z }` and `λ(x:&Nat). match *x { Z => *x := S(Z), S _ => () }`, and `J` along that identification proves `Eq(Nat, S(Z), Z)`.

A.4 Typing

The typing judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ says that from Ω the term `t` runs to the value `v`, of type `A`, leaving Ω'; its only other outcome is `err`, a type error. It is the machine with types: each rule below performs the step of the machine rule of the same name and also computes a type. Where the machine would be stuck on the checked program, typing splits instead ([Split]), so the typing judgement is never stuck. Callee bodies run in the machine, where stuck calls close off ([App-close]). We write $\Omega \vdash A \Downarrow T$ type for $\Omega \vdash A \Downarrow T : s \dashv \Omega'$ with `s` a sort and Ω' discarded, for a term `A` in a type position, which must also satisfy [Type-pos] below.

Sorts. Sorts are ordered $\mathsf{Prop} < \mathsf{Type}_0 < \mathsf{Type}_1 < \ldots$, and $s \sqcup s'$ is the larger. Universes are *not cumulative*: a type has exactly one sort, and no rule converts between sorts. *Sorts are syntactic* ([Type-pos]): whatever is written in a type position must have a declared type (Section A.4.1) that is syntactically a sort, so `T(Z)` with `T(n) : P(n)` is rejected there even when `P(Z)` computes to `Prop`. Evaluation preserves declared types, so the sort of a type value is its declared sort: the declared sort `s` of `D` for an inductive type $\mathsf{D}(\overline{v})$ (Type_0 for `Nat` and `Unit`, `Prop` for `False`, `True` and `And`); Type_0 for `&T` (with `T : Type₀`); `Prop` for `Eq`; Type_1 for `Prop` and Type_{i+1} for Type_i; for a Π-closure, `Prop` if its class is "returns proofs" (impredicativity), and otherwise the largest sort of its parameter types and codomain; $\Delta(\sigma)$ for an abstract σ whose type is a sort; and for a sealed program `[L; C]` whose head call's function has a declared codomain that is literally a sort `s`, the sort `s`. So there is one notion of proposition, and every value of a proposition is $\star$.

checker: sortOf, sealedSort?, isPropV, typePos

A.4.1 Erasure

A term's *declared type* is read from the declared types of its heads, without normalising: a variable's is the type declared with its binding (a parameter's type, a `let`'s annotation or the declared type of its right-hand side; a captured value keeps its binding's); a call's is its callee's codomain term; a constructor's is its inductive type; a type former's is its sort; an annotated term's is its annotation; a `let`, sequence or match has its tail's, or each arm's; a match some of whose arms are proofs and some not has no declared type, and a definition containing one is rejected, whether or not typing reaches it (typing skips the arms a known scrutinee cannot take, but erasure reads the declared type of every term the machine runs); and the declared type of `J(A, a, b, P, h, t)` is `P(b)`. A type term has *declared sort* `Prop` when it is `Eq`, `Id`, an inductive type $\mathsf{D}(\overline{a})$ with `D` declared in `Prop` (such as `False`, `⊤` or `P ∧ Q`), a `Π` into such a term, a variable declared `: Prop`, a call whose head's codomain term is syntactically `Prop`, or a `let`, sequence or match whose tail or arms have declared sort `Prop`. A Π-type *returns types* if its codomain term is syntactically a sort, *returns proofs* if the codomain's declared sort is `Prop`, and is *other* otherwise; this is its *class*, recorded when it is formed ([T-Pi]), and a function value has the class of its declared Π-type. An occurrence of a term is *erased* when:

1. it is a *proof* (a *declared proof*): its declared type has declared sort `Prop`, or it is a `rewrite`, or a match with no arms whose type (its annotation, or the goal in tail position) is a proposition, or a match on a proof whose inductive has several constructors (Section A.4.6 requires every arm to be a proof); or
2. it *forms a type*: it is a type former (a sort, `Π`, an inductive type $\mathsf{D}(\overline{a})$, `&`, `Eq`, `Id`), a call whose callee returns types, or it stands in a type position or an erased position (Section A.1), or inside one.

A `let`, sequence or match that computes a type is not erased as a whole: its parts decide, so `let T = (c := S(Z); F(&c)); …` writes `c` and erases only the call `F(&c)` if `F` returns types. Nothing is normalised: `U(n)` with `U : Π(n : Nat). Type₁` has declared sort `Type₁`, whatever `U(n)` computes to. By [Type-pos] a type's declared sort is its computed sort, so a term is a proof exactly when its type is a proposition. Every clause reads syntax, a declaration or a sort, so the two paths of Conjecture 2 take the same decisions; notes 1–3 of Section A.7 show what goes wrong otherwise.

$$\frac{t \text{ erased} \qquad \Omega \vdash_0 t \Downarrow v : A \dashv \Omega'}{\Omega \vdash t \Downarrow v^{\bullet} : A \dashv \Omega} \text{ T-Erase}$$

Here $v^\bullet = \star$ if `t` is a proof (clause 1) and $v^\bullet = v$ otherwise, $\vdash_0$ applies the rule for `t`'s own form at the root. So an erased term is typed like any other term, on its own private copy of the environment, and leaves no trace: whatever it writes, borrows or moves is undone with the copy, as at runtime, where it does not run. Erasure is decided from syntax alone, so the two evaluation paths agree on which terms are erased, and with every erased term on its own copy neither path keeps an effect the other discards. A proof may mutate its own locals, and may hand outer places to other erased calls, as `AddMZero`'s recursive call `AddMZero(&p)` does.

checker: `eval`, `fnClass`, `preFlags`, `declOf`, `stmtCopy`, `typeDecl`, `typeClass`, `jErased`, `evalType`, `onCopy (Env.lean)`

A.4.2 Places, sequencing and data

$$\frac{\langle \Omega, p\rangle \Downarrow \langle \Omega', v\rangle}{\Omega \vdash p \Downarrow v : \mathrm{type}_\Omega(p) \dashv \Omega'}\ \textsc{T-Read} \qquad \frac{\langle \Omega, \&p\rangle \Downarrow \langle \Omega', v\rangle \quad \mathrm{type}_\Omega(p) = T : \mathsf{Type}_0}{\Omega \vdash \&p \Downarrow v : \&T \dashv \Omega'}\ \textsc{T-Borrow}$$

$$\frac{\begin{array}{c}\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad A \equiv \mathrm{type}_{\Omega_1}(p) \\ \mathrm{acc}_p^A(\Omega_1 \cdot v) = \Omega_2 \cdot v' \quad \mathrm{drop}\big(\Omega_2, \mathrm{content}_{\Omega_2}(p)\big) = \Omega_3\end{array}}{\Omega \vdash p := t \Downarrow () : \mathsf{Unit} \dashv \Omega_3[p \mapsto v']}\ \textsc{T-Assign}$$

$$\frac{\begin{array}{c}\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 + (x : A \mapsto v) \vdash u \Downarrow w : B \dashv \Omega_2 + (x : A' \mapsto v') \\ \mathrm{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'\end{array}}{\Omega \vdash \mathbf{let}\ x = t;\ u \Downarrow w' : B \dashv \Omega_3}\ \textsc{T-Let}$$

$$\frac{\begin{array}{c}\Omega \vdash A \Downarrow T\ \mathrm{type} \quad \Omega \vdash^T t \Downarrow v : T' \dashv \Omega_1 \quad T' \equiv T \\ \Omega_1 + (x : T \mapsto v) \vdash u \Downarrow w : B \dashv \Omega_2 + (x : T'' \mapsto v') \quad \mathrm{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'\end{array}}{\Omega \vdash \mathbf{let}\ x : A = t;\ u \Downarrow w' : B \dashv \Omega_3}\ \textsc{T-Let-ann}$$

$$\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \qquad \mathrm{drop}(\Omega_1, v) = \Omega_2 \qquad \Omega_2 \vdash u \Downarrow w : B \dashv \Omega_3}{\Omega \vdash t;\ u \Downarrow w : B \dashv \Omega_3}\ \textsc{T-Seq}$$

A binding stores the type of its value, and [Split] refines stored types, so `A'` in [T-Let] is `A` as refined. In [T-Let-ann], $\vdash^T$ checks `t` against `T`: if `t` is a match, it is [Split] with annotation `T`.

checker: `placeType`, `evalCore (.place, .borrow, .assign, .letIn, .seq, .ascribe)`

$$\frac{\begin{array}{c}\mathsf{C}(g_1 : A_1, ..., g_k : A_k) \text{ a constructor of } \mathsf{D}\ \big(\overline{x} : \overline{X}\big) \\ \Omega \vdash \overline{t} \Downarrow^* \overline{v} : \overline{A'} \dashv \Omega' \quad A'_j \equiv A_j[\overline{a}/\overline{x}]\end{array}}{\Omega \vdash \mathsf{C}(\overline{a}; \overline{t}) \Downarrow \mathsf{C}(\overline{a}; \overline{v}) : \mathsf{D}(\overline{a}) \dashv \Omega'}\ \textsc{T-Ctor}$$

The typed argument judgement $\Downarrow^*$ is that of [T-Call]. In [T-Ctor] the parameters $\overline{a}$ are evaluated and typed against $\overline{X}$ as in [T-Ind] (premises omitted); where the examples omit them, they are inferred from the fields' types or the annotation. A constructor of an inductive declared in `Prop` is a declared proof, so [T-Erase] gives it the value $\star$: `refl : T`, and $\langle h, k\rangle : P \wedge Q$ for `h : P` and `k : Q`.

checker: `evalCore (.ctor, .zero, .succ, .tt)`, `evalCtor`

A.4.3 Type formers and transport

All type formers are erased, so each of these rules runs on a private copy ([T-Erase]).

$$\frac{}{\Omega \vdash \mathsf{Prop} \Downarrow \mathsf{Prop} : \mathsf{Type}_0 \dashv \Omega}\ \textsc{T-Sort} \qquad \frac{}{\Omega \vdash \mathsf{Type}_i \Downarrow \mathsf{Type}_i : \mathsf{Type}_{i+1} \dashv \Omega}\ \textsc{T-Type}$$

$$\frac{\begin{array}{c}\mathbf{inductive}\ \mathsf{D}\ \big(\overline{x} : \overline{X}\big) : s := ... \quad \Omega \vdash \overline{a} \Downarrow^* \overline{v} : \overline{A'} \dashv \Omega' \\ \Omega;\ (x_1 : T_1 \mapsto v_1, ..., x_{i-1} : T_{i-1} \mapsto v_{i-1}) \vdash X_i \Downarrow T_i\ \mathrm{type} \quad A'_i \equiv T_i\end{array}}{\Omega \vdash \mathsf{D}(\overline{a}) \Downarrow \mathsf{D}(\overline{v}) : s \dashv \Omega'}\ \textsc{T-Ind} \qquad \frac{\Omega \vdash A \Downarrow T\ \mathrm{type} \qquad T : \mathsf{Type}_0}{\Omega \vdash \&A \Downarrow \&T : \mathsf{Type}_0}\ \textsc{T-Ref}$$

$$\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v : T_a \equiv T \quad \Omega \vdash b \Downarrow w : T_b \equiv T}{\Omega \vdash \mathsf{Eq}\ A\, a\, b \Downarrow \mathrm{eq}(T, v, w) : \mathsf{Prop}}\ \text{T-Eq}$$

$$\frac{\langle \Omega, \Pi\big(\overline{x} : \overline{A}\big).B\rangle \Downarrow \langle \Omega', \Pi\rangle \quad \mathrm{sort}(\Pi) = s \quad B = \&T \text{ implies some } A_i = \&T_i}{\Omega \vdash \Pi\big(\overline{x} : \overline{A}\big).B \Downarrow \Pi : s \dashv \Omega'}\ \text{T-Pi}$$

A Π-closure records its class and whether its codomain is a borrow type, both read from the codomain term, and [Conv-pi] compares them, so a function value is used only at a type of its own class. A Π-type whose codomain is a borrow type must have a borrow parameter: a returned borrow always derives from a borrow argument, as in Rust's lifetime elision, except that Ochr has no `'static` borrows (note 7 of Section A.7); [Def] imposes the same condition on the type of every `fix`. [T-Ref] admits borrows of types in Type_0 only (note 11). In [T-Ind] the parameters are typed as the arguments of a call are ([Call-type]). In [T-Eq] the sides are typed one after the other on the same private copy (the whole `Eq` is a type); in [T-Pi] the sort is computed at the generic arguments. The rule for `Id` is [Id] of Section A.3.

checker: `evalCore (.sort, .nat, .unit, .ref, .eq, .pi, .id, .tind)`, `evalTInd`, `isDataType`, `sortOf`

$$\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v_a : T_a \equiv T \quad \Omega \vdash b \Downarrow v_b : T_b \equiv T \quad \Omega \vdash P \Downarrow F : \Pi \quad v_a \equiv v_b \quad \Omega \vdash h \Downarrow \star : H \equiv \mathrm{eq}(T, v_a, v_b) \quad \Omega \vdash F(v_a) \Downarrow P_a : s \quad \Omega \vdash F(v_b) \Downarrow P_b : s \quad \Omega \vdash t \Downarrow v : T_t \dashv \Omega' \quad T_t \equiv P_a}{\Omega \vdash \mathsf{J}(A, a, b, P, h, t) \Downarrow v : P_b \dashv \Omega'}\ \text{T-J}$$

$$\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v_a : T_a \equiv T \quad \Omega \vdash b \Downarrow v_b : T_b \equiv T \quad v_a \not\equiv v_b \quad \Omega \vdash P \Downarrow F : \Pi \quad \Omega \vdash h \Downarrow \star : H \equiv \mathrm{eq}(T, v_a, v_b) \quad \Omega \vdash F(v_a) \Downarrow P_a : s \quad \Omega \vdash F(v_b) \Downarrow P_b : s \quad \Omega \vdash t \Downarrow v : T_t \equiv P_a \text{ on a private copy} \quad \langle \Omega, \mathrm{block}(\Omega, \mathsf{J}(T, v_a, v_b, F, \star, t), P_b, M)\rangle \Downarrow \langle \Omega', w\rangle}{\Omega \vdash \mathsf{J}(A, a, b, P, h, t) \Downarrow w : P_b \dashv \Omega'}\ \text{T-J-stuck}$$

`refl` proves `⊤` ([T-Ctor]), and so by conversion every reflexive equation. `J` takes its endpoints explicitly, because `Eq(A, a, a)` computes to `⊤` and no longer records them. When the endpoints are not convertible the machine is stuck ([J-stuck]), so typing checks `t` against `P(a)` on a private copy and closes the `J` off as a stuck block of type `P(b)` ([T-J-stuck]), as [Split] closes off a stuck match. `M` is the set of places `t`'s run moves out of. The block's code embeds the endpoints, the motive and the equation as values, since they are erased and formed once, so it captures only `t`'s places. Its sealed programs run `t` once a refinement makes the endpoints convertible. Its motive may have any sort; with a motive into `Prop`, `J` is a proof and is erased, and neither [T-J] nor [T-J-stuck] applies. `A` is in a type position, `a`, `b` and `P` in erased positions, and `h` is a proof, so all five are typed on private copies, and the motive's calls $F(v_a)$, $F(v_b)$ are erased calls.

checker: `evalCore (.prim "J")`, `closeOffMatch`, `evalCtor`, `unifyParams`

$$\frac{\Omega \vdash h \Downarrow \star : H \equiv \mathrm{eq}(T, v_a, v_b) \quad G \text{ a proposition} \quad G' = G[\sigma/v_b]^{+}[v_a/\sigma],\ \sigma \text{ fresh} \quad \Omega \vdash^{G'} t \Downarrow \star : T_t \dashv \Omega' \quad T_t \equiv G'}{\Omega \vdash^{G} \mathbf{rewrite}\ h\ \ \mathbf{in}\ t \Downarrow \star : G \dashv \Omega}\ \text{T-Rewrite}$$

`rewrite h in t` needs a known goal `G`: it is checked in checking mode, in tail position, as a call's argument or under an annotated `let`. It replaces every occurrence of `b`'s normal form in `G` by `a`, using the replacement of [Split-gen] locally to the goal, and checks `t` against the result; `rewrite ← h in t` swaps `a` and `b`. It is `J(A, a, b, λz. G[z/b], h, t)` with the motive read off the goal, a proof, and never runs, so [J]'s condition on the endpoints does not arise.

checker: `evalCore (.prim "rewrite")`, `Surface.resolve (.rewrite)`

A.4.4 Functions and calls

$$\frac{f \in \Sigma}{\Omega \vdash f \Downarrow f : \Sigma(f).\text{type} \dashv \Omega}\ \textsc{T-Global}$$

$$\frac{c \in \Sigma}{\Omega \vdash c \Downarrow \Sigma(c).\text{value} : \Sigma(c).\text{type} \dashv \Omega}\ \textsc{T-Const} \qquad \frac{\langle \Omega, \textbf{fix} \ldots \rangle \Downarrow \langle \Omega', F \rangle \qquad \vdash F \text{ ok}}{\Omega \vdash \textbf{fix} \ldots \Downarrow F : \text{typeof}(F) \dashv \Omega'}\ \textsc{T-Fix}$$

A `fix` is checked where it is formed, by [Def] at its own generic call, with its captured values; its value is $\star$ if its Π-type returns proofs.

checker: `evalCore (.const, .fix)`, `checkFix`

[Call-type]. At the call point Ω of a call with function type $\Pi = \langle \overline{\kappa} \vdash \Pi(x_1 : A_1 \ldots x_n : A_n).B \rangle$ and arguments $\overline{w}$ of types $\overline{A'}$, push the frame of captured values and bind the parameters one by one, on a private copy:

$$\frac{\begin{array}{c}\Omega;\ (\overline{\kappa}, x_1 : T_1 \mapsto w_1, \ldots, x_{i-1} : T_{i-1} \mapsto w_{i-1}) \vdash A_i \Downarrow T_i \text{ type} \quad A_i' \equiv T_i \quad (1 \le i \le n) \\ \Omega;\ \big(\overline{\kappa}, \overline{x} : \overline{T} \mapsto \overline{w}\big) \vdash B \Downarrow T_B \text{ type}\end{array}}{\text{callty}_\Omega\big(\Pi, \overline{w} : \overline{A'}\big) = T_B}\ \textsc{Call-type}$$

The free variables of `B` other than $\overline{x}$ were captured when Π was formed; the caller's frames below are not in scope, but the footprints and owners of observations in `B` see them, which is how an induction hypothesis sees its caller's context. A borrow argument is moved into its parameter.

checker: `callType`

Typed arguments are evaluated like [Args], each into a temporary, collecting their types: $\Omega \vdash \overline{u} \Downarrow^* \overline{w} : \overline{A'} \dashv \Omega'$.

$$\frac{\begin{array}{c}\Omega \vdash t_0 \Downarrow F : \Pi \dashv \Omega_0 \quad \Omega_0 \cdot F \vdash \overline{u} \Downarrow^* \overline{w} : \overline{A'} \dashv \Omega_1 \cdot F' \quad \text{no } w_i \text{ is } \bot \\ \text{callty}_{\Omega_1}\big(\Pi, \overline{w} : \overline{A'}\big) = B' \quad \text{rec}_{\Omega_1}(F', \overline{w}) \quad \langle \Omega_1, F'(\overline{w}) \rangle \Downarrow_{\text{app}} \langle \Omega', v \rangle\end{array}}{\Omega \vdash t_0(\overline{u}) \Downarrow v : B' \dashv \Omega'}\ \textsc{T-Call}$$

$$\frac{\begin{array}{c}\Omega \vdash t_0 \Downarrow F : \Pi \dashv \Omega_0 \quad \Omega_0 \cdot F \vdash \overline{u} \Downarrow^* \overline{w} : \overline{A'} \dashv \Omega_1 \cdot F' \quad \text{no } w_i \text{ is } \bot \\ \text{callty}_{\Omega_1}\big(\Pi, \overline{w} : \overline{A'}\big) = B' \quad F' \text{ returns proofs} \quad \text{rec}_{\Omega_1}(F', \overline{w})\end{array}}{\Omega \vdash t_0(\overline{u}) \Downarrow \star : B' \dashv \Omega_1}\ \textsc{T-Call-proof}$$

[T-Call-proof] applies to a call whose callee returns proofs (Section A.4.1), and [T-Erase] then discards Ω_1: a proof call's arguments are evaluated and checked, and its body is not run. In [T-Call] the callee's body runs in the machine ($\Downarrow_{\text{app}}$), since the callee was checked by its own [Def]; a call with a neutral head closes off at once.

checker: `evalCall`, `callFn`, `callType`

[Rec]. The [Rec] stack holds an entry $\big(F, j, \sigma_j\big)$ for every enclosing function being checked that declares `by` $\mathtt{x}_j$, where σ_j is the entry value of x_j at its generic call (through the borrow, for &`D`). $\text{rec}_\Omega(F, \overline{w})$ holds when, for every entry $\big(F, j, \sigma_j\big)$ of the stack for this `F`, the argument w_j, or its content if $w_j = \text{borrow}_\ell u$, is in $\text{sub}\big(\rho^*\big(\sigma_j\big)\big)$: a strict subterm, through any field, of the entry value as refined so far. Entries stay on the stack while nested functions and block arms are checked, so recursive calls there are checked too.

checker: `recCheck`, `headOnly`

A.4.5 Matches and case splitting

Let $m = \textbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1, \ldots, \mathsf{C}_m \Rightarrow t_m\}$ match on a place of inductive type $\mathsf{D}(\overline{a})$, with `D` declared in $\mathtt{Type}_0$ and constructors $\mathsf{C}_1, \ldots, \mathsf{C}_m$ (the place's stored type must be $\mathsf{D}(\overline{a})$ for the `D` whose constructors the arms name: a match is never typed from its arms), and let $r_i = \mathsf{C}_i\big(\sigma_{i1}, \ldots, \sigma_{ik_i}\big)$ be the refinement to C_i with fresh field values (Section A.1.4, item 7). On a constructor the typing judgement takes the arm, as the machine does, and checks only that arm. On an abstract value it splits ([Split]): each arm is checked under its refinement, and the match is then closed off as a stuck block, from which the rest of the program is checked once.

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = \mathsf{C}_i(\overline{w}) \quad \Omega_1 \vdash t_i \Downarrow v : A \dashv \Omega'}{\Omega \vdash m \Downarrow v : A \dashv \Omega'}\ \textsc{T-Match}$$

$$\frac{\begin{array}{c}\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = \sigma \quad \Omega_1[\sigma := r_i] \vdash t_i \Downarrow v_i : A_i \dashv \Omega_i \quad (1 \le i \le m) \\ B = T \text{ with } A_i \equiv T[r_i/\sigma] \text{ for all } i, \quad \text{or, without annotation, } B = A_1 \equiv \ldots \equiv A_m \\ M = \{x \mid \Omega_1(x) \neq \bot, \text{some } \Omega_i(x) = \bot\} \quad \langle \Omega_1, \mathrm{block}(\Omega_1, m, B, M)\rangle \Downarrow \langle \Omega', v\rangle\end{array}}{\Omega \vdash m \Downarrow v : B \dashv \Omega'}\ \textsc{Split}$$

$$\frac{\begin{array}{c}\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = n \text{ a sealed program or an inert loan} \\ \sigma \text{ fresh}, \Delta(\sigma) = \mathrm{type}_{\Omega_1}(p) \quad \Omega_1[\sigma/n]^+ \text{ with } n := \sigma \text{ recorded in } \rho \quad \vdash m \Downarrow v : B \dashv \Omega'\end{array}}{\Omega \vdash m \Downarrow v : B \dashv \Omega'}\ \textsc{Split-gen}$$

In [Split] every arm is checked from Ω_1, and the arms' states and values are discarded; each refinement is applied to the environment, the goal and every stored type. `T` is the annotation of an enclosing `let x : T = m` (checking mode, $\vdash^T$), whose type is then `T`; without one, the arms' types must be convertible. The block is run by the machine, from the unrefined Ω_1. [Split-gen] generalises a neutral head first (Lean's `generalize`): it replaces the neutral everywhere in Ω, the goal, the stored types and the annotation, and records it in ρ, so that the replacement persists when the neutral is derived again; the record survives a private copy in which it was made, and is dropped when the arm that made it ends, since sibling arms are exclusive worlds in which the same program text can have a different type. Types are terms, so a match inside a type is checked in the same way.

checker: `evalMatch`, `evalMatchInd`, `splitArmsThenClose`, `ctorRefinement`, `generalizeNeutral`, `refine`

$$\frac{\begin{array}{c}n \text{ is the first link, in pre-order over the goal's sealed programs, of the chains their runs are stuck on} \\ \text{(a sealed program, or an abstract value standing for a recorded one) whose head call is a call of } f \\ \sigma = n \text{ if } n \text{ is an abstract value; otherwise } \sigma \text{ fresh, } \Delta(\sigma) \text{ the type of } n, \text{ and } n := \sigma \text{ recorded in } \rho \\ \Omega[\sigma/n]^+ \vdash \mathbf{match}\ \sigma\{\mathsf{C}_1 \Rightarrow t, \ldots, \mathsf{C}_m \Rightarrow t\} \rightsquigarrow \mathcal{L}\end{array}}{\Omega \vdash \mathbf{split}\ f\ \ \mathbf{in}\ t \rightsquigarrow \mathcal{L}}\ \textsc{T-Split-goal}$$

`split f in t` is [Split-gen] and [Tail-split] with the scrutinee found in the goal instead of written as a place, as Lean's `split` is: it generalises the first result of a call of `f` on which the goal's sealed programs are stuck (never one of those programs itself), and checks `t` in every arm; `split f { C₁ => t₁, … }` gives each arm its own term. It never runs and adds nothing to the model. The head function is named because a goal's chain of stuck results usually has several sealed links, and different proofs need different ones. The checker reads $\Delta(\sigma)$ from the head's declared result type, at the call's arguments.

checker: `checkTail`, `splitTarget`, `findSplit`

Tail position. The body of a definition is checked by a judgement $\Omega \vdash t \rightsquigarrow \mathcal{L}$ whose result is the finite set $\mathcal{L}$ of the *leaves* of its case tree: triples (Ω', v, A), each in the state of its own path, with the goal refined along that path. A match is in tail position when only trailing drops follow it.

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = \sigma \quad \Omega_1[\sigma := r_i] \vdash t_i \rightsquigarrow \mathcal{L}_i \quad (1 \le i \le m)}{\Omega \vdash m \rightsquigarrow \mathcal{L}_1 \cup \ldots \cup \mathcal{L}_m}\ \textsc{Tail-split}$$

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = n \quad \Omega_1[\sigma/n]^+ \vdash m \rightsquigarrow \mathcal{L}}{\Omega \vdash m \rightsquigarrow \mathcal{L}}\ \textsc{Tail-gen}$$

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{content}_{\Omega_1}(p) = \mathsf{C}_i(\overline{w}) \quad \Omega_1 \vdash t_i \rightsquigarrow \mathcal{L}}{\Omega \vdash m \rightsquigarrow \mathcal{L}}\ \textsc{Tail-match}$$

$$\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 + (x : A \mapsto v) \vdash u \rightsquigarrow \mathcal{L}}{\Omega \vdash \mathbf{let}\ x = t;\ u \rightsquigarrow \{\mathrm{pop}_x(\ell) \mid \ell \in \mathcal{L}\}}\ \textsc{Tail-let}$$

$$\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \qquad \mathrm{drop}(\Omega_1, v) = \Omega_2 \qquad \Omega_2 \vdash u \rightsquigarrow \mathcal{L}}{\Omega \vdash t;\, u \rightsquigarrow \mathcal{L}}\ \textsc{Tail-seq}$$

$$\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega' \qquad t \text{ is not a match, let or sequence}}{\Omega \vdash t \rightsquigarrow \{(\Omega', v, A)\}}\ \textsc{Tail-end}$$

Here $\mathrm{pop}_x(\Omega_2 + (x : A' \mapsto v'), w, B) = (\Omega_3, w', B)$ with $\mathrm{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'$, and an annotated `let` checks `t` against its annotation as in [T-Let-ann]. In [Tail-gen], σ is fresh and $n := \sigma$ is recorded in ρ, as in [Split-gen]. An error on any path is an error.

checker: `checkTail`

A.4.6 Matches on proofs

A match whose arms name the constructors of an inductive declared in `Prop` is a match on a proof. Its scrutinee's value is $\star$ (a proof parameter is bound to $\star$ at the generic call, and an erased term returns $\star$), so it is decided by the scrutinee's type, $\mathrm{type}_{\Omega_1}(p) = \mathsf{D}(\overline{v})$, and never by its content, and nothing is refined. `D` is a *subsingleton* if it has no constructors, or one constructor all of whose field types have declared sort `Prop`; `False`, `True` and `And` are subsingletons. Arm i is checked from Ω_1^i, which is Ω_1 if every field of C_i is a proof, so that its fields hold $\star$ (Section A.1.4, item 1), and otherwise $\Omega_1[p \mapsto \mathsf{C}_i(\overline{v}; \overline{w})]$ with $w_j = \star$ for a proof field and w_j a fresh abstract value of the field's type for a data field, as [Def] binds parameters. The second case arises only in an erased match, on its private copy.

$$\frac{\begin{array}{c}\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{type}_{\Omega_1}(p) = \mathsf{D}(\overline{v}),\ \mathsf{D} \text{ declared in } \mathsf{Prop} \text{ with the one constructor } \mathsf{C}_1 \\ \Omega_1^1 \vdash t_1 \Downarrow v : A \dashv \Omega' \quad t_1 \text{ is a declared proof unless } \mathsf{D} \text{ is a subsingleton}\end{array}}{\Omega \vdash \mathbf{match}\ p\{\mathsf{C}_1 \Rightarrow t_1\} \Downarrow v : A \dashv \Omega'}\ \textsc{T-Match-prop}$$

$$\frac{\begin{array}{c}\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{type}_{\Omega_1}(p) = \mathsf{D}(\overline{v}),\ \mathsf{D} \text{ declared in } \mathsf{Prop} \text{ with constructors } \mathsf{C}_1, ..., \mathsf{C}_m,\ m \geq 2, \text{or } \mathsf{D} \text{ with no constructors and } B \text{ a proposition} \\ \Omega_1^i \vdash t_i \Downarrow \star : A_i \dashv \Omega_i,\ t_i \text{ a declared proof} \quad (1 \leq i \leq m) \\ B = T \text{ with } A_i \equiv T, \text{or, for } m \geq 2 \text{ without annotation, } B = A_1 \equiv ... \equiv A_m\end{array}}{\Omega \vdash m \Downarrow \star : B \dashv \Omega_1}\ \textsc{T-Match-erased}$$

$$\frac{\begin{array}{c}\mathrm{acc}_p^M(\Omega) = \Omega_1 \quad \mathrm{type}_{\Omega_1}(p) = \mathsf{D}(\overline{v}),\ \mathsf{D} \text{ with no constructors} \\ B = T \text{ not a proposition} \quad \langle \Omega_1, \mathrm{block}(\Omega_1, m, B, \emptyset)\rangle \Downarrow \langle \Omega', v\rangle\end{array}}{\Omega \vdash \mathbf{match}\ p\{\} \Downarrow v : B \dashv \Omega'}\ \textsc{T-Match-none}$$

$$\frac{\mathrm{acc}_p^M(\Omega) = \Omega_1 \qquad \mathrm{type}_{\Omega_1}(p) = \mathsf{D}(\overline{v}),\ \mathsf{D} \text{ declared in } \mathsf{Prop} \text{ or with no constructors} \qquad \Omega_1^i \vdash t_i \rightsquigarrow \mathcal{L}_i \quad (1 \leq i \leq m)}{\Omega \vdash m \rightsquigarrow \mathcal{L}_1 \cup ... \cup \mathcal{L}_m}\ \textsc{Tail-prop}$$

In [T-Match-erased], `T` is the annotation of an enclosing `let x : T = m`; with no arms, the annotation, or tail position, is the only source of the result type. With no arms, the match is a proof only when that type is a proposition; otherwise [T-Match-none] closes it off as a stuck block, which is stuck as [Match-none] is, so it never has the value $\star$ at a data type (D58). [T-Match-erased] is a declared proof (Section A.4.1), so [T-Erase] applies to it. In [Tail-prop], each t_i is a declared proof unless `D` is a subsingleton; with no arms $\mathcal{L} = \emptyset$, and [Def]'s check on every leaf holds vacuously, which is ex falso. *Subsingleton elimination* is the side condition "a declared proof unless `D` is a subsingleton": it is Lean's, where without it proof irrelevance is inconsistent; in Ochr it is needed for the model (note 10 of Section A.7). The decision is stable under refinement, which changes a type's parameters but never its head; a scrutinee whose type is not (yet) an inductive, such as ⌈`Le(σ, σ')`⌉ before a split, has no rule, so it is a type error until a split has computed its type. The machine takes the same decisions from syntax alone: [Match-prop] runs a one-constructor match, and every other match on a proof, and every match with no arms in a proof position, is a declared proof it never runs; a match with no arms elsewhere is stuck.

checker: `byTypeMatch`, `scrutType`, `evalMatchByType`, `checkTail`, `bindDataFields`, `largeElim`, `fieldTermIsProp`

A.4.7 Definitions

[Def]. Let `F` be a top-level function or a closure with captured values $\overline{\kappa}$ and code **fix** $f(x_1 : A_1 ... x_n : A_n) : B$ $[\mathbf{by}\ x_j] := b$, and let $\Pi = \text{typeof}(F)$, with generic state $\Gamma = \Gamma(\Pi)$, parameter frame $\varphi = \varphi(\Pi)$ and parameter types $\overline{T}$ (Section A.1.4, item 10). Let φ^+ be φ with $f : \Pi \mapsto F$ added when `F` declares `by`.

$$\frac{\begin{array}{c} F \text{ top-level implies } f \notin \Sigma \\ f \text{ occurs in } b \text{ only as the head of a call } \quad [\mathbf{by}\ x_j] : T_j \in \{\mathsf{D}(\overline{v}), \&\mathsf{D}(\overline{v})\},\ \mathsf{D} \text{ declared in } \mathsf{Type}_0 \\ B = \&T \text{ implies some } T_i = \&T_i' \\ \Gamma;\ \varphi \vdash B \Downarrow G \text{ type} \quad \Gamma;\ \varphi^+ \vdash b \rightsquigarrow \mathcal{L} \quad \text{with goal } G \text{ and the [Rec] entry } \big(F, j, \sigma_j\big) \text{ pushed} \\ \text{for every } (\Omega', v, A) \in \mathcal{L} \text{ with refined goal } G' : \quad \text{pop}(\Omega', v) \text{ succeeds and } A \equiv G' \end{array}}{\vdash F \text{ ok}}\ \textsc{Def}$$

In the first premise, `f` must also occur only as a call head inside nested functions and block arms. The goal `G` is the [Call-type] of the generic call $F(\overline{a})$: `B` evaluated with each parameter bound to its generic argument, borrow parameters to borrows of the fresh owned places c_i, proof parameters to $\star$. Parameter types are evaluated left to right and stored with their bindings, where [Split] refines them. Popping the body's frame drops its bindings, so each borrow parameter ends and its owned place receives the final content; the result's type must then convert to the goal as refined on that path. A top-level `f` is added to Σ, with its type, code and erasure flag, before its body is checked, so that its recursive calls resolve; it is kept only if the check succeeds.

checker: `checkFix`, `checkDef (Check.lean)`, `declared (Check.lean)`

$$\frac{\begin{array}{c} \mathsf{D}, \mathsf{C}_1, ..., \mathsf{C}_m \notin \Sigma \text{ and pairwise distinct} \\ s \in \{\mathsf{Type}_0, \mathsf{Prop}\} \quad m \geq 0 \quad \varepsilon;\ (x_1 : T_1 \mapsto \sigma_1, ..., x_{i-1} : T_{i-1} \mapsto \sigma_{i-1}) \vdash X_i \Downarrow T_i \text{ type}, T_i \text{ has no } \& \quad (1 \leq i \leq l) \\ g_{ij} : A_{ij} : \quad A_{ij} \text{ is a parameter, is built from } \mathsf{D} \text{ , declared inductive types, } \times \text{ and type functions (@app-dep), or is an equation over first-orde} \\ \Sigma + \mathsf{D}\ ;\ \Big(\overline{x} : \overline{T} \mapsto \overline{\sigma}, g_{i1} : T_{i1} \mapsto \tau_1, ..., g_{i(j-1)} : T_{i(j-1)} \mapsto \tau_{j-1}\Big) \vdash A_{ij} \Downarrow T_{ij} : s_{ij} \quad s_{ij} \in \{\mathsf{Prop}, \mathsf{Type}_0\} \end{array}}{\vdash \Big(\mathbf{inductive}\ \mathsf{D}\ \Big(\overline{x} : \overline{X}\Big) : s := \mathsf{C}_1\Big(\overline{g_1 : A_1}\Big) \mid ... \mid \mathsf{C}_m\Big(\overline{g_m : A_m}\Big)\Big) \text{ ok}}\ \textsc{Ind}$$

$$\frac{c \notin \Sigma \qquad \varepsilon \vdash A \Downarrow T \text{ type} \qquad \varepsilon \vdash t \Downarrow v : T' \dashv \Omega' \qquad T' \equiv T \qquad \Sigma(c) = (T, v^\bullet)}{\vdash (c : A := t) \text{ ok}}\ \textsc{Const}$$

Parameters are checked like the parameters of a [Def], each bound to a fresh abstract value (or $\star$) while later ones and the fields are checked, and none has a borrow type, since a parameter's argument is recorded in the type and in every constructor value. An argument of $\mathsf{D}(\overline{a})$ or $\mathsf{C}(\overline{a}; \overline{t})$ at a type parameter stands in a type position, and one at a value parameter in an erased position (Section A.1). Field types are first-order data, parameters, calls of type functions (Section A.4.8), or equations `Eq(T, a, b)` with `T` first-order data, which are proof fields: they may mention the type being declared, but contain no `Π` and no `&`, which is strict positivity in its simplest form (note 4 of Section A.7). An occurrence of `D` in a field type may take any arguments: first-order types at type parameters, and erased terms over the parameters and the earlier fields at value parameters, so a family indexed by values is Forded, `Vec(A, n) := VNil(h : Eq(Nat, n, Z)) | VCons(m : Nat, x : A, xs : Vec(A, m), h : Eq(Nat, n, S(m)))`. No argument of an occurrence of `D`, and no part of an equation field, mentions `D`, and a parameter that an equation field mentions is not nestable (Section A.4.8); a field's type may mention the fields before it, each bound to a fresh generic value τ ($\star$ for a proof field). There may be no constructors (`False`), and the sort may be `Prop`, for any fields; what a match on a proof may produce is restricted instead (Section A.4.6). A definition without parameters is a constant: its body is checked against its type from the empty environment, and its value is stored ($v^\bullet = \star$ if `T` has declared sort `Prop`). A definition may not mention itself in its type. Σ has one namespace: $x \notin \Sigma$ says that no inductive type, constructor or definition of Σ is named x. An identifier is read as a type, then as a constructor, then as a definition, so a declaration that reused a name could never be reached, and a check that reported it by name would report another declaration.

checker: `checkInd (Check.lean)`, `checkDef (Check.lean)`, `declared (Check.lean)`

A.4.8 Dependent fields

A field's type may mention the constructor's earlier fields, as in the sized vector `Vec(E) := MkVec(n : Word, items : Array(E, n))`. [Ind-decl] checks field `j`'s type with the parameters and the fields before it bound to fresh generic values, and rejects a type that mentions field `j` or a later one; an earlier field occurs only as an argument inside a later field's type. A field that a later field's type mentions is an *index field*, and a constructor with one is *dependent*. A field type may be a call $F(\overline{a})$ of an earlier definition whose declared codomain is a sort (`Array(E, n)`, or a proof field `Sorted(xs)`), with arguments that are parameters, fields, numerals or first-order types not mentioning `D`, and which at the generic fields computes to no Π-type, borrow type or sort (a call that computes to an equation is a proof field, like a written one). `D` may occur inside another inductive `D'(...)` only at a *nestable* parameter of `D'`, one that occurs in its field types only as a whole field type or at a nestable parameter of an inductive; otherwise `MkNBox(n : Word, f : NegIf(n, A), h : IsSucc(n))`, where `NegIf(n, A)` is `Unit` at `Zero` and `Π(x : A). Void` at a successor, would let `MkBad(b : NBox(Bad))` bring back the non-positive `Bad` of note 4. A field that is a Π at the generic fields, such as `Neg(A) := Π(x : A). Void`, is rejected already.

Typing by position. The type of a field place $p.g_j$ of a dependent constructor is its declared type with the parameters taken from `p`'s type and each earlier field bound to the current content of `p`'s field, so $\mathsf{type}(p.\mathsf{items}) = \mathsf{Array}(E, \mathsf{content}(p.n))$ ([Field]). [Split] refines $\sigma := \mathsf{C}(\overline{a}; \sigma_1, ..., \sigma_k)$ with each σ_i fresh at the type computed from $\sigma_1, ..., \sigma_{i-1}$, [T-Ctor] checks each argument against the type computed from the earlier arguments, and [Assign] types the value it writes by the target place's type.

Open and repack. A dependency may be broken while a value is mutated, and is re-checked where the value must be whole again. Writing, borrowing or moving out a field of a dependent constructor *opens* the value ([Open]). While it is open, a data field whose declared type mentions earlier fields takes its content's type whenever the content does not have its telescope type, and assigning it is a strong update, typed by the value; a write or borrow through an index field sets every proof field whose type mentions it to $\bot$, which is an error to read until it is assigned, against its type at the current contents. While part of the value is lent, the assignment is checked with what each such borrow holds a fresh abstract value of its type: the borrow may still write there, not through a field place, so nothing would set the proof to $\bot$ again, and [Repack] cannot check a proof. A proof re-assigned while an element of the value is lent must therefore hold whatever the borrow leaves, which an invariant such as "this cell is full" does and "this field is not zero" does not. At every *whole-again* point, each value of a dependent constructor inside the value must have its type by its telescope, with no $\bot$ inside ([Repack]); a failure is a type error there. The points are a place read, moved or borrowed whole (so a borrow variable passed on), the end of a borrow of the value (a borrow parameter's when the function returns, and, since a stuck block is a call, the block's borrow of a place at the end of each arm, in the state [Split] checks that arm in), a function's or constant's result, `Id`'s observed result and owners, and the captures of a stuck block, closure or Π-type. The direct path runs the match without a block, so no borrow ends there; checking the arms' ends on the generic path is the stricter of the two, and rejects a lie that a later match would repair. So in a match on `*v`, the arm `MkVec(n, items) => (n := W(2); items := Two)` is accepted with the two assignments in either order: `*v` is open between them and whole at the return. The machine's own runs, the concrete calls a checked program makes and the re-runs of sealed programs, do not check [Repack], because they are instances of generically checked code.

Injectivity, restricted. For a dependent constructor, [Eq-inj] applies only when every index field is convertible on both sides, and the field types are then computed from the left side; otherwise the equation stays as it is. This fails safe and is incomplete: the full rule of observational type theory needs a dependent conjunction. Proof fields are $\star$ on both sides and never block. [Close], sealed programs, observation and the model are unchanged; in the model a dependent constructor denotes a Σ-type.

checker: `checkInd (Check.lean)`, `paramNestable (Check.lean)`, `IndDecl.indexFields (Env.lean)`, `fieldTele`, `depFieldType`, `openWrite`, `lentProofAssign`, `strongUpdate`, `repackCheck`, `repackAt`, `mkEqM`

A.5 Well-formed environments

A state (Ω, Δ, Σ) is *well formed* when the following hold. They are the four conditions of Section 7.1, spelled out, plus typing.

1. **Borrows are unique and outermost.** Each label ℓ has at most one borrow_ℓ in Ω, and a borrow occurs only as the whole value of a binding or temporary: never inside data, a sealed program, a closure or a type.
2. **Loans are bound.** Every loan_ℓ in Ω has its borrow_ℓ in Ω, except inside a [Seal] run, where a loan whose borrow lies outside the run is inert. Outside sealed programs a label occurs as a loan at most once; inside sealed programs it may occur any number of times, since the hole of a returned borrow sits in the fill of every borrowed argument (Figure 13).
3. **Loans are acyclic.** The relation "loan_ℓ occurs in the content of borrow_m" is acyclic, so owners is well defined.
4. **Captures are values.** Closures and Π-closures capture no borrow and no $\bot$, and sealed programs are closed.
5. **Arguments are exclusive.** At every call point, each argument is loan-free or is $\text{borrow}_\ell u$ with `u` loan-free. This is [Close]'s precondition, and [Access] maintains it.
6. **Values have their types.** For each binding `x : A ↦ v`: $v = \bot$ (moved out or ended), or $v = \text{loan}_\ell$ and the content of borrow_ℓ has type `A`, or `v` has type `A` (a neutral having the type recorded for it). Every σ occurring in Ω, in a stored type or in the goal has a type in Δ, and every top-level name has an entry in Σ.

That the machine preserves conditions 1 to 5 is property 1 of Figure 9; it is not proved, and Section A.8 assumes it for its fragment (Assumption 11).

A.6 The model

This section gives the interpretation that Section 7.1 summarises, as a sketch.

Types. An inductive declaration in $\texttt{Type}_0$ denotes the least set closed under its constructors, at each value of its parameters, so `Nat` denotes $\mathbb{N}$; constructors are injective and have disjoint images. A declaration with value parameters, or whose recursive occurrences take other arguments than its parameters, denotes a family of such sets, one least fixed point over all parameter values at once, the standard construction for indexed inductive definitions; an equation field denotes $\{\bullet\}$ if its sides are equal and $\emptyset$ otherwise. One in `Prop` denotes a subsingleton, $\{\bullet\}$ if that least set is inhabited and $\emptyset$ otherwise: `False` denotes $\emptyset$, `True` denotes $\{\bullet\}$, and `And(P, Q)` denotes $P^\dagger \cap Q^\dagger$. `Eq(A, a, b)` denotes $[a^\dagger = b^\dagger] \subseteq \{\bullet\}$. For a function type with borrow parameters $\texttt{x}_i$ `:` $\texttt{\&T}_i$ at positions $i \in I$, let $D_i = T_i^\dagger$ for $i \in I$, $D_j = A_j^\dagger$ otherwise, and $\text{Fin} = \prod_{i \in I} T_i^\dagger$, the final contents of the borrowed places:

$$\big(\Pi(\overline{x} : \overline{A}).B\big)^\dagger = R_{\Pi(\overline{x}:\overline{A}).B} \subseteq \Pi\big(\overline{d} : \overline{D}\big).\text{Out}_B\big(\overline{d}\big), \quad \text{Out}_B\big(\overline{d}\big) = \begin{cases} B^\dagger\big(\overline{d}\big) & \text{if } B \text{ is a proposition,} \\ B^\dagger\big(\overline{d}\big) \times \text{Fin} & \text{if } B \text{ is borrow-free data,} \\ T^\dagger \times (T^\dagger \to \text{Fin}) & \text{if } B = \&T. \end{cases}$$

Here $B^\dagger\big(\overline{d}\big)$ is B at the generic call of [Def]. We write fwd_F and back_F for the two components of $F\big(\overline{d}\big)$. For a returned borrow, $\text{back}_F\big(\overline{d}\big)$ is Aeneas's backward function: it maps the value the caller eventually leaves in the borrow to the final contents. A proposition has no Fin component, because proofs run on a private copy (Section 6).

Definition 5 (injectivity relation). $R_A \subseteq A^\dagger$ is defined by induction on A. For propositions and universes $R_A = A^\dagger$; at an inductive type, R holds of a constructor value when it holds of each field at the field's type, so it reaches functions stored under a type parameter, such as `Box(Π(y : &Nat). &Nat)`; and $R_{\&T} = R_T$. For a function type, $F \in R_{\Pi(\overline{x}:\overline{A}).B}$ iff, for all $\overline{d}$ with $d_i \in R_{T_i}$ for $i \in I$ and $d_j \in R_{A_j}$ otherwise, $\text{fwd}_F\big(\overline{d}\big) \in R_B$, and, if $B = \&T$, $\text{back}_F\big(\overline{d}\big)$ is injective: equal tuples of final contents come only from equal final values of the returned borrow.

R is forced: by [Call-type], Ochr proves of every abstract `g : Π(x : &Nat). &Nat` that writing `0` or `1` through `g(x)` leaves different states, which is false of a Rust function that returns a leaked `'static` borrow (Section 10).

Terms. Abstract values become variables, and a loan ℓ becomes a variable h_ℓ, a *hole* for the final content of its borrow. The *view* $\Omega^\dagger$ reads a borrow as its current content and a loan as its hole; the *resolution* ρ_Ω substitutes each hole by its borrow's content, that is, it ends every borrow. A term translates to a state-passing function on views (Figure 14); a sealed program denotes the result of running it from the empty view; a definition is a well-founded recursion on the entry value of its decreasing parameter; and `Id` is the equation between the tuples

of result and resolved footprint. A symbolic and a concrete run of one term can place their borrows differently (Section 7.2), so runs are related through their resolutions.

Four invariants of the machine make this well defined (Section A.5): each borrow is held once, every loan is bound by a held borrow or lies inside a sealed program (the only place a loan may occur twice), loans nest in borrows acyclically, and every value read, moved or passed is loan-free. That machine steps preserve them is property 1 of Figure 9, which is assumed, not proved.

The translation of terms follows the clauses of Figure 14.

$$
\begin{aligned}
\mathrm{End}_\ell: &\quad s \mapsto (s \setminus \mathrm{holder}(\ell))[h_\ell := s.\mathrm{holder}(\ell)] \\
\text{match } p\{...\}: &\quad s \mapsto \mathrm{natCase}\ (s.p)\ \left(t^\dagger_Z(s[p := 0])\right)\ \left(\lambda\sigma'.\ t^\dagger_S(s[p := \mathrm{succ}\ \sigma'])\right) \\
f(\overline{a}),\ B \text{ data}: &\quad s \mapsto \text{let } (r, \overline{\varphi}) = f^\dagger(\overline{u}, \overline{w}) \text{ in } \left(r, (s \setminus \overline{a})\left[h_{\ell_i} := \varphi_i\right]\right) \\
f(\overline{a}),\ B = \&T: &\quad s \mapsto \text{let } (c, \beta) = f^\dagger(\overline{u}, \overline{w}) \text{ in } \left(c, (s \setminus \overline{a})\left[h_{\ell_i} := \beta_i(h_k)\right]\right)
\end{aligned}
$$

Figure 14: Key clauses of the translation. $\overline{u}$ are the contents of the borrow arguments, whose loans are ℓ_i, and $\overline{w}$ the other arguments; k is fresh. Ending a borrow is a substitution, and a call is one application, with Aeneas's region abstraction as the substitution $h_{\ell_i} := \beta_i(h_k)$. An erased term leaves the view unchanged.

A.7 Notes on the definition

Condition	What goes wrong without it
The two paths agree (Section 6.3)	
Erasure by declared type, never by value or normal form	A local function into `U(n)`, where `U(Z)` computes to `Prop`, runs at the generic `n` but would be erased at `n = Z`, proving `False`; a variable read as a proof by its value `⋆` is one at an instance but not at the generic call.
Each erased term runs on its own private copy	Proof irrelevance identifies `λx. ⋆` with `λx. (*x := 7; ⋆)`; a proof block run inline keeps effects that the same block closed off discards.
Pattern variables are places; a block moves what an arm moves	`match *x { S p => p := Z }`, closed off, captures `*x` by copy, proving `False`; a borrow moved in only some arms stays usable after the block.
All owners observed	A borrow returned into one of two arguments leaves its hole in both; observing one proves `Eq(Nat, 0, 1)`.
Recursion on entry values; `f` only as a call head; no `f` without `by`	`*x := S(*x); match *x { S p => f(&p) }` recurses on the original value; passing `f` to a helper, or a `by`-less body, avoids the check. Each proves `False`.
[Close]'s row and `&` read from the declared type	A codomain that computes to `&Nat` is closed off as data at the generic call but returns a live borrow at an instance.
A stuck block's arms end its borrows ([Repack])	`match *v { MkV(n, x) => n := Zero }; ()`, with `x : Fin1(n)`, is closed off and its arm never reaches a whole-again point, so the direct path, which runs the arm, leaves `*v` with a field of the wrong type; with a proof field, every concrete call fails when the caller's borrow ends.
Generalisation records survive private copies but not their arm; names never reused	A generalisation made while forming a type is lost with its private copy and its name reused; one kept past its arm gives a program text the type it had in a sibling arm (Section 6.4). Each proves `False`.
Types carry what the machine needs (Section 6.4)	
Π-types record their class and whether they return a borrow	`H(x : &Nat) : P0 := (*x := S(Z); ⊤)`, with `P0 := Prop`, is passed at `RefPred(0)`, which computes to `Π(x : &Nat). Prop`; a run that reads a call's class from the function value erases the call at the generic call and runs it at the instance, proving `False`.
Sorts are syntactic; universes are not cumulative	`Π(x : &Nat). T(Z)`, with `T(n) : P(n)` and `P(n)` computing to `Prop`, is a proposition whose inhabitants a data function tells apart.
Functions compared by their observation	Comparing results alone identifies `λx. (*x := S(Z))` with `λx. ()`; without writing through a returned borrow, `λ(x, y). x` is identified with `λ(x, y). y`.
Stuck, never ill-typed (Section 6.5)	
Exclusive access; matches end loans in neutral heads	A live loan copied into a sealed program, or a hole generalised away, lets an accepted program write through an ended borrow.
`J` computes only on convertible endpoints	Under an absurd hypothesis a cast puts `5` at type `Bool`, or embeds the untyped λ-calculus; the checker fails or diverges on good programs.
A zero-arm match outside a proof is stuck	Re-running a sealed program under a refinement that falsifies a hypothesis yields `⋆` as data, and true theorems are rejected.
Conditions of the set model (Section 7.1)	
Borrows only of $\texttt{Type}_0$	`Π(x : &Type₀)(a : *x). *x : Type₀` makes $\texttt{Type}_0$ impredicative, embedding System U⁻.
A borrow-returning function type takes a borrow	`g : Π(n : Nat). &Nat` returns a borrow that no owner observes, so Ochr refutes a type that safe Rust inhabits with a leaked `'static` borrow.
Strict positivity	`inductive Bad := MkBad(f : Π(x : Bad). False)` gives a closed proof of `False` that is never run.
Inductive parameters have no borrow type	`inductive RefP (r : &Word)` would let `RefP(&x)`, and every value of it, record a borrow: a borrow inside a type and inside data.
Equation fields are over first-order data and do not mention the declared type	No closed proof of `False` is known; kept so that positivity stays syntactic (`SelfEq`, `Loop` are rejected) and no field equates functions ([Conv-fun]).
Subsingleton elimination	`IsL` (Section 6.6) has no set-theoretic meaning; if proofs kept their constructors, it would prove `False`.

Figure 15: The side conditions of Ochr, by the principle they apply, and the counterexamples that forced them.

Notes 1–11 explain the side conditions of Figure 15 that are finest-grained in this appendix: each gives a program that, without the condition, is a closed proof of a false equation (or, for note 8, makes normalisation diverge; note 3′s program also needs [Type-pos] off); by the disjointness rule of Section A.3, an equation between distinct

constructors, such as `Eq(Nat, Z, S(Z))`, is `False` itself. Notes 12–27 record choices where the definition could have gone either way.

1. **A function's erasure class is read from its codomain term.** A local `fix` is formed, and so checked by [Def], each time the term containing it is evaluated, and its codomain may depend on captured values. Deciding its class from the codomain's *value* would give different answers on the two paths:

   ```
   U(n : Nat) : Type₁ := match n { Z => Prop, S _ => Prop }
   V(n : Nat) : U(n) := match n { Z => ⊤, S _ => ⊤ }
   LieL(n : Nat) : Id(Nat, (let h = λ(x : &Nat) : U(n) => (*x := S(Z); V(n)); let c = Z;
   h(&c); c), S(Z)) := refl
   BoomL : Eq(Nat, Z, S(Z)) := LieL(Z)
   ```

   At the generic `n` the codomain is `⌈U(σ)⌉`, not a sort, so `h(&c)` runs and the statement is `⊤`. At `n = Z` the codomain is `U(Z) = Prop`, a sort, so `h(&c)` would be erased, `c` would stay `Z`, and `LieL(Z) : Eq(Nat, Z, S(Z))`. Read from the term, `U(n)` is not syntactically a sort and has declared sort `Type₀`, so `h(&c)` runs on both paths.
2. **A stuck block is erased only when it is a proof.** Treating the block as a call, whose codomain is the match's type, erases a block that computes a type, while the same match run directly is not a call and is not erased:

   ```
   LieB(n : Nat) : Id(Nat, (let c = Z; let T = match n { Z => (c := S(Z); ⊤), S _ => (c :=
   S(Z); ⊤) }; c), Z) := refl
   BoomB : Eq(Nat, S(Z), Z) := LieB(Z)
   ```

   A block, like any match, is erased exactly when it is a proof, which by [Type-pos] is when each of its arms is. An earlier version also needed a rule against erasing a block by its type inferred from the arms, which is computed; with syntactic sorts that type and the arms' declared type have the same sort, and the rule is subsumed. A block that is not erased is always safe, because its sealed programs re-run the arms, which decide for themselves.
3. **A proof is declared, not computed.** Whether a variable is a proof is read from its binding's declared type, never from its value ⋆. Before sorts were syntactic, a variable of a type such as `V(Z)`, with `V(n) : U(n)` and `U(Z)` computing to `Prop`, held ⋆ at an instance and a sealed program at the generic call, and reading proofs by their value gave a closed proof of `False`:

   ```
   LieH(g : Π(y : Nat). V(Z)) : Id(Nat, let c = Z; let h = g(0); (c := S(Z); h); c,
   S(Z)) := refl
   BoomH : Eq(Nat, Z, S(Z)) := LieH((λ(y : Nat) : V(Z) => refl))
   ```

   At the generic call `h` holds a sealed program, so `(c := S(Z); h)` runs and the statement is `⊤`; at the instance `g(0)` returns ⋆, a value-based reading erases the sequence, `c` stays `Z`, and the statement is `Eq(Nat, Z, S(Z))`. [Type-pos] now rejects `V(Z)` as a type, since its declared type `U(Z)` is not a sort, so declared and value-based readings agree on every program that checks.
4. **Inductive declarations are strictly positive.** A negative field gives a closed proof of `False` that is never run:

   ```
   inductive Bad : Type₀ := MkBad(f : Π(x : Bad). False)
   L(b : Bad) : False := match b { MkBad(f) => f(b) }
   Bad4 : False := L(MkBad(λ(x : Bad) : False => L(x)))
   ```

   `L(…)` is a proof call, so it is typed by [Call-type] alone and its diverging body never runs. [Ind-decl]'s first-order fields exclude `Bad`.
5. **Generalisation records survive private copies, and fresh names are never reused.** A match inside a type whose scrutinee is a sealed program is generalised on the type's private copy, and the formed type mentions the new σ:

```
inductive Box := MkBox(x : Nat)
Double(n : Nat) : Nat by n := match n { Z => Z, S p => S(S(Double(p))) }
Esc(n : Nat, m : Box) : Id(Nat, (let b = Double(n); match b { Z => 0, S _ => 1 }),
(match m { MkBox(x) => match x { Z => 0, S _ => 1 } })) :=
  match m { MkBox(x) => match x { Z => refl, S _ => refl } }
Bad5 : Eq(Nat, 1, 0) := Esc(1, MkBox(0))
```

If the record `⌈Double(σ)⌉ := σ_g` were discarded with the copy, and σ_g's name reissued to the field `x` when the body splits `m`, the goal would equate the two and `Esc` would check; its instance is `Eq(Nat, 1, 0)`.

6. **A borrow-typed result is observed through a written value** ([Obs-borrow]). Ending a returned borrow with its current content forgets where it points, so `PickX(x, y : &Nat) : &Nat := x` and `PickY(x, y : &Nat) : &Nat := y` would be convertible; transport from `Π(x y : &Nat). Id(Unit, let r = h(x, y); *r := S(Z), *x := S(Z))`, true of `PickX`, then proves it of `PickY`, whose instance at two zeros is `Eq(Nat, 0, 1) ∧ Eq(Nat, 1, 0)`, which is `False`.
7. **A borrow-returning function type needs a borrow parameter** ([T-Pi], [Def]). Closing off a call `g(5)` with `g : Π(n : Nat). &Nat` would return a borrow whose hole lies in no owner, so writes through it are unobservable. The function `PF(x : &Nat, e : Id(Unit, *x := 0, *x := 1)) : False := e` checks, since at its generic call the statement observes the owner of `x` and computes to `Eq(Nat, 0, 1)`, which is `False`. Then `QF(g : Π(n : Nat). &Nat) : False := PF(g(5), refl)` checks too, because at this call site the statement observes nothing and computes to `⊤`: it proves that `Π(n : Nat). &Nat` is empty, although safe Rust inhabits it with a leaked `'static` borrow, and any such opaque inhabitant gives a closed proof of `False`.
8. **The head guard covers neutral-headed calls** ([App-neutral-head]). Closing off a sealed program's head call when its head is an abstract function with codomain `&T` and two or more borrow arguments creates a new hole; reading an owner ends it, re-normalises a sealed program of the same shape, and so on for ever.
9. **Confinement is a fail-safe.** For a correctly classified term the private copy already discards its effects. If the two paths ever disagreed about whether a term is erased, the path that erases a term with outside effects rejects it rather than silently discarding effects the other path keeps. A place *outlives* an erased term when its root is a position of the environment the term starts from; the exception for erased calls covers exactly the borrows and moves that evaluate their arguments.
10. **Subsingleton elimination** (Section A.4.6). A match on a proof of an inductive with two constructors cannot produce data. With `inductive Bool := false | true`:

```
inductive Or (P : Prop) (Q : Prop) : Prop := Inl(p : P) | Inr(q : Q)
IsL(h : Or(True, True)) : Bool := match h { Inl(p) => true, Inr(q) => false }
Irr(h : Or(True, True), k : Or(True, True)) : Eq(Bool, IsL(h), IsL(k)) := refl
Boom : False := Irr(Inl(refl), Inr(refl))
```

Without the condition, `IsL` and `Irr` check: at the generic call `h` and `k` are both ⋆, so both sides are the stuck `⌈IsL(⋆)⌉`. `Boom` is still rejected, because `Inl(refl)` is a proof, whose value is ⋆, so `Irr`'s instance is `Eq(Bool, ⌈IsL(⋆)⌉, ⌈IsL(⋆)⌉)`, which is `True`. Only if a proof also kept its constructor would `Boom` prove `False`, as it would in a system where proofs keep their constructors. The condition is required all the same: `IsL` has no meaning in the set model once proofs are irrelevant (`Eq(Bool, IsL(Inl(refl)), IsL(Inr(refl)))`, provable by `refl`, reads `Eq(Bool, true, false)` there); `IsL(Inl(refl))` is a closed `Bool` that is stuck for ever, against canonicity and adequacy; and a two-constructor match on a proof, which cannot take an arm, has a value only because every arm is a proof. For `False`, `True` and `And` the type determines the arm and the fields are proofs, so nothing is chosen.

11. **Borrows are only of `Type₀`, and `&` only at the top of a declared type** ([T-Ref], [T-Borrow]). If `&A` were allowed for every type `A`, a borrow of a type would quantify over `Type₀` inside `Type₀`:

```
Impred : Type₀ := Π(x : &Type₀) (a : *x). *x
PolyId(x : &Type₀, a : *x) : *x := a
SelfApp(u : Unit) : Impred := let T = Impred; PolyId(&T, PolyId)
```

`Π(X : Type₀)(a : X). X` lives in `Type₁`, but through the borrow it would live in `Type₀`, which becomes impredicative: with the impredicative `Prop` below it, the rules contain Girard's System U⁻, in which Hurkens' paradox is a closed term, and no set model exists. And if a codomain could compute to a borrow type, as in `F(n : Nat, x : &Nat) : (match n { Z => &Nat, S _ => Nat })`, [Close] would read the data row at the generic call, where the match is stuck, while `F(0, &a)` returns a live borrow at the instance; `G(n, a) := let r = F(n, &a); let b = a; let r2 = r; ()` is accepted at the generic call and, at `n = 0`, reads `r` after reading `a` has ended it. A type variable is borrowable exactly when it is declared in `Type₀`: `SwapT(A : Type₀, x : &A, y : &A)` is well formed, and `SwapT1(A : Type₁, x : &A, y : &A)` is not, since there `A` could be `Type₀` itself.

12. **Proofs are not run.** A proof call's arguments are evaluated and its [Call-type] and [Rec] checked, on a private copy; its body is never run ([T-Call-proof]), and the machine skips proofs altogether ([Erase-proof]). Running and skipping agree, since erased terms run on a private copy.
13. **The order of [Access].** Loans are ended from the root of the place outward, then left to right inside its content. The resolution does not depend on the order (property 4 of Figure 9, assumed, not proved).
14. **A match on a constructor checks only the arm taken**, in typing as in the machine ([T-Match]).
15. **[Close]'s row is chosen by the declared codomain** (`&T`, or anything else), as Conjecture 2 (4) requires. An earlier version had a third row, returning `()` for a codomain written `Unit`; a codomain that computed to `Unit` without being written so then got the data row, and a statement such as `Id(Unit, let c = *x; G(&c, Z), ())`, where `G`'s codomain `UU(n)` has `UU(Z) = Unit`, needed induction on `*x`. With η for `Unit` the row is unnecessary, and that statement holds by `refl`.
16. **A discarded value** (`t; u`) holding a live loan is an error, like a dying owned binding.
17. **Stuck-block moves.** A whole variable is moved into a block if some arm, when checked, leaves it $\bot$, or if it is a borrow variable read or assigned as a whole. Captures are ordered by the position of their root, then by length, and their arguments are evaluated in that order.
18. **Footprint.** A place rooted at a borrow variable contributes its owners even when it is only read, and a variable holding $\bot$ contributes nothing. Owners are positions, so a temporary can be an owner, when a type is formed while arguments are in flight.
19. `J.` `A` is a type position, `a`, `b` and `P` are erased positions, and `h` is a proof, so none of them runs in the machine; `J`'s value is `t`'s when `a ≡ b`, and it is stuck otherwise. Reducing regardless of the endpoints would be equality reflection for open terms: under a hypothesis equating `Nat → Nat` with `(Nat → Nat) → Nat`, casts give an untyped λ-calculus inside the checker, and a cast of `5` to `Bool` puts a number where a boolean is expected. The motive may have any sort. `J` is a proof when its motive's codomain term is syntactically `Prop`.
20. **Universes.** `&A` lives in `Type₀`, an inductive type in its declared sort (so `A × B` in `Type₀`, its parameters being in `Type₀`), and `Π` is impredicative in `Prop`. `And`'s parameters have type `Prop`, so [T-Ind] requires both conjuncts to be propositions.
21. **Recursion** requires the declared decreasing parameter to have an inductive type declared in `Type₀`, or to be a borrow of one (so recursion is never on a proof); strict subterms range over all fields.
22. **Conversion** is the least relation of Section A.3. Π-types are compared by instantiating their binders at generic arguments ([Conv-pi]); comparing their captured values and code instead would reject a closure that captures a variable where a closed Π-type is expected, and would not unfold type abbreviations under binders. There is no η rule; values at `Unit` are η-normal (Section A.3).
23. **An unannotated non-tail match** needs its arms' types, each computed in its own refined state, to be convertible; an annotated one checks each arm against the annotation refined for that arm.
24. **Syntax.** `Nat` and `Unit` are presented as declared types. The implementation builds them in with the same behaviour, declares `Pair`, `False`, `True` and `And` in its library exactly as in Figure 3, and checks those

declarations like any other ([Ind-decl]); it also has conveniences outside the core (`cong`, `trans`, `symm`, the ascription `(t : A)`, `λ` and `→`).

25. **A match on a proof is decided by the scrutinee's stored type** (Section A.4.6), the type kept with its binding and refined by [Split], not by its declared type term. In `SubM` (Section 2), `h` is declared `Le(y, *x)`, and its stored type becomes `False` after the two splits, so `match h {}` checks there. Requiring the declared term itself to be an inductive would also be stable, and would make `SubM` write `let e : False = h; match e {}`. This consults a normal form, but only in a way that fails safe: a split only refines the stored type, which keeps an inductive head, so a less refined path sees a neutral and rejects, and never sees a different head. A match with no arms in a proof position is a declared proof. Anywhere else it is stuck when reached, which happens only while a sealed program is re-normalised under a refinement that makes a hypothesis false; it then stays a neutral instead of producing a value of the wrong type.
26. **Injectivity and disjointness.** eq takes two values built by the same constructor apart into the equations between their fields, and sends two distinct constructors to `False`, so `Eq` computes on constructor values by structure; pairs have no rule of their own. Both are sound in the model, where constructors are injective and have disjoint images; each is an instance of `propext`.
27. **Captured values keep their declared types.** A closure or Π-closure records, with each captured value, the declared type and proof flag of the binding it was read from. So a `λ` or `Π` whose body reads a captured proof, or captured neutral data such as a sealed program, can be typed: `let n = clone(*x)` after `AddM(&*x, 1)`, followed by `λ(y : Nat) : Nat => clone(n)`, is accepted. Whether a captured variable is a proof is read from its declaration, never from its value ⋆ (note 3).

A.8 Soundness of a typed fragment, conditional on naturality and the machine's invariants

This section reduces soundness of a fragment F to an assumption that the two evaluation paths simulate each other. F has natural numbers, `Unit`, the propositions `True`, `False` and `And`, `Eq` and `Id`, first-order top-level functions with borrow parameters (some of which return a borrow), [Close], and [Split] with refinement and generalisation. For F we prove that, if the assumption holds, two things are true of every accepted program:

- every data function runs without error at every concrete input;
- every lemma is true at every concrete input at which its hypotheses are true.

Consistency of F and adequacy of `Id` follow (Corollary 20, Corollary 21).

The assumption, naturality (Assumption 10), says one step at a time that a symbolic run and the ground run agree once every borrow is ended. It carries most of the operational content. It holds only because of the [Drop] rule's treatment of lent places (Section A.8.7). A second assumption (Assumption 11) collects four invariants of the machine that the proof uses but does not prove: preservation of well-formedness, the frame property, order-independence of [End], and injectivity of contexts. Everything else is proved here, and each assumption is named where it is used (Section A.8.3).

A.8.1 The fragment F

F is the calculus of the appendix restricted as follows. Each restriction says why; Section A.8.8 lists where each is used.

1. **Data.** The only inductive declarations are the library's `Nat`, `Unit`, `False`, `True` and `And`. Data types are `Nat` and `Unit`; borrow types are `&Nat` and `&Unit`. There are no pairs and no user inductives. *Why:* ground values are then numerals and `()`.
2. **First order, with bodies.**
   - Programs are sequences of top-level definitions with bodies, $\mathbf{def}\ f(x_1 : A_1 ... x_n : A_n) : B\ \left[\mathbf{by}\ x_j\right] := b$, where $n \geq 0$ and $n = 0$ is a constant. There are no opaque definitions, since an opaque lemma is an axiom.
   - Each A_i is a data type, a borrow type or a proposition. `B` is a data type, a borrow type (and then some A_i is one) or a proposition.

- There are no `fix` or `λ` inside bodies, no Π-types as values, no function parameters (so no abstract functions), no universes other than as the sorts of these types, no parameters of sort `Prop`, no type families and no Prop-valued functions.
- A definition mentions itself only as the head of a call in its body, never in a type: not in its parameter types or codomain, and not in an annotation, `Eq` or `Id` inside its body.
- A definition is a *data function* if `B` is data or a borrow type, and a *lemma* if `B` is a proposition.

Why: conversion on data is then syntactic (Lemma 12), the type of every sealed program is a closed data type (Lemma 13), and every call inside a type is to an earlier definition.

3. **Propositions.** $P ::= \mathsf{True} \mid \mathsf{False} \mid P \wedge Q \mid \mathsf{Eq}\ D\,a\,b \mid \mathsf{Id}\ D\,t\,u$, with $D \in \{\mathsf{Nat}, \mathsf{Unit}\}$. The terms `a`, `b`, `t`, `u` inside a type contain no `match`. *Why:* a match inside a type that gets stuck is closed off as a stuck block, which F leaves out. It also means the typed run of a statement reaches every place occurrence in it (Lemma 14).
4. **No stuck blocks.** A `match` on data occurs only in tail position in a definition's body: the body itself, an arm of a match in tail position, or the tail of a `let` or sequence in tail position. It never occurs on the right of a `let`. So [Split] is only [Tail-split] and [Tail-gen]. No stuck block is ever formed, and [Split-gen] and [T-Split-goal] never run.
5. **Proof forms.** Proof terms are:
   - `refl`, $\langle h, k \rangle$, proof variables and their field places, and lemma calls;
   - a `let` or sequence whose tail is a proof;
   - a match on a proof of `And` or `True` (one arm);
   - `match h {}` on a proof of `False`, which occurs only in lemma bodies.

   There is no `J`, `rewrite` or `split`. *Why:* a data function's run then never depends on the truth of its hypotheses (Theorem 19, part (i)).
6. **No move out through a borrow.** In code that runs, a read of a place under `*` (such as `*x`, or `(*x).0` of a number) is written `clone(…)`. Reading a borrow variable itself, which moves the borrow, is allowed. Types and proofs read as code does (D68), so the restriction covers them too. *Why:* no borrow ever holds a moved-out `⊥` (Lemma 8). Resolution is then defined at every ground state the proof visits, and every call's borrow arguments are whole. It excludes the `mem::replace` pattern `let v = *x; *x := S(v)`, which must be written with `clone(*x)`.

Moves are otherwise as in the rules (D53): `Nat` is not a copy type, so a read of an owned `Nat` place moves it and leaves `⊥`, in types and proofs as in code.

The proof uses three properties of the rules, all stated in the appendix:

- **[Drop] of a lent place** (Section A.1.4). When a dying owned value holds a live loan, a borrower *held in a binding* (a variable, or a place inside one) is ended, as a write through [Access] would end it. A borrower that is a *value in flight* (a call argument, the value being assigned, a let-block's or arm's result, or a call's result while its frame pops) makes the drop an error. So a local may die while a variable that borrowed it is never used again, but a block or a function may not return a borrow of its own local.
- **Moved-out places** (Section A.2). A match on a place that holds `⊥` is an error ([Match-err]).
- **[Seal]'s final read** (Section A.2.7). The final read of a sealed program copies, so a sealed program never moves out through a borrow.

A.8.2 Ground instances, truth, resolution and agreement

Definition 6 (ground, valuation). A value is *ground* if it contains no abstract value, no sealed program and no inert loan; a ground value of type `Nat` is a numeral and one of type `Unit` is `()`. A state is ground if its values are; it may hold live borrows and loans. A *valuation* α assigns to each abstract value σ in its domain a ground value of type $\Delta(\sigma)$. For a state Ω whose abstract values α covers, $\Omega\alpha$ is $\Omega[\alpha(\sigma)/\sigma]^{+}$, substitution followed by normalisation; it is *defined* when every normalisation it triggers ends without error. α *respects* the records of the state if $\alpha(\sigma) = r\alpha$ for every refinement $\sigma := r$ and $\alpha(\sigma) = \mathrm{nf}(n\alpha)$ for every generalisation record $n := \sigma$.

The *generic state* of a definition `d` is Γ; φ, its generic call's environment. Borrow parameters are bound to $\text{borrow}_{\ell_i}\sigma_i$, with owned cells $c_i \mapsto \text{loan}_{\ell_i}$. Data parameters are bound to fresh σ_i, and proof parameters to $\star$ with their types stored.

- A *ground instance* of `d` is a valuation β of $\sigma_1, ..., \sigma_n$; its state is $\Gamma\beta$; $\varphi\beta$.
- Its *hypotheses* are the proof parameters' types, evaluated at the ground instance left to right.
- Its *goal* is `B` evaluated there, which is the [Call-type] of the call $d(\overline{a}\beta)$ from a caller whose frame holds the cells.
- A call made at a ground state is *at* the ground instance whose values are its arguments: a data argument's numeral, and the content of a borrow argument, which is a numeral by Lemma 8.

A run is in *runtime mode* when it is code, and in *erased mode* when it runs a statement: a type, a proof, a side of `Eq` or `Id`, or an argument of a call that returns a type or a proof. A statement runs on its own private copy of the state, which is discarded at its end, and its reads move exactly as code's do (D68).

Definition 7 (truth). At a ground state, `eq` on two numerals computes to `True`, `False` or an `And` of such, and `eq` on `Unit` to `True`, unless $\perp$ occurs in either side, when the equation is stuck ([Eq-stuck]). So a proposition value computed at a ground state is built from `True`, `False`, `And` and stuck equations. *Truth* $\vDash$ is defined on these by $\vDash$ `True`, $\neg \vDash$ `False`, $\vDash P \wedge Q$ iff $\vDash P$ and $\vDash Q$, and $\vDash E$ for every stuck equation `E`.

A stuck equation asserts nothing, so it counts as true. That is safe for what follows. No proof form of F eliminates a stuck equation (it is neither `True`, `False` nor an `And`), so its truth is never used to derive another proposition; a proof can only pass it on. `False` is not stuck, so Corollary 20 is unaffected, and Corollary 21 says nothing about a place whose equation is stuck. A stuck equation arises at a ground state only from an `Id` whose side leaves an observed place moved out.

Resolution. For a well-formed state Ω, $\rho(\Omega)$ ends every borrow, and $\rho_\Omega(w)$ is the value `w` with each live loan replaced by the resolved content of its borrow, which is well defined by acyclicity.

Lemma 8 (resolution). In F no borrow ever holds a moved-out $\perp$. At every well-formed ground state of F, ϱ is defined, and it does not depend on the order in which borrows are ended. Ending some borrows first does not change it.

Proof.

- **No borrow holds a moved-out $\perp$.** A read of non-copy data leaves $\perp$ in the place read. By F6 no such read is under *, and a sealed program's final read copies. So such a $\perp$ appears only at an owned position or inside one. [Borrow] refuses a place that is not whole. So a borrow's content never holds one, and [End]'s wholeness premise always holds.
- **Order independence.** This is Assumption 11 (T3), whose premise the first bullet supplies: resolution exists in a well-formed state, every order of the held borrows reaches it, and ending some borrows first does not change it.
- **What this needs.** Well-formedness of F's states is Assumption 11 (T1).
- **Why only ground states.** At a symbolic state, ending a borrow re-normalises the sealed programs it substitutes into, which this argument does not cover. Every use of ϱ at a symbolic state goes through (N3) of Assumption 10.

□

Definition 9 (agreement). Let Ω_s be a state on the symbolic path, α a valuation that covers and respects it, and Ω_g a ground state with the same frames, bindings and temporaries. $\Omega_s \approx_\alpha \Omega_g$ holds when: (A1) $\Omega_s\alpha$ is defined; (A2) $\rho(\Omega_s\alpha) = \rho(\Omega_g)$ up to a renaming of loans; (A3) every position that holds a borrow in Ω_s holds a borrow in Ω_g; (A4) at every position that holds borrow_ℓ in Ω_s and $\text{borrow}_{\ell'}$ in Ω_g, $\text{owners}_{\Omega_g}(\ell') \subseteq \text{owners}_{\Omega_s}(\ell)$; (A5) at every position that holds $\text{borrow}_\ell w_s$ in Ω_s and $\text{borrow}_{\ell'} w_g$ in Ω_g, $\rho_{\Omega_s\alpha}(w_s\alpha) = \rho_{\Omega_g}(w_g)$: each live borrow holds the same value once the loans inside it are resolved.

The clauses do the following:

- (A2) is equivalent to "some sequences of [End]s take $\Omega_s\alpha$ and Ω_g to one state" (Lemma 8). The stronger form, equality before resolution, fails: in a counterexample, ending a borrow early changes which borrows a later

assignment ends, although both states agree after resolution. (A2) survives an [End] on either side, which is how the symbolic path may end a borrow earlier than the ground path (the `Pick` example of Conjecture 3).
- (A3) says the symbolic path has ended at least the borrows the ground path has.
- (A4) says its owners over-approximate: a returned borrow's hole sits in the fill of every place it may point into.
- (A5) fixes what each live borrow holds. Without it the relation would admit pairs that resolve alike but in which a borrow points at different parts of its target. For example, symbolically `p` borrows the predecessor field of `x`'s target, and on the ground it borrows the whole target. `match *p` would then take different arms, and a decreasing argument behind `p` would have different sizes.

The consequence the proof uses is this. At agreeing states, a place that is readable on both paths has the same resolved content: by (A2) at an owned position, and by (A5) through a borrow. In particular, after [Access], a scrutinee has the same head constructor on both paths. A loan-free value in flight, such as a call's data argument, is the symbolic value under α. And the content of a borrow argument, which is loan-free (well-formedness condition 5) and whole (Lemma 8), is the symbolic content under α.

A.8.3 What the proof assumes

Assumption 10 (naturality, step by step). Let $\Omega_s \approx_\alpha \Omega_g$, and consider one step of a rule of F, in runtime mode or in erased mode, that succeeds from Ω_s on the symbolic path (the typing judgement, in which stuck calls close off). (N1) *Primitive steps.* For [Access] (acc^R and acc^M), [Read], [Copy], [Move], [Borrow], [Assign], [Clone], [Ctor], binding a `let` and dropping its binding, dropping a discarded value, pushing a frame of agreeing values, and popping a frame: the same step succeeds from Ω_g in the same mode, and the resulting states, with the step's value as a temporary, agree. (N2) *Calls that close off.* At agreeing call points, whose arguments are temporaries of both states, suppose the symbolic call closes off by [Close] and the ground call runs to completion without error. Then the resulting states and results agree. (N3) *Resolution commutes with valuation.* $\rho(\Omega_s)\alpha = \rho(\Omega_s\alpha)$ up to a renaming of loans, including through [Seal]'s re-normalisation of a fill whose hole is inert.

This is naturality in relational form. It is stated per step, so that the proof can use agreement at every call point and every pushed frame.

- A call whose symbolic run returns by [App] ran its body without inspecting an abstract value, so its steps are (N1) steps in the callee's frame. Only a call that closes off needs (N2).
- (N2) assumes only that the ground call it relates is safe. The proof supplies that call by call (Lemma 17), at the arguments the ground run actually passes. By F6 every such call is at a ground instance.
- (N1) and (N2) cover erased mode as well as runtime mode.

The risks we know of are:

- agreement after a [Drop], and the preservation of (A3): early ends are deep, so no reborrow outlives its early-ended borrow on the symbolic path (Section A.8.7);
- the preservation of (A5) by [Close]. Under α, a closed-off call's fills place the hole where the ground call's returned borrow points (property 3 of Figure 9);
- (N3) through [Seal] with inert loans. The lemma behind it is *hole parametricity*: a fill's run writes its hole and reads its cells, but never inspects the hole.

A proof of Assumption 10 would go rule by rule; Section A.8.8 lists what it needs.

Assumption 11 (invariants of the machine). (T1) *Well-formedness.* The rules of F preserve well-formedness conditions 1–5 (Section A.5; property 1 of Figure 9). (T2) *Frame.* The frame property holds for F: a run changes the environment below its own frame only through the loans held by that frame's borrows, and a call's effect is its isolated run, plugged back into the caller (property 2). (T3) *Order-independence.* In a well-formed state of F in which no borrow holds a moved-out $\perp$, resolution exists, ending the held borrows one at a time in any order reaches it, and ending some of them first does not change it (property 4). (T4) *Injectivity of contexts.* A context that holds a loan is injective in the loan's final value (property 6).

None of (T1)–(T4) is proved, on paper or in Lean. (T1)–(T3) are about runs of the machine and (T4) is about values. The table lists where each is used.

Assumption	Used for
(T1)	Lemma 8; values in flight are loan-free in Lemma 15; one owner per loan in Lemma 16
(T2)	Lemma 14; Lemma 16; the calls of Theorem 19
(T3)	Lemma 8
(T4)	Lemma 16

A.8.4 Lemmas

Lemma 12 (truth and conversion). (a) If $P \equiv Q$ are proposition values at a ground state, then $\vDash P$ iff $\vDash Q$. (b) In F, if $v \equiv w$ then $v\alpha \equiv w\alpha$ for every valuation α for which both are defined.

Proof.

- **Which rules apply.** In F the conversion rules that apply are [Conv-refl], [Conv-cong] and [Conv-unit]. F has no function values and no Π-closures (F2), and its only η is `eq`'s for `Unit`. On data values, [Conv-cong] compares sealed programs by their programs, component by component. So on data, $\equiv$ is equality up to the names of bound variables.
- **(a).** At a ground state a proposition is built from `True`, `False` and `And` (Definition 7). [Conv-cong] relates `And` to `And` componentwise, and [Conv-unit] removes `True` conjuncts. Both preserve truth, by induction on the derivation of $\equiv$.
- **(b).** A valuation substitutes and then normalises, rebuilding every `Eq` type with `eq`. It sends equal values to equal values, and the unit laws commute with substitution.

□

Lemma 13 (generalisation). Let a node of a [Def] case tree have state Ω_s. Let α cover and respect every abstract value of Ω_s except those introduced by the generalisation records made on the path to that node, and let $\Omega_s\alpha$ be defined once α is extended. Extend α by $\alpha(\sigma) := \mathrm{nf}(n\alpha)$ for each such record $n := \sigma$, in the order the records were made. Then: (a) the records visible at the node are exactly those made by [Tail-gen] on the path from the root to it; (b) $\alpha(\sigma)$ is a ground value of type $\Delta(\sigma)$; (c) $(\Omega_s[\sigma/n]^+)\alpha = \Omega_s\alpha$, so generalising is invisible under α.

Proof.

- **(a).** In F, [Split-gen] and [T-Split-goal] never run (F3, F4), so records come only from [Tail-gen], in tail position of the body being checked, and never on a private copy. A record belongs to the [Split] arm that made it and is dropped when the arm ends. A record made while checking an earlier definition is keyed by a text that mentions that check's abstract values or loan labels, and fresh names are never reused, so no state of this check can derive that text again. What remains is exactly the records made on this path.
- **(b).** Take a record $n := \sigma$ made at a node on the path. `n` occurs in that node's state, so its normal form under α exists by (A1) at that node, which the walk of Theorem 19 maintains. In F, `n` has one of the two row shapes of [Close], with a head call of a data function (F2, F4). Its normal form under α is a value of that function's declared codomain, or of the declared type of the cell a fill row reads. [Tail-gen] gives σ the type of the matched place, which in F is the same closed data type (F1, F2).
- **(c).** Every occurrence of `n` becomes σ, and every later re-derivation of `n` on the path normalises to $\rho^*(\sigma)$ ([Seal-stuck]). Under α both become $\mathrm{nf}(n\alpha)$, which is what `n` itself becomes, since normalisation is deterministic.

No record crosses a sibling arm, so there is no cross-arm consistency to show. The closed proof of `False` of Section 6.4 needed exactly such a crossing, through a type family. □

Lemma 14 (footprints). Let $\Omega_s \approx_\alpha \Omega_g$, and let `t` and `u` be statement terms of F whose typed runs from Ω_s succeed. Let W_s and W_g be the footprints $W(t,u)$ at Ω_s and at Ω_g, as sets of positions. Then: (a) every $\pi \in W_g \setminus W_s$ is owned only through borrow variables that hold $\bot$ at Ω_s, and `t` and `u` only assign those variables, as whole variables; (b) for each π in $W_s \setminus W_g$ or in $W_g \setminus W_s$, the erased-mode ground runs of `t` and of `u` from Ω_g leave the resolved content of π unchanged, or one of them moves π out, so that π's equation is stuck; (c) each observed place has the same type on both paths.

Proof. **The frame fact.** By Assumption 11 (T2), a run changes the environment below its own frame only through the loans held by that frame's borrows, and in its own frame only the places it writes or borrows. A read

changes only the owned place it moves out of, which then holds `⊥` (D68). By F6 no read moves out through a borrow, so a move is a read of an owned place by name, which both paths make alike. Ending a borrow, including the old borrow that an assignment to a borrow variable drops, does not change any resolved content. So, apart from a move, the resolved content of a position π can change only if `t` writes π or writes through a live borrow that π owns. Writing through a borrow takes a place under it written or borrowed, or the borrow variable read whole and passed on; a read through a borrow is a `clone` by F6 and writes nothing. So π is among the owners of the places `t` writes or borrows and of the borrow variables it reads whole, which is $W_g(t)$ by the definition of the footprint.

(a). Let $\pi \in W_g$ be owned by the root of an occurrence `p` in `t` or `u`.

- If `p` is rooted at an owned variable, it contributes that variable on both paths.
- If `p` is rooted at a borrow variable `x`, then `x` holds a borrow at Ω_g, since a variable holding $\perp$ contributes nothing. At Ω_s it holds a borrow or $\perp$; it never holds a loan, since `&&D` is not a type. If it holds a borrow, (A4) gives $\mathrm{own}_{\Omega_g}(x) \subseteq \mathrm{own}_{\Omega_s}(x)$, so $\pi \in W_s$.
- Suppose it holds $\perp$. The statement terms are match-free (F3), so the typed run of `t` reaches every occurrence in `t`, including those in erased positions, which the typing judgement runs on private copies ([T-Erase]).
  - Every occurrence rooted at a variable holding $\perp$ is an error except an assignment to the whole variable. A read of `x` or of a place under it is [Read-err]. A borrow is [Borrow-err]. A match is [Match-err]. A `clone` is excluded by [Clone]'s premise. An assignment through `x` fails because its content is undefined.
  - `x := t'` drops $\perp$, which is loan-free, and binds a new borrow. After it, `x` holds that same new borrow on both paths.
  - So `t` and `u` use `x` only by assigning it as a whole. The test `AssignBot` (the artifact's `Drops` block) is such a statement.

(b). By the frame fact, a position outside W_g is changed by neither ground run, except by a move. That covers $W_s \setminus W_g$.

- Let $\pi \in W_g \setminus W_s$. By (a), π is owned only through borrow variables that hold $\perp$ at Ω_s, which `t` and `u` only assign.
- On the ground, such an assignment drops the variable's old borrow, which ends it and changes no resolved content.
- If `t` wrote π through any other place `q`, the root of `q` would hold a live borrow at Ω_s, or be π itself. By (A4), π would then be in W_s.
- So neither run changes π, except by a move.

A run that moves π leaves `⊥` in π's content, so π's equation is stuck.

(c). Observed places have type `Nat` or `Unit`. A place's type is read from its stored type; a temporary's is read from its value's type, which is a numeral's, $\Delta(\sigma)$'s or a sealed program's declared codomain. Agreement relates positions of the same binding, and in F no refinement changes these types (F1, F2). □

The symbolic side's extra positions do occur, and their conjuncts do become `True`: a hypothesis formed at an abstract `n` through `Pick(n, &a, &b)` has type `False ∧ ⊤` in arm `Z`, and a proof can use its second conjunct as `True`.

Lemma 15 (types agree). Let $\Omega_s \approx_\alpha \Omega_g$ and let `A` be a proposition of F. Suppose every call made in erased mode by the ground evaluation of `A` runs to completion without error. If `A` evaluates to T_s at Ω_s (in a type position, on a private copy), then it evaluates to some T_g at Ω_g, and $\vDash T_s\alpha$ iff $\vDash T_g$.

Proof. By induction on `A`. The runs involved are in erased mode.

- **`True`, `False`.** $T_s = T_g$.
- **`P ∧ Q`.** [T-Ind] keeps the head `And`, and its parts are evaluated one after the other on one private copy. By Lemma 17 agreement holds between them. Use the induction hypothesis.
- **`Eq(D, a, b)`.** By Lemma 17 the ground runs of `a` and `b` succeed and their values agree. Values in flight are loan-free (T1), so $a_s\alpha = a_g$ and $b_s\alpha = b_g$. $T_s = \mathrm{eq}(D, a_s, b_s)$, and valuation rebuilds `Eq` types with `eq`. `eq` may already have dropped a `True` conjunct symbolically that valuation does not restore, so $T_s\alpha$ and $\mathrm{eq}\big(D, a_g, b_g\big) = T_g$ need not be equal as values. But they have the same truth, which is what the lemma claims.

- **`Id(D, t, u)`.** By Lemma 17 the ground runs of `t` and `u` succeed in agreement with the symbolic ones. The observation then ends every borrow. By (N3) and (A2), each symbolic observation under α equals the ground one after resolution, position by position: $r_s\alpha = r_g$, and $w_s(\pi)\alpha = w_g(\pi)$ for every position π; the same holds for `u`. So:
  - $T_s\alpha$ has the truth of the conjunction of $\mathrm{eq}\big(D, r_g, r'_g\big)$ and of $\mathrm{eq}\big(T_\pi, w_g(\pi), w'_g(\pi)\big)$ over $\pi \in W_s$;
  - T_g is the same conjunction over $\pi \in W_g$;
  - by Lemma 14 (b), each conjunct over a position in only one of W_s and W_g compares equal values, so it is `True`, or it is stuck, which also counts as true (Definition 7); by (c) the conjuncts' types agree;
  - the truth of a conjunction does not depend on the order of its conjuncts.

□

Lemma 16 (call types). Let `L` be a lemma of F with parameters $\overline{x} : \overline{A}$ and codomain `B`, called at a ground call point Ω_g with arguments $\overline{w}$. Let β be the ground instance the call is at. Then each parameter type, and `B`, are true at the call site ([Call-type]) iff they are true at the ground instance.

Proof.

- **Setup.** At the call site the types are evaluated in Ω_g; $(\overline{x} \mapsto \overline{w})$, and at the instance in $\Gamma\beta$; $\varphi\beta$. The two states differ only below the parameter frame. At the instance the borrow parameters' loans sit in the cells c_i; at the call site they sit in the caller's places.
- **Runs.** Apply (T2) with the parameter frame as the core; its ports are exactly the cells. Each run made while evaluating a type at the call site is then the run at the instance, with the ports' final values substituted into the caller's frames. So results, and the final contents of the parameter frame's own positions, coincide.
- **Borrow parameters.** An `Id` conjunct over a borrow parameter's owner compares, at the instance, the final contents of c_i. At the call site it compares the final contents of the owners of ℓ_i.
- **One owner.** At a ground state a loan occurs at most once: only sealed programs may hold a loan twice (well-formedness condition 2, by T1), and a ground state has none. So ℓ_i has one owner, reached through a chain of borrow contents, and its final content is $K[v]$ for the borrow's final value `v` and a context `K` that holds the hole once.
- **Injectivity.** By Assumption 11 (T4), `K` is injective, so $\mathrm{eq}(K[v], K[v'])$ is true iff $\mathrm{eq}(v, v')$ is.

□

Lemma 17 (runs agree). Let $\Omega_s \approx_\alpha \Omega_g$. Let `t` be a term of F that is not a match on data, and suppose $\Omega_s \vdash t \Downarrow v_s : A \dashv \Omega'_s$ on the symbolic path, in runtime or erased mode. Suppose also that every call that the ground run of `t` makes at a call site of `t` runs to completion without error, in that mode, at the ground instance it is at. Then:

- the ground run of `t` in that mode terminates without error, as $\langle \Omega_g, t\rangle \Downarrow \langle \Omega'_g, v_g\rangle$;
- the states agree at every call point of `t`, and there each ground argument is the symbolic argument under α (for a borrow argument, its content);
- the final states, with the values as temporaries, agree.

Proof. By induction on `t`, one step at a time.

- **Steps.** Each primitive step is (N1).
- **Calls.** At a call, the arguments are evaluated into temporaries by the induction hypothesis. The consequence of agreement stated after Definition 9 then gives the ground arguments. Loan-free data arguments equal the symbolic ones under α, and borrow arguments' contents are whole and equal the symbolic contents under α, so the ground call is at the ground instance the lemma names.
  - The ground call runs to completion by hypothesis.
  - If the symbolic call closes off, (N2) gives agreement after it.
  - If it returns by [App], its body's steps are (N1) steps in the callee's frame. The calls they make are parts of the ground call's run, which completes, so the same argument applies to them, by induction on the depth of the run. Agreement after the call follows.
- **Termination.** F has no loops, a match on data is never part of `t` (F4), and a match on a proof takes its one arm or is never run. So the ground run of `t` visits each subterm of `t` at most once. In addition it performs

finitely many [End]s at each [Access] and [Drop], since each replaces a borrow by $\perp$, and it makes the calls, which end by hypothesis. So it terminates.

What is assumed about a call is its safety at the arguments it actually receives. That is weaker than safety of every call at every argument list. □

Lemma 18 (modes). If a data function's runtime-mode run from a ground instance completes without error, so does its erased-mode run, with the same result.

Proof.

- **The modes agree on F.** Since D68 an erased read moves as a runtime read does. The one remaining difference, the Fn rule's check of a call through a place, concerns function values, which F does not have (F2). So the two runs take the same steps from the instance, and reach the same result.

□

A.8.5 The theorem

Theorem 19 (soundness of F, conditional on Assumption 10 and Assumption 11). Let $\mathcal{P} = d_1 ... d_m$ be a program of F whose definitions are accepted in order by [Def] and [Const]. Then for every d_k and every ground instance β of d_k: (i) if d_k is a data function, its body runs from the ground instance to completion without error, in runtime mode and in erased mode, and the pop of its frame succeeds; if d_k returns a borrow, the result is a live borrow, not $\perp$; (ii) if d_k is a lemma, or a constant of proposition type, and its hypotheses at β are true, then its goal at β is true.

Proof. **The induction.** By induction on the pairs $\big(k, |\beta(\sigma_j)|\big)$, ordered lexicographically. $|v|$ is the size of the numeral that is d_k's decreasing argument, read through the borrow for `&Nat`; it is 0 without `by`. Fix `k` and β, and assume (i) and (ii) for every smaller pair.

- A constant ($n = 0$) is checked by [Const] from the empty environment, without a case tree. A data constant's value is computed once, at checking time. For a proof constant, (ii) is the claim below with $\alpha = \emptyset$, as in Corollary 20.
- For (i), erased mode follows from runtime mode by Lemma 18. The rest of the proof is about runtime mode and about definitions with parameters, checked by [Def].

Which calls the walk needs, and why each is safe. Lemma 17 needs every call that the ground run makes at a call site of the body to run to completion at the ground instance it is at. There are three kinds.

- A call of d_j with $j < k$, since a definition is checked against the ones before it: safe by the induction hypothesis (i), in either mode.
- A recursive call of d_k at a call site on the walk's path. The symbolic check applied [Rec] there ([T-Call]'s premise), so the symbolic decreasing argument, or its content, is a strict subterm of $\rho^*(\sigma_j)$. By Lemma 17 the ground argument at that call point is the symbolic one under α. α respects ρ, so it is a strict subterm of $\alpha(\sigma_j) = \beta(\sigma_j)$, and the pair is smaller.
- A call made while evaluating a type. By F2 it is to an earlier definition, and it is in erased mode; safe by the induction hypothesis (i) and Lemma 18.

Proof calls are never run at ground ([Erase-proof]); only their types are evaluated, which is the third kind. The induction hypothesis (i) is about a run from the callee instance's own state. A call at a ground call point completes because, by Assumption 11 (T2), its run is that isolated run, plugged back into the caller.

The walk. We follow d_k's case tree $\mathcal{L}$, from [Def], along β. Throughout we keep five things:

- a node of the tree with its symbolic state Ω_s;
- a valuation $\alpha \supseteq \beta$ that covers and respects Ω_s;
- the ground state Ω_g that the ground run of the body has reached;
- agreement, $\Omega_s \approx_\alpha \Omega_g$;
- when d_k is a lemma, the invariant (I): every proof place of Ω_s (a binding declared a proof, or a field place of one) has a type T with $\vDash T\alpha$.

At the root, $\Omega_s = \Gamma$; φ^+, $\Omega_g = \Gamma\beta$; $\varphi^+\beta$ and $\alpha = \beta$. Agreement is immediate, since $\Omega_s\beta = \Omega_g$. (I) holds by Lemma 15, applied to each hypothesis in turn: its type at the generic state is true under β iff it is true at the ground instance, where it is true by assumption. The tail rules move down the tree:

- **[Tail-let], [Tail-seq].** `t` is not a match on data (F4). By Lemma 17 its ground run succeeds and agreement is kept; (N1) covers the drop. (I) is kept: a proof bound by the `let` has a true type by the claim below.
- **[Tail-match].** [Access] is (N1). By the consequence of agreement stated after Definition 9, the scrutinee then has the same head constructor on both paths, so both take the same arm.
- **[Tail-split] on σ.** After [Access], by the same consequence, the ground scrutinee's head is that of $\alpha(\sigma)$, say $\mathtt{C}_i$, so the ground run takes arm `i`. We move to the tree's arm `i` and extend α to the arm's fresh field values by the fields of $\alpha(\sigma)$. α respects $\sigma := r_i$ and $(\Omega_s[\sigma := r_i])\alpha = \Omega_s\alpha$, so agreement and (I) carry over.
- **[Tail-gen] on a sealed program `m`.** Extend α by Lemma 13 and continue as for [Tail-split].
- **[Tail-prop] (lemma bodies).**
  - With one arm (`And`, `True`), move to it. Its field places have the field types, which are true, since $\vDash (P \wedge Q)\alpha$ gives $\vDash P\alpha$ and $\vDash Q\alpha$; (I) is extended to them.
  - With no arms, the scrutinee's type is `False`, and (I) would give $\vDash$ `False`. So no ground instance reaches this node, and there is nothing to prove. This is where ex falso is sound.
- **[Tail-end] `t`.**
  - This is a leaf (Ω'_s, v, A). [Def] gives $\mathrm{pop}(\Omega'_s, v)$ succeeding and $A \equiv G'$, the goal refined along the path.
  - By Lemma 17 the ground run of `t` succeeds, and by (N1) so does the ground pop. The ground run of the body follows the walk's path one tail rule at a time. The path is finite, and each segment terminates (Lemma 17). So the body runs to completion without error.
  - **The result is live.** In F a term of borrow type never evaluates to $\bot$ on the symbolic path. Reading $\bot$ is [Read-err]. [Borrow] makes a live borrow. A call's result is live by [Close]'s borrow row, or, for a call that runs, by the same argument for the callee's tail term. There are no stuck blocks (F4), and a stuck block is the one way an arm could return a borrow ended by a drop and have [Close] revive it.
  - The pop does not end the result either. A dying local lent to the result, which is a value in flight during the pop, makes the pop an error rather than an end ([Drop] of a lent place). So the symbolic result is a live borrow, held as a temporary after the pop. By (A3) the ground result, at the same position, is a borrow too. This is (i).
  - For (ii), `v` is a proof, and the claim below gives $\vDash A\alpha$. By Lemma 12 (b), $A\alpha \equiv G'\alpha$.
  - $G'\alpha = G\beta$, because α respects the path's refinements and records (Lemma 13 (c)) and `G` mentions only generic values. By Lemma 12 (a), $\vDash G\beta$.
  - By Lemma 15 at the root, the goal at the ground instance is true. The calls made in evaluating it are to earlier definitions (F2).

Claim (proof terms have true types). Let $\Omega_s \approx_\alpha \Omega_g$ with (I), and $\Omega_s \vdash t \Downarrow \star : A \dashv \Omega'_s$ for a proof term `t` of F. Then $\vDash A\alpha$. By induction on the derivation:

- `refl` has type `True`.
- $\langle h, k\rangle$: [T-Ctor] types `h : P'` and `k : Q'` with $P' \equiv P$ and $Q' \equiv Q$. Use the induction hypothesis and Lemma 12.
- A proof place has its type, true by (I). A proof constant `c` has its declared type ([T-Const]), which is true by the theorem's induction hypothesis (ii), since `c` is earlier in the program.
- **A `let` or sequence whose tail is a proof** (F5). The proof runs on a private copy, in erased mode, and the machine never runs it. By Lemma 17, in erased mode with the calls justified above, the same steps run from Ω_g succeed and keep agreement. So there are ground states in agreement at every intermediate state, and the induction hypothesis applies to the proof subterms there.
- A match on a proof: as for [Tail-prop]. With no arms (under an annotation, [T-Match-erased]), the premise gives `h : False`, against (I), so the case is vacuous.
- **A lemma call $L_j(\overline{u})$ ([T-Call-proof]).**
  - The arguments evaluate at Ω_s to $\overline{w}_s$, ending at Ω_{s1}. By Lemma 17 the ground evaluation gives $\overline{w}_g$ at Ω_{g1}, in agreement, with each ground argument the symbolic one under α.

- The parameter types T_i and the call's type `B'` are evaluated after pushing the parameter frame, which is (N1). By Lemma 15 each has the truth of the corresponding type at the ground call site; the calls in them are to definitions before L_j (F2).
- A proof argument u_i has type $A'_i \equiv T_i$, which is true under α by the induction hypothesis and Lemma 12. So the ground call site's hypotheses are true, and by Lemma 16 so are those of the ground instance β_j of L_j the call is at.
- The pair $\big(j, |\beta_j|\big)$ is smaller than the current one: $j < k$, or $j = k$ and [Rec] ([T-Call-proof]'s premise) made the decreasing argument a strict subterm, as for data calls. So the theorem's induction hypothesis (ii) makes L_j's goal at β_j true.
- By Lemma 16, `B'` at the ground call site is true, and by Lemma 15, $\vDash B'\alpha$.

□

This theorem reduces soundness of F to Assumption 10 and Assumption 11; it is not a proof of soundness outright.

- **What is proved.** The case analysis at tail matches and [Rec]'s decrease under α. Termination at the level of calls. The truth bookkeeping, footprints, call types and generalisation.
- **What is assumed.** That each step of a symbolic run and the corresponding ground step agree, with agreement including what each live borrow holds (A5). And the four invariants of the machine (Assumption 11): preservation of well-formedness, the frame property, order-independence of [End] and injectivity of contexts.

A.8.6 Corollaries

Corollary 20 (consistency of F, conditional). Under Assumption 10 and Assumption 11, no closed term of F has type `False`. More generally:

- if a closed proof term `t` of F is typed from the empty environment, $\varepsilon \vdash t \Downarrow \star : A \dashv \Omega'$, after the definitions of an accepted program, then $\vDash A$;
- every accepted constant of proposition type is true.

In particular no closed term has type `Eq(Nat, Z, S(Z))`, which computes to `False`.

Proof. The empty state is ground, and $\varepsilon \approx_\emptyset \varepsilon$ holds with (I) vacuous. The claim in the proof of Theorem 19, with $\alpha = \emptyset$ and Theorem 19 itself supplying the lemma calls and the calls in types, gives $\vDash A$. `False` is not true, and truth is invariant under conversion (Lemma 12 (a)), so $A \not\equiv$ `False`. □

Corollary 21 (adequacy of `Id`, conditional). Under Assumption 10 and Assumption 11, suppose a lemma $L : \Pi\big(\overline{x} : \overline{A}\big)$. `Id` $D\,t\,u$ of F is accepted. Then at every ground instance whose hypotheses are true, the *observations* of `t` and `u` are equal: their runs on private copies, in erased mode, give the same result and the same final contents of every place either run writes or borrows, except a place that a run leaves moved out. That place's equation is stuck ([Eq-stuck]), and the corollary says nothing about it.

Proof. By Theorem 19 (ii), $\mathrm{and}(\mathrm{eq}(D, r, r'), \mathrm{eq}(T_\pi, w_\pi, w'_\pi), ...)$ is true. A conjunct that is not stuck is true only if its sides are equal: at a ground state `eq` on two numerals is `True` iff they are equal, and `Unit` has one value. The results are never stuck, since a value read is whole. By the definition of the footprint and Lemma 14, every place that `t` or `u` writes or borrows is observed. □

The corollary is about observations. Since D68 an observation reads as runtime code does, on its own copy of the state: `Id(Nat, let z = a; a, a)` is rejected, as `let z = a; a` is, since `a` is used twice. By Lemma 18, an accepted data function's runtime and erased runs from a ground instance agree, so for calls of accepted functions the distinction does not matter.

Remarks on stability. For a term of F at agreeing states, the decisions of Conjecture 2 are:

- (1) and (4), erasure, matches decided by a proof's type, and each call's class and [Close] row, are read from declared types, constructor names and declared codomains. F never changes those.
- (2): F forms no stuck blocks.
- (3): at a ground instance every match takes its arm, so no arm is checked.
- (5): the same types for observed places (Lemma 14 (c)).

- (6): footprints may differ in both directions, but every position in only one of them contributes a conjunct that is `True` (Lemma 14). The symbolic side's extras are the common case, from over-approximate owners. The ground side's extras come only from borrow variables that the symbolic path has already ended and that a statement reassigns (`AssignBot`).

These are remarks, not a corollary. At ground valuations most items hold trivially. The refinements that matter for [Split], such as $\sigma := \mathsf{S}\ \sigma'$, are not ground, and this section says nothing about them. Soundness uses Theorem 19, not stability.

A.8.7 [Drop] after an early end

The symbolic path can end a borrow earlier than the ground path, which (A2) allows. The [Drop] rule for a lent place is what keeps that from mattering. If a drop failed whenever the dying value held a live loan, this program would be accepted and go wrong:

```
Bad2(n : Nat, b : Nat, x : &Nat) : Unit := let a = 0; x := Pick(n, &a, &b); let z = b; ()
```

- **Symbolic path.** At the abstract `n`, `Pick` closes off, and its hole loan_k sits in the fills of both `a` and `b`. Reading `b` ends `k`, because [Access] ends every loan inside the content, sealed programs included. So `x` holds $\bot$, and `a` holds data when it goes out of scope. The [Drop] succeeds.
- **Ground path, `Bad2(0, 5, &y)`.** `Pick` returns a borrow of `a`, and `b` holds no loan. Reading `b` ends nothing, and `a` goes out of scope while `x` still borrows it. Under that rule, the drop would fail.

A match on a fill with a single owner does the same: the match ends every loan inside the fill, while on the ground the loan sits deeper and survives the match. The test `Bad4` of the `Drops` block has this shape.

The rule. When a dying owned value holds a live loan, [Drop] ends the borrower if the borrower is held in a binding, and fails if the borrower is a value in flight (Section A.8.1). `Bad2` and `Bad4` are then accepted and run without error. Returning a borrow of a local (`let a = 0; &a`, the test `RetLocal`) is still rejected, with no separate check: the result is a value in flight when the local is dropped.

Ending every borrower, including those in flight, would be unsound. A stuck block's arm could return a borrow of an arm-local, which the drop turns into $\bot$. [Split] discards the arm's value, and [Close]'s borrow row then gives the block a fresh live borrow. So the ground run would meet $\bot$ where the symbolic run holds a live borrow.

Why the [Drop] clause of (N1) holds. The argument is part of Assumption 10. A ground drop fails only if the dying place is lent to a value in flight. Take that temporary's symbolic counterpart, at the same position.

- If it is live, (A4) puts the dying place among its symbolic owners, so the symbolic drop fails too.
- If it is $\bot$, the symbolic path ended it early while it was in flight. In F, a borrow is in flight during a call's argument evaluation, during an assignment's [Access], during a let-block's final drops, during a pop, and, as the discarded value of `t; u`, during its own drop. Constructor fields are data. Type formers and observations run on private copies, a callee's parameters are bindings once its frame is pushed, and [Close] consumes its borrow arguments, so none of these adds a case.
  - A call argument ended in flight is [Call-err] on the symbolic path.
  - During drops and pops, only borrowers held in bindings are ended, so a temporary is never ended there.
  - An assignment's new value is never ended by the assignment's own [Access], on either path, so `x := Pick(n, &*x, &b); *x := 5` is accepted. That [Access] ends only the loans in the part of the place's content that the place owns. A place that receives a borrow has borrow type, and what it owns is its old borrow, not the data behind it. A place of data type receives loan-free data. While the new value is in flight, the only drop is of the old content, a borrow, so the drop ends it and never fails. After the assignment the value is a binding.
  - A discarded value of `t; u` that is a borrow is ended by its own drop, which never fails.

So when the symbolic drop succeeds, the ground drop succeeds. The borrowers the ground drop ends are those held in bindings whose owners include the dying place. By (A4) their symbolic counterparts are ended by the symbolic drop too, or were already $\bot$. So (A3) is kept, and (A2), (A4) and (A5) are unaffected, since ending borrows does not change the resolution.

Early ends are deep. An early symbolic end does not leave a reborrow alive that the ground path later ends. In F, an early end comes from a hole inside a fill: [Access] or [Drop] meets the loan inside a sealed program.

- The ended borrow's content, reborrows included, is substituted into that same fill.
- Hole parametricity keeps the fill stuck: its run never inspects the hole. So those loans are still inside a neutral at the head of the place being accessed.
- The same [Access] or [Drop] keeps ending loans until none remains there, so it ends the reborrows too.

A [Match]'s [Access] ends only loans on the path and loans inside a neutral at the head, not loans deeper in a value. An assignment's [Access] ends only the loans in the part of the place's content that the place owns, including loans inside neutrals in that part (L^A, Section A.1.4). A loan behind a borrow the place holds, even one inside a neutral, is left alone on both paths, and the drop of that borrow carries it back to its owner. In each case, an early end that the [Access] makes is made inside a neutral, so the argument goes through the fill staying stuck. So a program that reborrows through `Pick`'s result and then assigns `x := &c` is accepted: the reborrow's loan sits behind the borrow `x` held, and that borrow's drop carries it back.

Pairs would break the argument: a shallow read of one field would leave a reborrow in the other field alive. F has none (F1). The argument is part of Assumption 10, as the preservation of (A3).

The reborrow-and-replace idiom is in F, and it is accepted. `Trav(x : &Nat) : Unit := match *x { Z => (), S p => (x := &p; *x := 0) }`, the test `Trav` of the artifact's `Reborrows` block, satisfies F1–F6. The new borrow's loan sits behind the borrow that `x` held, which the assignment's [Access] leaves alone. The drop of the old borrow carries the loan back to its owner, where it stays live. So Theorem 19 covers it.

A.8.8 What would remove the assumptions

1. **Prove Assumption 10 for F.** This is a simulation, rule by rule, between the typing judgement's symbolic run and the ground machine, with agreement (A1)–(A5) as the relation. It uses order-independence at every [End], the equations of [Close] (property 3 of Figure 9) at every [Close], invariance of the machine under renaming of loans for (A2), and the frame property at every call. The risks are:
   - the preservation of (A3) (early ends are deep), (A4) and (A5);
   - agreement after a drop;
   - (N3) through [Seal] with inert loans.
2. **Prove Assumption 11** for the current rules, restricted to F.
3. **Widen F.** Each addition brings back a decision of Conjecture 2:
   - moves through borrows (F6): resolution over a moved-out `⊥`;
   - stuck blocks (F4): captures and modes;
   - closures and Π-values: class, captured types, [Conv-fun];
   - user inductives and pairs: disjointness and injectivity in `eq`;
   - type families: the types of generalised values;
   - `J` and `rewrite`: transport and stuck casts.

Where F's restrictions are used.

- F1: ground values (Definition 7); Lemma 13 (b); Lemma 14 (c); early ends are deep (Section A.8.7).
- F2: Lemma 12 (conversion on data is syntactic); Lemma 13 (b); Lemma 14 (c); calls in types are to earlier definitions (Theorem 19).
- F3: Lemma 14 (a) (every occurrence in a statement is reached); with F4, Lemma 13 (a) (records come only from [Tail-gen]).
- F4: the walk forms no stuck blocks; Lemma 17; the live result of Theorem 19 (i).
- F5: Theorem 19 (i) (a data function's run never depends on the truth of a hypothesis); the claim's `let` case.
- F6: Lemma 8; the whole borrow arguments of Lemma 17; Lemma 18.
- The [Drop] rule for lent places: the [Drop] clause of (N1), and the live result of Theorem 19 (i) (Section A.8.7).

A.9 The artifact

The checker runs inside Lean's editor. An Ochr program is written as `ochr B uses … { def … }` blocks in an ordinary Lean file. Each block is checked, natively, as soon as it is elaborated, whether in the editor as you type

or in `lake build`. A rejected definition is an error placed on the smallest piece of source responsible, with the checker's reason: an ill-typed argument, the read of a moved or ended borrow, the later argument whose access ended an earlier borrow, a `refl` whose two sides have different normal forms. A `reject def` (a negative test) is silent while it is rejected, and hovering it shows why. Hovering any term shows what the checker found there, on each path through the case splits, labelled by that path's refinements. For a term this is its value in the abstract environment Ω and its type, and for a borrow the place it borrows from. For a match's scrutinee it is its content before the split and in each arm. For a proof it is the goal it was checked against, as a normal form. For an argument it is the type expected of it, and the paper's rules applied there are listed by name. A hole, `?` or `sorry`, stands for a value of whatever type its context requires, so the rest of the definition is checked around it, and the build warns at the hole with a goal view like Lean's: the bindings in scope with their types and values, then $\vdash$ the goal. `lake exe tests` also runs every block programmatically and prints its table of verdicts.